

JAVA REVISION BOOK

Active-recall handbook: Java + DSA, Day 1 to Day 28

Personal learning book of

Shubham Rathod

*Built from your real **JAVA.zip**: every file read, old attempts and bugs kept honestly.*

● mastered ● familiar ● weak / needs recall ○ introduced

Understand → Build → Small verification → Move on

October 2026

Shubham Rathod

Contents

README - How to use this book	3
CURRENT STATE	5
ARCHIVE AUDIT - what is really inside JAVA.zip	6
INDEX	9
QUICK REFERENCE (open this before coding - 2 minutes)	11
DAY BY DAY (Day 1 to Day 29)	13
Core Java 1 - Foundations and Conditions	26
Core Java 2 - Loops, Patterns and Digit Programs	28
Core Java 3 - Methods	31
Core Java 4 - Arrays	33
Core Java 5 - Strings (a known retention weakness: revise often)	36
DSA 1 - Searching and Scanning Arrays	40
DSA 2 - Two Pointers, Scanner/Writer and a Sorting Preview	45
DSA 3 - Binary Search (Day 28) - special handling	48
PATTERN BANK (30 patterns you can rebuild)	51
ACTIVE RECALL SYSTEM	60
BLANK PAGE RECONSTRUCTION	63
MASTER CHECKLIST (status from Day 1 to Day 28)	65
COMMON MISTAKES / BUG BOOK (from your real code)	67
COMPLEXITY CHEAT SHEET (Big-O with the "why")	72
REPAIR LAB - Second distinct largest	74
Appendix A - Your code archive (as saved)	77

README - How to use this book

Owner: Shubham Rathod | **Built from:** JAVA.zip (Day 1 to Day 28, plus a day29.java found in the archive) | **Latest file save in the ZIP:** 30 Sep 2026

This is not a summary of your ZIP. It is a **revision system** built from what your files actually contain: the programs, the old attempts, the corrections, the comments and the order you learned things in.

The one idea behind the book

Note: Your main weakness (in your words)

You can understand a solution when you see it, but you sometimes struggle to build the logic from a blank file. So this book teaches **patterns you can rebuild**, not programs to memorise.

PROBLEM -> PATTERN -> VARIABLES -> PSEUDOCODE -> JAVA -> DRY RUN
(what?) (which?) (what do I need?) (plain words) (code) (prove it)

Mastery legend

Badge	Meaning	My evidence rule (applied to your files)
● MASTERED	Rebuilt independently or repeatedly shown	Correct code on 3 or more different days , no later bug
● FAMILIAR	Understood, independent rebuild not proven	Correct on 1 to 2 days, or the same-day repeats, or you confirmed it yourself
● WEAK	Repeatedly forgotten, buggy attempts, or you flagged it	Buggy attempt(s) in the ZIP and/or you said you forgot it
○ INTRODUCED	Appeared, not practised enough	Appears once with no revisit

Tip: Honesty rule

Nothing is marked ● only because code exists. When the evidence is unclear the book says ●. Old buggy attempts are kept on purpose: they show how you learn.

Two things you must know before reading

Mistake box: 1. The Day 28 test array in the ZIP is already sorted

You described the Day 28 test data as {10, 20, 42, 10, 49, 38} (unsorted). The day28.java and day28.class inside the ZIP both use {10, 20, 30, 40, 50, 60} with target 50, which is sorted and valid. The unsorted array is therefore **not in the archive** (maybe an earlier run). The book still flags it as a Binary Search trap and shows exactly what goes wrong with it. See 03-DSA/binary-search.md.

Note: 2. A day29.java exists

You told me the current day is Day 28. The ZIP also has day29.java (saved the same evening as Day 28) with find-all-index, last index and **bubble sort**. I documented it as Day 29 marked ○/●, but CURRENT_STATE.md keeps **Day 28** as your official position, exactly as you asked.

Folder map

Folder	What is inside
CURRENT_STATE.md	Where you are, what to do next session
ARCHIVE_AUDIT.md	Every file in the ZIP: final, old, duplicate, buggy, unfinished

Folder	What is inside
00-Quick-Reference/	1-page pre-coding cheat sheet
01-Days/	Day 1 to Day 29 history with mistakes and mastery
02-Core-Java/	Foundations, loops, methods, arrays, strings
03-DSA/	Searching, two pointers, binary search
04-Pattern-Bank/	30 rebuildable patterns
05-Revision/	Active recall, blank-page reconstruction, master checklist
06-Mistakes/	Bug book from your real code
07-Complexity/	Big-O cheat sheet with the "why"
08-Repair/	Repair lab for "second distinct largest"

Daily routine (small, not heavy)

1. **5-minute recall** (`ACTIVE_RECALL.md`). Do not reread the book.
2. Pick **one** weak pattern and do a blank-page attempt.
3. Check yourself with the dry run, then move on.

Tip: Learning rule (yours)

Understand -> Build -> Small verification -> Move on. If you are mentally tired, stop drilling the same thing. Come back tomorrow.

CURRENT STATE

Where you are

Item	Status
Current day	Day 28
Day 27	Closed
Day 28 - Binary Search	● understood
Binary Search core algorithm	Constructed (left / right / middle, move left or right)
Day 28 dry run	Not completed (session became mentally tiring)
Day 28 mastery	Not fully verified
String <code>alreadySeen</code> / <code>charFrequency</code>	Spaced-recall topic (keep revisiting)
day29.java in the ZIP	Found (find-all-index, last index, bubble sort). Treated as a preview, ○ introduced, not your official position

Next session plan

1. 5-10 min Binary Search recall (blank page, then dry run)
2. If it is retained -> continue forward (do NOT restart fundamentals)
3. If it is shaky -> one more dry run, then stop. Do not over-drill.
4. Keep 5 min for ONE spaced-recall item (see queue below)

Binary Search recall script (use this first)

1. Say out loud: "Binary Search needs a ___ array."
2. Write on blank paper: `left`, `right`, `middle` and what each means.
3. Write the three branches (`found` / `go right` / `go left`).
4. Dry run `{10,20,30,40,50,60,70}` for target `60`, then for target `25`.
5. Only then compare with `03-DSA/binary-search.md`.

Spaced-recall queue (one per day, not all at once)

Priority	Topic	Why it is here
1	Binary Search dry run	Day 28 not mastery-verified
2	Second distinct largest	You said you forgot it; Day 23 attempt was buggy
3	String <code>alreadySeen</code> / <code>charFrequency</code>	Your stated retention weakness
4	<code>firstLast</code> not-found case	Returns <code>{0,0}</code> today (bug)
5	Second largest start index	Day 13 / Day 23 latent bugs

Rules to keep

Tip: Learning rule

Understand -> Build -> Small verification -> Move on.

Do not force repeated same-day drilling when you are mentally exhausted.

- Do not restart Java fundamentals. The foundation (digits, loops, methods, arrays, basic strings) is solid.
- Do not mark Day 28 as mastered until you have written Binary Search from a blank page **and** dry-run it.
- Future topics (OOP, Collections, Exceptions, Generics, File Handling, Multithreading, SQL, JDBC, HTML/CSS, JavaScript, React, Spring, Spring Boot, REST, JPA/Hibernate) stay **FUTURE**. None of them appear in the ZIP.

ARCHIVE AUDIT - what is really inside JAVA.zip

93 files: 49 readable source/text files (+ 44 `.class` files, which add nothing new except one check: `day28.class` and `day29.class` confirm the array values). Your source files were **not modified**.

How the files were classified

Tag	Meaning
FINAL	Active (uncommented) code that works
HISTORY	Old attempts kept in <code>//</code> comments (kept as learning evidence)
BUG	Active or latest code with a real mistake
DUP	Duplicate of another file
UNFINISHED	Planned features missing
EMPTY / NON-JAVA	Nothing to learn from

Foundation files (no day number)

File dates show two blocks: **14 to 22 Jun 2026** and **25 Aug to 2 Sep 2026** (a break of about two months in between). Methods appear in the June files and are introduced again on Day 6.

File	Content	Status
<code>hello.java</code>	Read two ints, print sum	FINAL
<code>java0.java</code>	Even / odd with Scanner	FINAL
<code>conditional.java</code>	if / else-if ladder (button 1,2,3). Older tries: adult check, even/odd, compare a and b	FINAL + HISTORY
<code>switch1.java</code>	Calculator with <code>switch</code> on char	BUG: no <code>break</code> after <code>'*'</code> , and <code>/</code> by zero not guarded
<code>switch2.java</code>	Number to weekday with <code>switch</code>	FINAL
<code>project1.java</code>	Marks, total, percentage, grade	BUG: integer division in percentage
<code>loop.java</code>	for / while / do-while history; active: times table	FINAL + HISTORY
<code>loop1.java</code>	break, continue, star/number patterns, digit programs. Active: palindrome number	FINAL + HISTORY
<code>table.java</code> , <code>reverse.java</code>	Times table; **reverse.java prints odd numbers 1 to 19** (name does not match)	FINAL (misnamed)
<code>plus.java</code>	File name says "plus" but active code is factorial (sum is commented)	FINAL (misnamed)
<code>pluseven.java</code>	Sum of even numbers up to n	BUG: tests <code>num % 2</code> instead of <code>i % 2</code>
<code>fact.java</code>	Factorial	FINAL
<code>count.java</code> , <code>count1.java</code>	Digit count; <code>count.java</code> handles input 0, <code>count1.java</code> does not	FINAL / improved
<code>pattern.java</code> , <code>adpattern.java</code>	Star, number, hollow, Floyd, pyramid patterns	HISTORY (almost all commented) + FINAL pyramid
<code>ex.java</code> , <code>funmeth.java</code> , <code>exercise.java</code>	Methods: greater, factorial, average, power, GCD, Fibonacci, etc.	FINAL + HISTORY; exercise has small bugs (see Mistakes)
<code>Array.java</code>	First array program: input, then search	BUG: prints "not found" for every non-matching element
<code>for.java</code> , <code>Text File.txt</code>	0 bytes	EMPTY

File	Content	Status
01.py	Python turtle drawing	NON-JAVA (ignored)

Numbered day files

File	Active content	Status
day6.java	Digit programs + first methods	HISTORY, all work
day7.java	Return-value methods, digit methods	largest(a,b,c) starts at 0 (BUG for negatives)
day8.java	Recall + first arrays	largest = 0, smallest = 1000 (BUG sentinels)
day9.java	Big Saturday revision	findMax still starts at 0
day10.java	Array methods	FINAL: arr[0] start fixed the sentinel bug
day11.java	contains, index, count, reverse, average	FINAL
day12.java	Recall batch	FINAL
day13.java	Index, count, sum, reverse, second largest, duplicate	Fully commented; secondL has latent bug (loop starts at 0)
day14.java	Recall + first frequency / alreadySeen	FINAL (file re-saved 18 Sep, out of order)
day15.java	Recall + frequency + mostFrequent	FINAL
day16.java	Digit revision + Analyzer menu (numbers + marks)	FINAL; factorial of whole number overflows
day17.java	Same analyzer as Day 16	DUP; no day17.class, so it was probably never compiled
day18.java	Analyzer refactored into methods	FINAL; int factorial overflows after 12!
day19.java	String basics	FINAL
day20.java	String palindrome (buggy then fixed), countChar, charFrequency	Attempt 1 BUG, attempt 2 FINAL
day21.java	String Analyzer menu + Fibonacci	FINAL
day22.java	Student Analyzer (UNFINISHED) + find index / all indices / count	Also holds a stray day23 class fragment
day23.java	Smallest, second largest (distinct), sum, avg, second distinct (smallest-based), move zeros	secondLargest and secondDistinct have BUGs
day24.java	Move zeros, remove duplicates, sorted check, asc/desc/equal/neither	FINAL
day25.java	Remove duplicates, move zeros, second distinct largest (correct), most frequent	FINAL
day26.java	charFrequency on "banana"	Works; not clean (see Mistakes)
26.java	0 bytes	EMPTY
day27.java	Find index, find all index, first/last, firstLast	firstLast returns {0,0} if absent (BUG)
day28.java	Binary Search	FINAL (test array {10,20,30,40,50,60}, target 50)
day29.java	Find all index (linear), last index, bubble sort	FINAL (beyond Day 28)

Duplicated problems (same problem, many versions)

Problem	Appears in
Reverse an array	Day 11, 12, 13, 14, 15, 16/17
Second largest	Day 10, 12, 13, 14, 15, 16/17, 23
Frequency with alreadySeen	Day 14, 15, 20, 21, 26
Move zeros	Day 23, 24, 25
Remove duplicates (sorted)	Day 24, 25

Problem	Appears in
Find all indices	Day 22, 27, 29
Most frequent	Day 15, 25
Last index	Day 27, 29

Note: Reading rule

Repeating the same problem across days is how this book decides mastery. It is good evidence of practice, but identical code every time can also mean copying from memory. That is why the book keeps most items at ● until you rebuild them from a blank page.

INDEX

Start here	File
Where am I now?	CURRENT_STATE.md
What is in my ZIP, file by file?	ARCHIVE_AUDIT.md
2-minute cheat sheet	00-Quick-Reference/QUICK_REFERENCE.md
Day 1 to 29 history	01-Days/DAY_BY_DAY.md

Core Java

Topic	File
Foundations and conditions	02-Core-Java/foundations.md
Loops, patterns, digits	02-Core-Java/loops.md
Methods	02-Core-Java/methods.md
Arrays	02-Core-Java/arrays.md
Strings (alreadySeen, charFrequency)	02-Core-Java/strings.md

DSA

Topic	File
Searching and scanning	03-DSA/searching.md
Two pointers, scanner/writer, bubble sort	03-DSA/two-pointers.md
Binary Search	03-DSA/binary-search.md

Practice

Topic	File
30 patterns	04-Pattern-Bank/PATTERN_BANK.md
Active recall	05-Revision/ACTIVE_RECALL.md
Blank-page reconstruction	05-Revision/BLANK_PAGE_RECONSTRUCTION.md
Master checklist	05-Revision/MASTER_CHECKLIST.md
Bug book	06-Mistakes/COMMON_MISTAKES.md
Big-O cheat sheet	07-Complexity/COMPLEXITY_CHEATSHEET.md
Repair lab: second distinct largest	08-Repair/SECOND_DISTINCT_LARGEST.md

Find it fast

I forgot...	Open
second distinct largest	08-Repair/SECOND_DISTINCT_LARGEST.md
alreadySeen / banana	02-Core-Java/strings.md
first vs last vs all index	03-DSA/searching.md
move zeros / remove duplicates	03-DSA/two-pointers.md
Binary Search	03-DSA/binary-search.md
Big-O	07-Complexity/COMPLEXITY_CHEATSHEET.md
my old bugs	06-Mistakes/COMMON_MISTAKES.md

QUICK REFERENCE (open this before coding - 2 minutes)

DIGITS	ARRAY
%10 -> last digit	i = scanner (reads every element)
/10 -> remove last digit	pos = writer (where the next good value goes)
loop: while (num > 0)	length = count, last index = length - 1
reverse = reverse*10 + digit	int[] result = new int[count] (count first!)
SEARCH	STRING
contains -> return true early	s.length() s.charAt(i)
first -> return i at once	vowel: "aeiou".indexOf(ch) != -1
last -> keep updating	digit: ch >= '0' && ch <= '9'
all -> keep scanning	space: ch == ' '
not found -> return -1	equals(), not ==
count->allocate->fill (all)	alreadySeen -> check ONLY previous chars (j < i)
BINARY SEARCH	METHOD
array must be SORTED	input -> parameters -> processing -> return -> caller
left = 0, right = length-1	int result = method(args); (USE the return value)
while (left <= right)	arrays are shared: method changes the caller's array
middle = (left+right)/2	int/double parameters are copies
target == arr[middle] -> return middle	
target > arr[middle] -> left = middle + 1	
target < arr[middle] -> right = middle - 1	
end of loop -> return -1	

Start values (the #1 source of bugs)

Job	Start with	Never start with
largest	arr[0]	0 (breaks with negatives)
smallest	arr[0]	1000 (breaks with big values)
sum, count	0	-
product, factorial	1	0
not-found index	-1	0 (0 is a real index)
second largest	first two elements compared, loop from i = 2	second = arr[0]
second distinct largest	second = Integer.MIN_VALUE + secondFound = false	-

Pattern one-liners

Pattern	One line
Assume -> Compare -> Update	Assume arr[0] is best, compare the rest, update when beaten
Traverse -> Condition -> Counter++	Walk once, if matches, count++
Found -> Return	First match is enough: return i immediately
Last occurrence	Never return early, overwrite last = i each match
Count -> Allocate -> Fill	Count matches, new int[count], fill with pos
Two pointers	i at start, j at end, move inward while i < j
Remove duplicates (sorted)	pos = 1, copy arr[i] when arr[i] != arr[i-1]
Move zeros	Copy non-zeros forward with pos, then fill zeros from pos
alreadySeen	Inner loop j < i: did this value appear before?
Asc / desc / equal	Two flags, adjacent compare, both true means equal

Switch / loop reminders

- Every `case` needs `break` (Day 1 calculator forgot it on `'*'`).
- `for (init; condition; update);` do-while runs **at least once**.
- After `sc.nextInt()`, call `sc.nextLine()` once before reading a full line.
- Integer division drops decimals: use `(double) total / n`.
- `int` factorial breaks after `12!`; `long` after `20!`.

Complexity in one glance

Shape	Cost
One loop over the array	$O(n)$
Loop inside a loop	$O(n^2)$
Halving the range each step	$O(\log n)$
Only a few variables	$O(1)$ space

DAY BY DAY (Day 1 to Day 29)

Note: How days were identified

Days 6 to 29 come from `day6.java` to `day29.java`, so they are exact. Days 1 to 5 have no number in the ZIP. I grouped the numberless files by topic order, so the Day 1 to 5 boundaries are **approximate**. File dates show a two-month break (22 Jun to 25 Aug 2026), so early topics were revisited. Dates below are the file's last-saved date, not necessarily the day it was first written.

Timeline at a glance

Phase	Days	Topics
Foundations	1 to 5 (approx.)	Input, conditions, switch, loops, patterns, digits
Methods + arrays	6 to 15	Methods, return values, array algorithms, frequency
Analyzers	16 to 18	Menus, methods calling methods, big programs
Strings	19 to 21, 26	<code>charAt</code> , palindrome, <code>alreadySeen</code> , <code>charFrequency</code>
Array DSA	22 to 25, 27	Find indices, move zeros, duplicates, second distinct largest
Searching and sorting	28 to 29	Binary Search, bubble sort preview

Day 1 (approx.) - Program skeleton, input, conditions

Files: `hello.java`, `java0.java`, `conditional.java`

- **Topics learned:** `public class`, `main`, `Scanner`, `int`, `+`, `if / else`, even or odd with `% 2`
- **Concepts:** `sc.nextInt()` reads a number. `%` gives the remainder. `==` compares, `=` assigns.
- **Important code/patterns:** `if (num % 2 == 0) even else odd`
- **Supposed to understand:** How input flows into variables; how a condition chooses a branch.
- **Mistakes found:** None important. Older tries in `conditional.java` show `age > 18` for "adult" (18 itself is excluded).
- **Correct logic:** `age >= 18` when 18 should count.
- **Complexity:** $O(1)$ time, $O(1)$ space
- **Mastery:** 🟢 (repeated in almost every later file)
- **Revision questions:** 1. What does `num % 2 == 0` test? 2. Difference between `=` and `==`? 3. What does `sc.nextInt()` return?
- **Reconstruction challenge:** Read two numbers and print which is bigger, or "equal".
- **Patterns added:** Method-less input $\rightarrow$ condition $\rightarrow$ output

Day 2 (approx.) - else-if ladder, switch, first mini-project

Files: `conditional.java`, `switch1.java`, `switch2.java`, `project1.java`

- **Topics learned:** `else if`, `switch`, `case`, `break`, `default`, char input with `sc.next().charAt(0)`, grade calculator
- **Concepts:** A ladder picks the first true branch. `switch` jumps to a case and **keeps falling** until `break`.
- **Important code/patterns:** Weekday switch (1 to 7); calculator with `+` `-` `*` `/`; percentage then grade bands (85 / 75 / 65).
- **Supposed to understand:** Why `break` matters; how a menu choice maps to an action.

- **Mistakes found:** `switch1.java`: case '*' has no break, so after MUL it also prints "Invalid operation". Division by zero is unguarded. `project1.java`: $(m1+m2+m3)*100/300$ is integer division, so 85.67 becomes 85.
- **Correct logic:** Add `break`; to every case. Use (double) or / 3.0 for percentages.
- **Complexity:** $O(1)$
- **Mastery:** ● (switch break bug appears once and was not retested here; later menus on Days 16, 18 and 21 use break correctly, which is good evidence for the habit)
- **Revision questions:** 1. What happens if break is missing? 2. What does default do? 3. Why is 85.67 printed as 85.0?
- **Reconstruction challenge:** Write a 4-option calculator that never falls through.
- **Patterns added:** Menu with switch; ladder for ranges

Day 3 (approx.) - Loops

Files: `loop.java`, `loop1.java`, `table.java`, `reverse.java`, `pluseven.java`, `plus.java`, `fact.java`

- **Topics learned:** for, while, do-while, break, continue, `i += 2`, times table, sum 1..n, factorial
- **Concepts:** Loop = init, condition, update. do-while always runs once. continue skips one round.
- **Important code/patterns:** for (`int i = 1; i <= n; i++`) `sum += i`; even numbers with `i += 2` instead of if.
- **Supposed to understand:** Accumulator (sum, fact) and counter ideas.
- **Mistakes found:** `pluseven.java` checks `num % 2` instead of `i % 2`, so it sums everything when num is even and nothing when odd (fixed in Day 9). `reverse.java` prints odd numbers 1 to 19, not a reverse. `plus.java` computes factorial.
- **Correct logic:** Test the loop variable i inside the loop, not the input n.
- **Complexity:** Best / Avg / Worst $O(n)$, space $O(1)$
- **Mastery:** ●
- **Revision questions:** 1. How many times does for (`int i=1;i<=5;i++`) run? 2. Difference between while and do-while? 3. What does continue do? 4. Why start fact at 1?
- **Reconstruction challenge:** Print the sum of all even numbers from 1 to n.
- **Patterns added:** Accumulator, counter

Day 4 (approx.) - Nested loops and patterns

Files: `pattern.java`, `adpattern.java`, plus patterns inside `loop1.java`

- **Topics learned:** Rectangle, hollow rectangle, triangles, inverted triangles, right-aligned triangle, number triangles, Floyd, checkerboard `i + j`, pyramid, diamond halves, palindrome pyramid
- **Concepts:** Outer loop = rows, inner loop = columns. Spaces = $n - i$. Stars in a pyramid = $2*i - 1$.
- **Important code/patterns:** Hollow rectangle: print * if `i == 1 || j == 1 || i == m || j == n`.
- **Supposed to understand:** For each row, how many spaces and how many symbols?
- **Mistakes found:** None visible in active code. Most of `pattern.java` is commented history, which shows many variations were tried.
- **Correct logic:** Decide per row: `spaces(i)` then `symbols(i)` then newline.
- **Complexity:** $O(n^2)$ time, $O(1)$ space
- **Mastery:** ● (practised in 3 files, never recalled later)
- **Revision questions:** 1. What does the outer loop control? 2. How many spaces on row i of a right-aligned triangle? 3. Why `System.out.println()` after the inner loop?
- **Reconstruction challenge:** Print a 5-row pyramid of *.

- **Patterns added:** Nested-loop pattern

Day 5 (approx.) - Digit loops, early methods, first array

Files: `count.java`, `count1.java`, `loop1.java`, `ex.java`, `funmeth.java`, `exercise.java`, `Array.java`

- **Topics learned:** Count / sum / reverse / even-count / palindrome of a number; method with return; Q1 to Q10 exercise set; array input and search
- **Concepts:** `num % 10` last digit, `num / 10` removes it. Methods return a value to the caller.
- **Important code/patterns:** `reverse = reverse*10 + digit`; `palindrome = original == reverse`; GCD by trying every `i` up to the smaller number.
- **Supposed to understand:** Peel digits one by one until the number becomes 0.
- **Mistakes found:** `count1.java` returns 0 digits for input 0 (`count.java` handled it). `Array.java` prints "Element not found" on every non-matching element and has no found flag. `exercise Q4` uses `age > 18`; Q10 prints two terms even when `n = 1`.
- **Correct logic:** Search with a found flag (or `return i / -1`). Check the zero case when digits matter.
- **Complexity:** Digits $O(d)$ where d = number of digits; GCD $O(\min(a,b))$; Fibonacci $O(n)$
- **Mastery:** ● digits; ● methods; ○ arrays (first contact)
- **Revision questions:** 1. What does `% 10` give? 2. Why `/ 10`? 3. Why does reverse multiply by 10? 4. How is a palindrome number checked?
- **Reconstruction challenge:** Count the digits of a number, then handle input 0.
- **Patterns added:** Digit peel, Accumulator, Method + return

Day 6 - Digits recap and methods begin

*File: `day6.java` (saved 2 Sep) - marked "old concepts"

- **Topics learned:** Digit count / sum / even count / largest digit / combined; `static void greet(String)`, `mul(a,b)`, `static int add(a,b)`
- **Concepts:** A `void` method prints; an `int` method returns a value to the caller.
- **Important code/patterns:** One loop collecting count, sum, largest and even count together.
- **Supposed to understand:** Method -> parameters -> processing -> return -> caller.
- **Mistakes found:** Digit loops use `num > 0`, so input 0 gives count 0 (the earlier `count.java` special case was dropped).
- **Correct logic:** Add `if (num == 0) count = 1`; when 0 must count as one digit.
- **Complexity:** $O(d)$ time, $O(1)$ space
- **Mastery:** ● digits, ● methods
- **Revision questions:** 1. Difference between `void` and `int` methods? 2. What is a parameter? 3. Where does the returned value go?
- **Reconstruction challenge:** Write `static int add(int a, int b)` and call it from `main`.
- **Patterns added:** Method -> params -> return -> caller

Day 7 - Methods that return values

File: `day7.java` (saved 4 Sep)

- **Topics learned:** `square`, `isEven` (returns `n % 2 == 0`), `largest(a,b,c)`, `sumDigit`, `even`, `odd`, `evenSum`
- **Concepts:** A condition can be returned directly as a `boolean`.

- **Important code/patterns:** `return n % 2 == 0;` (compact boolean method)
- **Supposed to understand:** Each method does one job and returns one answer.
- **Mistakes found:** `largest(a,b,c)` starts with `largest = 0`, which fails when all inputs are negative. Unused variables (`orig`, `count`) remain in code.
- **Correct logic:** Start with the first value: `int largest = a;`
- **Complexity:** $O(1)$ for three numbers; $O(d)$ for digit methods
- **Mastery:** ●
- **Revision questions:** 1. Why can `isEven` return a comparison? 2. Why is `largest = 0` risky? 3. What is an unused variable?
- **Reconstruction challenge:** Write `largest(a,b,c)` that works for negatives.
- **Patterns added:** Boolean-returning method

Day 8 - Recall, then arrays begin

File: `day8.java` (saved 5 Sep)

- **Topics learned:** Array literal, index, `arr.length`, traversal, sum, largest, smallest, even count, print reversed, found flag, average
- **Concepts:** Indexes start at 0; last index is `length - 1`.
- **Important code/patterns:** `for (int i = 0; i < arr.length; i++)`
- **Supposed to understand:** Visit every element once and update one variable.
- **Mistakes found:** `largest = 0` and `smallest = 1000` are guessed starting values. The variable named `avg` is really the running total.
- **Correct logic:** Start `largest` and `smallest` with `arr[0]`. Name the sum `total`.
- **Complexity:** $O(n)$ time, $O(1)$ space
- **Mastery:** ● (first day; fixed on Day 10)
- **Revision questions:** 1. What is the last valid index? 2. Why is `i < arr.length` and not `<=`? 3. What breaks when `largest = 0`?
- **Reconstruction challenge:** Find the smallest value in {22, 39, 12, 53, 24, 14}.
- **Patterns added:** Traverse -> Condition -> Counter++, Assume -> Compare -> Update (early form)

Day 9 - Saturday revision (big recall)

File: `day9.java` (saved 5 Sep evening)

- **Topics learned:** `nextLine` vs `nextInt`; arithmetic menu; sign check; table; sum 1..n; factorial; digit reverse/sum/even-odd; methods; `findSum(int arr[])`, `findMax(int arr[])`
- **Concepts:** An array can be passed to a method. A method gets the same array.
- **Important code/patterns:** `static int findSum(int arr[])` called with `findSum(numbers)`.
- **Supposed to understand:** Pass the array in, return the answer out.
- **Mistakes found:** `findMax` still starts at `largest = 0` (bug returns after Day 8). `num1 / num2` unguarded for zero. `isEven` written with `if/else return true/false` after Day 7 already had the one-liner.
- **Correct logic:** `int largest = arr[0];` loop from `i = 1`. Prefer `return n % 2 == 0;`.
- **Complexity:** $O(n)$
- **Mastery:** ●

- **Revision questions:** 1. How do you pass an array to a method? 2. What is wrong with `largest = 0`? 3. When do you use `nextLine()`?
- **Reconstruction challenge:** Write `findMax` that works on `{-5, -2, -9}`.
- **Patterns added:** Method + array parameter

Day 10 - Methods + arrays (the sentinel fix)

File: `day10.java` (saved 7 Sep)

- **Topics learned:** Print array, print evens, `sumArray`, `findMax` (fixed), `newSmall`, even count, odd count, even sum, second largest
- **Concepts:** Start from the first element so any data works.
- **Important code/patterns:** Assume $\rightarrow$ Compare $\rightarrow$ Update with `arr[0]`; second largest by comparing the first two, then looping from `i = 2`.
- **Supposed to understand:** Why `largest = arr[0]` fixes negative arrays (the test uses `{-10, -111, -23, ...}` and prints `-10`).
- **Mistakes found:** (Fixed here) the `0` and `1000` starting values.
- **Correct logic:** Compare `arr[0]`, `arr[1]` first, then for `i = 2`: bigger than `largest`? shift `largest` $\rightarrow$ `second`. Else bigger than `second`? update `second`.
- **Complexity:** $O(n)$ time, $O(1)$ space
- **Mastery:** ● largest/smallest; ● second largest
- **Revision questions:** 1. Why is the loop `i = 1` after starting with `arr[0]`? 2. In second largest, why start at `i = 2`? 3. What does `second = largest` do?
- **Reconstruction challenge:** Second largest of `{23, 53, 14, 65, -13, 43}`.
- **Patterns added:** Assume $\rightarrow$ Compare $\rightarrow$ Update, Largest / second largest

Day 11 - Searching: contains, index, count, reverse, average

File: `day11.java` (saved 8 Sep)

- **Topics learned:** `contains` (boolean), `indexS` (return index or `-1`), count occurrences, in-place reverse, average (double)
- **Concepts:** `return -1` means "not found"; a real index is never negative.
- **Important code/patterns:** Reverse with two indexes: `i` from the start, `j` from the end, swap with `temp`, `j--`.
- **Supposed to understand:** Return early when found.
- **Mistakes found:** `contains` has no early return; it always scans the whole array (correct but wasteful).
- **Correct logic:** `if (arr[i] == target) return true`; inside the loop, `return false`; after it.
- **Complexity:** Contains: best $O(1)$, worst $O(n)$. Reverse: $n/2$ swaps = $O(n)$, space $O(1)$.
- **Mastery:** ● index, count and reverse (correct on Days 11 to 16; the code is identical every time, so rebuild it from a blank page to be sure)
- **Revision questions:** 1. What does `-1` represent? 2. Why a `temp` in swap? 3. When does reverse stop (`i < j`)? 4. Why does this method change the caller's array?
- **Reconstruction challenge:** Reverse `{10, 20, 30, 40, 50}` in place.
- **Patterns added:** Found $\rightarrow$ Return, `-1` = Not Found, Two pointers, Reverse by swap

Day 12 - Recall batch

File: `day12.java` (saved 9 Sep)

- **Topics learned:** `findLargest`, `findSmallest`, `countEvenOdd`, `searchElement`, `secondL`, `reverse`, `average`, `index`
- **Concepts:** `(double) total / arr.length` stops integer division.
- **Important code/patterns:** Cast **before** dividing.
- **Supposed to understand:** Same patterns, rebuilt from memory.
- **Mistakes found:** `searchElement` again scans fully without returning early.
- **Correct logic:** Use `return true` the moment you find it.
- **Complexity:** $O(n)$
- **Mastery:** ●
- **Revision questions:** 1. What does `(double) total / n` do differently from `total / n`? 2. Why `int total` plus a cast?
- **Reconstruction challenge:** Average of `{10, 34, 13, 53, 21}` as a decimal.
- **Patterns added:** Casting for division

Day 13 - Index, occurrence, duplicate (and a hidden bug)

File: `day13.java` (saved 10 Sep) - the whole file is commented out, kept as an archive

- **Topics learned:** `indexFind`, `findOccurance`, `sumOfElements`, `reverse`, `secondL`, **first duplicate** with nested loops
- **Concepts:** Compare each element with every later element using `j = i + 1`.
- **Important code/patterns:** Nested loop duplicate check for `j = i+1`.
- **Supposed to understand:** Why `j` starts at `i + 1` (never compare an element with itself or go backwards).
- **Mistakes found:** `secondL` loops from `i = 0` instead of `i = 2`. It passes its test only because `arr[0] < arr[1]`. If `arr[0] > arr[1]`, then at `i = 0` the check `arr[0] > second` is true and `second` becomes equal to `largest` (wrong).
- **Correct logic:** Start the loop at `i = 2`, because indexes 0 and 1 were already used to initialise `largest` and `second`.
- **Complexity:** Duplicate: best $O(1)$ (duplicate at start), worst $O(n^2)$. Others $O(n)$.
- **Mastery:** ● second largest (latent bug); ○ duplicate (appears once)
- **Revision questions:** 1. Why does the second-largest loop start at 2? 2. What goes wrong if it starts at 0 with `{50, 10, 30}`? 3. Why `j = i + 1`?
- **Reconstruction challenge:** Return the first value that appears twice, or -1.
- **Patterns added:** Nested-loop pair comparison

Day 14 - Recall, and the first `alreadySeen`

File: `day14.java` (saved 18 Sep, so you came back to it after Day 15 to Day 19)

- **Topics learned:** Recall of `reverse`, second largest (back to `i = 2`), occurrence; ****frequency of each distinct element****
- **Concepts:** To print each value once, skip values that appeared earlier.
- **Important code/patterns:** For each `i`, inner loop `j < i` sets `alreadySeen = true` and `breaks`. If not seen, call `frequency(num, num[i])`.
- **Supposed to understand:** Why `j < i` only looks at earlier elements.

- **Mistakes found:** None in code. The file being re-saved out of sequence suggests this day needed a second look.
- **Correct logic:** Output for {10,32,10,42,10,20,20}: 10>3, 32>1, 42>1, 20>2.
- **Complexity:** $O(n^2)$ time, $O(1)$ space
- **Mastery:** ● (spaced-recall topic)
- **Revision questions:** 1. What is `alreadySeen`? 2. Why `j < i`? 3. Why `break` after setting it? 4. What does `frequency` return?
- **Reconstruction challenge:** Print each distinct number of {4,4,7,4,9,7} once with its count.
- **Patterns added:** `alreadySeen`, Frequency counting, Method calling a method

Day 15 - Recall + most frequent

*File: `day15.java` (saved 12 Sep) - your own labels: "REVERSE DONE", "SECOND LARGEST DONE", "FREQUENCY OF EACH ELEMENT DONE"

- **Topics learned:** Recall of reverse, second largest, frequency; `**mostFrequent**`
- **Concepts:** Use the helper `frequency()` for each element and remember the best.
- **Important code/patterns:** Assume `arr[0]` is most frequent with its count, loop `i = 1`, update only when `count > highestFrequency`.
- **Supposed to understand:** Strict `>` keeps the earlier value on a tie.
- **Mistakes found:** None. (Slow: one full scan per element.)
- **Correct logic:** {10,10,20,13,2,3,23,2,3,3} returns 3.
- **Complexity:** Best / Avg / Worst $O(n^2)$ because `frequency` is $O(n)$ and is called n times; space $O(1)$
- **Mastery:** ●
- **Revision questions:** 1. What happens on a tie? 2. Why is it $O(n^2)$? 3. What are the two variables you update together?
- **Reconstruction challenge:** Most frequent element of {5,5,6,6,6,7}.
- **Patterns added:** Most frequent (frequency per element)

Day 16 - Saturday revision + first big Analyzer

File: `day16.java` (saved 13 Sep)

- **Topics learned:** Nine digit programs again (sum, reverse, factorial, palindrome, count, largest, smallest, even/odd counts, even sum), then a **do-while menu with switch** containing Number Analyzer and Student Marks Analyzer
- **Concepts:** Menu loop: `do { ... } while (choice != 3);`. Marks analyzer: total, average, largest, smallest, even count, search, occurrences, second largest, reverse.
- **Important code/patterns:** Reverse is done **last** (it changes the array that earlier steps read). Copies like `temp`, `temp2`, `temp3` are needed because the loops destroy `num1`.
- **Supposed to understand:** How to combine many small patterns in one program.
- **Mistakes found:** Factorial loops up to the entered number (a 5-digit input explodes: `long` overflows after 20!). Everything is inside `main`, so six temporary copies are needed. `largest2 = 0` initial value is replaced later, which is fine.
- **Correct logic:** Move each job into a method (done on Day 18). Methods get a copy of the number, so `temp` variables disappear.
- **Complexity:** $O(n)$ for the marks parts; $O(d)$ per digit loop
- **Mastery:** ● digit loops (3 or more days); ● menu structure

- **Revision questions:** 1. Why is reverse placed last? 2. Why do digit loops need temp copies in main but not in methods? 3. What makes the menu loop stop?
- **Reconstruction challenge:** A 3-option menu (Sum of digits, Reverse, Exit) using do-while + switch.
- **Patterns added:** Menu loop, Order of operations when mutating an array

Day 17 - Same Analyzer (duplicate)

*File: day17.java (saved 14 Sep) - identical to Day 16's analyzer; **no day17.class**, so it was probably never compiled*

- **Topics learned:** Re-saved the Day 16 analyzer (consolidation)
- **Concepts:** Same as Day 16
- **Important code/patterns:** Same
- **Supposed to understand:** Same
- **Mistakes found:** Duplicate file adds no new evidence; count Day 16 as the working version.
- **Correct logic:** Same as Day 16
- **Complexity:** Same
- **Mastery:** ● (not independently verified)
- **Revision questions:** 1. Which parts of the analyzer could be turned into methods? 2. Why is a duplicate file a poor study record?
- **Reconstruction challenge:** Write the menu skeleton from memory, no features.
- **Patterns added:** None new

Day 18 - Analyzer rebuilt with methods

File: day18.java (saved 21 Sep)

- **Topics learned:** Methods contains, countOccurrence, index, sumOfArray, arrayAnalyzer(sc, arr); digit methods sumDigits, reverse, isPalindrome (calls reverse), factorial, largestDigit, smallestDigit, countEvenDigits, sumEvenDigits, digitAnalyzer(sc); main menu "JAVA ANALYZER"
- **Concepts:** Method calling another method (isPalindrome uses reverse). Pass Scanner into methods. Nested menus.
- **Important code/patterns:** choice >= 1 && choice <= 8 read the number only for valid choices.
- **Supposed to understand:** Architecture: main menu -> sub menu -> small method.
- **Mistakes found:** factorial returns int, which overflows after 12!. isPalindrome keeps int original = num; although reverse(num) cannot change num (parameters are copies), so the extra variable is not needed. Digit methods still return wrong results for 0.
- **Correct logic:** Use long for factorial and cap input, and keep num as it is.
- **Complexity:** O(d) for digit methods; O(n) for array methods
- **Mastery:** ● (architecture); ● individual methods
- **Revision questions:** 1. Why does isPalindrome not need a temp copy? 2. How does the main menu reach digitAnalyzer? 3. Why pass Scanner sc as a parameter?
- **Reconstruction challenge:** isPalindrome(int) that calls your own reverse(int).
- **Patterns added:** Method calling method, Menu architecture, Pass-by-value copies

Day 19 - Strings begin

File: day19.java (saved 18 Sep)

- **Topics learned:** String, length(), charAt(i), traversal, countVowels ("aeiou".indexOf(ch) != -1), countDigits (ch >= '0' && ch <= '9'), countSpaces, reverseString by concatenation, palindromeString using equals
- **Concepts:** A String is indexed like an array but you call methods (length(), charAt()). char uses single quotes.
- **Important code/patterns:** Same loop as arrays: for i < str.length(), char ch = str.charAt(i).
- **Supposed to understand:** String vs char; use equals() for text.
- **Mistakes found:** Vowel check handles lowercase only. "shubham" gives 2 vowels (correct).
- **Correct logic:** Lowercase the text first, or include AEIOU.
- **Complexity:** O(n) time, O(1) space (reverse builds a new string: O(n) space)
- **Mastery:** ●
- **Revision questions:** 1. Difference between "a" and 'a'? 2. What does indexOf return when not found? 3. Why equals and not ==?
- **Reconstruction challenge:** Count digits in "a1b22" without Character helpers.
- **Patterns added:** String traversal, Traverse -> Condition -> Counter++ on text

Day 20 - String palindrome (wrong, then right) and charFrequency

File: day20.java (saved 19 Sep)

- **Topics learned:** Two-pointer palindrome, ignore-case version with toLowerCase(), countChar, charFrequency, **alreadySeen(str, index) as its own method**
- **Concepts:** alreadySeen loops i < index and compares earlier characters with the current one.
- **Important code/patterns:** if (alreadySeen(str, i)) continue; then countChar.
- **Supposed to understand:** Pointers move inward together; continue skips repeats.
- **Mistakes found:** Attempt 1: returns true on a mismatch and false at the end (inverted), and loops with i++ only while j stays at the last index. Attempt 2 is correct.
- **Correct logic:** while (i < j): mismatch -> return false; else i++; j--. After the loop return true.
- **Complexity:** Palindrome O(n) time, O(1) space. charFrequency O(n^2).
- **Mastery:** ● palindrome (one bug fixed); ● charFrequency (spaced recall)
- **Revision questions:** 1. What should a palindrome method return when characters differ? 2. Why must both i and j move? 3. Why i < j and not i <= j?
- **Reconstruction challenge:** Palindrome check that ignores case.
- **Patterns added:** Two pointers on a string, alreadySeen as a method

Day 21 - String Analyzer + Fibonacci

File: day21.java (saved 21 Sep)

- **Topics learned:** Menu program: Count Character, Vowels, Digits, Spaces, Frequency, Exit. Fibonacci with first, second, next.
- **Concepts:** After nextInt() the Enter key is still in the buffer; the code calls sc.nextLine() once before reading the sentence.
- **Important code/patterns:** char target = sc.next().charAt(0); Reuse countChar, countVowels, etc.
- **Supposed to understand:** String Analyzer architecture = main menu + string menu + small methods.

- **Mistakes found:** None blocking. Fibonacci prints with `println` plus a trailing space (cosmetic).
- **Correct logic:** Fibonacci: print `first`, then `next = first + second`; `first = second`; `second = next`.
- **Complexity:** Menu actions $O(n)$; frequency $O(n^2)$; Fibonacci $O(n)$ time, $O(1)$ space
- **Mastery:** ●
- **Revision questions:** 1. Why call `sc.nextLine()` before reading the string? 2. Which methods does option 5 use? 3. How does a char get read from `Scanner`?
- **Reconstruction challenge:** A 2-option menu (count vowels / exit) over an entered sentence.
- **Patterns added:** String Analyzer architecture, Fibonacci (two variables slide forward)

Day 22 - Student Analyzer (unfinished) + find indices

File: `day22.java` (saved 23 Sep, 1:05 a.m.)

- **Topics learned:** Parallel arrays `name[]` and `marks[]`, a static `int count` as the number of students stored, `addStudent`, `viewStudents`; `findIndex`, `findAllIndex`, `countOccurrence`, `largest`
- **Concepts:** Logical size vs physical array: capacity is 5, but only `count` slots are used, so you loop `i < count`.
- **Important code/patterns:** `if (count < 5) ... else "No space remaining";` found flag in `findAllIndex`.
- **Supposed to understand:** Why you loop to `count`, not `name.length`.
- **Mistakes found:** (1) Menu options 3 "Search" and 4 "Statistics" are printed but not implemented. (2) By tracing the code: `choice = sc.nextInt()` leaves the Enter key in the buffer, so `sc.nextLine()` in `addStudent` returns an empty name. Day 21 handled this with an extra `nextLine()`. (3) The file contains a stray `day23` class (`lowest`) pasted in the middle.
- **Correct logic:** Read the menu choice, then call `sc.nextLine()` once to eat the newline, before reading a name. Finish or remove unfinished menu items.
- **Complexity:** Add $O(1)$; view $O(\text{count})$; find all $O(n)$
- **Mastery:** ● find index / find all; ○ logical size (first use); UNFINISHED program
- **Revision questions:** 1. Why loop `i < count`? 2. What does `findAllIndex` print when nothing matches? 3. Why does `nextInt()` followed by `nextLine()` give an empty string?
- **Reconstruction challenge:** `addStudent` that stores up to 5 names without losing the first name.
- **Patterns added:** Logical size vs physical array, Flag boolean (`found`)

Day 23 - Second largest (distinct), second distinct (buggy), move zeros

*File: `day23.java` (saved 26 Sep) - contains your own note "handles duplicate too"

- **Topics learned:** `lowest`, `secondLargest` with `!= target`, `sumOfArray`, `avgOfArray` (`+=` and `cast`), `smallestInArray`, `secondDistinct` (smallest-based), `**moveZero**` with `pos`
- **Concepts:** `pos` is a writer index; `i` reads. Non-zeros are copied forward, then zeros fill the end.
- **Important code/patterns:** `if (arr[i] != 0) { arr[pos] = arr[i]; pos++; }` then `for (i = pos; ...) arr[i] = 0;`
- **Supposed to understand:** Two indexes with two jobs: scanner `i`, writer `pos`.
- **Mistakes found:** `secondLargest` starts with `second = arr[0]`, so when the first element is the largest (for example `{30, 10, 20}`) it returns 30 instead of 20. `secondDistinct` has a dead branch: `else if (arr[i] < smallest && arr[i] != smallest)` can never run because the first `if (arr[i] < smallest)` already catches it. For `{3, 5, 8}` it returns -1 although 5 exists. Test data `{3,3,3,3,3}` hides this.
- **Correct logic:** See Day 25 (largest-based version with `Integer.MIN_VALUE` and `secondFound`), and `08-Repair/SECOND_DISTINCT_LARGEST.md`.

- **Complexity:** Move zeros: $O(n)$ time, $O(1)$ space. Second largest: $O(n)$, $O(1)$.
- **Mastery:** ● second distinct (buggy attempt + you said you forgot it); ● move zeros (Days 23, 24, 25)
- **Revision questions:** 1. What does `i` do and what does `pos` do? 2. Why fill zeros from `pos`? 3. Why can the `else if` never run? 4. Why is `second = arr[0]` a bad start?
- **Reconstruction challenge:** Second distinct smallest of {8, 3, 3, 5, 9} (answer 5).
- **Patterns added:** Scanner vs Writer (`i` / `pos`), Move zeros, Second distinct (first attempt)

Day 24 - Move zeros, remove duplicates, sorted check, classify

File: `day24.java` (saved 25 Sep)

- **Topics learned:** `moveZero` (x2), `duplicate` = remove duplicates from a sorted array (x2, returns new length), `sorted`, ascending / descending / equal / neither classification
- **Concepts:** In a sorted array equal values sit next to each other, so compare `arr[i]` with `arr[i-1]`.
- **Important code/patterns:** `pos = 1; if (arr[i] != arr[i-1]) { arr[pos] = arr[i]; pos++; } return pos.`
Classification: two booleans `ascending`, `descending`; both true means all equal.
- **Supposed to understand:** Sorted-array reasoning; flags that start true and are knocked down by one counter-example.
- **Mistakes found:** `duplicate` assumes the array is sorted and not empty (`pos = 1` is wrong for length 0). The classifier method is called `neither`, but it returns all four answers.
- **Correct logic:** {50,50,50} -> both flags stay true -> "equal". {1,3,2} -> both flags false -> "neither".
- **Complexity:** All $O(n)$ time, $O(1)$ extra space. Sorted check best $O(1)$ (fails at the first pair).
- **Mastery:** ● move zeros; ● remove duplicates; ● sorted / classify (single day)
- **Revision questions:** 1. What does the return value of `duplicate` mean? 2. Why does `arr[i-1]` work for sorted data? 3. When are both flags true? 4. Can `sorted` accept equal neighbours?
- **Reconstruction challenge:** Classify {4,4,5,5} and {9,7,7}.
- **Patterns added:** Remove duplicates (sorted), Adjacent comparison, Asc / desc / equal classification

Day 25 - Second distinct largest (fixed), most frequent again

File: `day25.java` (saved 28 Sep)

- **Topics learned:** Recall of remove duplicates and move zeros; ****secondDistinct largest****; `mostFrequent` again
- **Concepts:** Track `largest` and `second`. A value qualifies as `second` only if it is `> second` and `< largest`.
- **Important code/patterns:** `second = Integer.MIN_VALUE; secondFound = false; when a new largest appears: second = largest; largest = arr[i]; secondFound = true;`
- **Supposed to understand:** Why a sentinel plus a flag, and what to return when there is no second value (-1).
- **Mistakes found:** -1 as "no second" is ambiguous if -1 is a real value. `mostFrequent` loop starts at 0 (harmless).
- **Correct logic:** {10,120,10,10,10} -> 10. {5,5,5} -> no second -> -1.
- **Complexity:** $O(n)$ time, $O(1)$ space. Most frequent $O(n^2)$.
- **Mastery:** ● second distinct largest (correct once; you told me you forgot it); ● most frequent
- **Revision questions:** 1. Why `< largest`? 2. Why a `secondFound` flag? 3. What if the array has only one distinct value?
- **Reconstruction challenge:** Rebuild second distinct largest from a blank page (see Repair Lab).
- **Patterns added:** Second distinct largest, Sentinel + flag

Day 26 - charFrequency on "banana"

File: day26.java (saved 29 Sep); 26.java is empty

- **Topics learned:** `alreadySeen` written inline inside `charFrequency`
- **Concepts:** Same logic as Days 14 and 20, now with characters.
- **Important code/patterns:** `for j < i` checks earlier characters; `continue` when seen.
- **Supposed to understand:** Why "banana" prints `b>1, a>3, n>2` only once each.
- **Mistakes found:** `count` is computed **before** the seen-check (wasted work); `int re = frequency(...)` is never used (called a method but ignored its result); the inner loop has no `break`.
- **Correct logic:** Check `alreadySeen` first, then count only for new characters, then print.
- **Complexity:** $O(n^2)$ time, $O(1)$ space
- **Mastery:** ● (spaced recall)
- **Revision questions:** 1. Why `j < i`? 2. Which characters of "banana" are skipped? 3. Why call `frequency` only after the seen-check?
- **Reconstruction challenge:** `charFrequency("mississippi")`.
- **Patterns added:** `alreadySeen` (inline form)

Day 27 - Find index family (closed)

File: day27.java (saved 29 Sep) - Day 27 is closed

- **Topics learned:** `findIndex` (prints every match), `findAllIndex` (returns `int[]`), `firstIndex`, `lastIndex`, `firstLast` in one pass
- **Concepts:** Count -> Allocate -> Fill: count matches, create new `int[count]`, fill with `pos`.
- **Important code/patterns:** First: `return i` immediately. Last: overwrite `lastIndex = i` and never return early. Both together: `if (firstIndex == -1) firstIndex = i; lastIndex = i;`
- **Supposed to understand:** First stops changing after the first match; last keeps changing.
- **Mistakes found:** `firstLast` creates new `int[2]` (default `{0,0}`) and fills it only on a match, so a missing target returns `{0,0}`, which looks like "found at index 0". It also writes `ar[0] / ar[1]` on every match.
- **Correct logic:** Fill the array after the loop: `return new int[]{firstIndex, lastIndex};` so a missing target gives `{-1,-1}`.
- **Complexity:** First: best $O(1)$, worst $O(n)$. Last: always n . All: $2n = O(n)$ time, $O(k)$ result.
- **Mastery:** ● each; ● `firstLast` (bug remains)
- **Revision questions:** 1. Why does `firstIndex` stop changing? 2. Why does `lastIndex` keep changing? 3. Why count before allocating? 4. What does `firstLast` return if the target is absent?
- **Reconstruction challenge:** All indices of 30 in `{10,30,20,30}` without hard-coding the result size.
- **Patterns added:** First / last occurrence, Count -> Allocate -> Fill, First + Last in one pass

Day 28 - Binary Search (current day)

File: day28.java (saved 30 Sep); test array in the ZIP: `{10, 20, 30, 40, 50, 60}`, target 50, result 4

- **Topics learned:** Binary Search on a sorted array: `left`, `right`, `middle`
- **Concepts:** Look at the middle. If it is the target, done. If the target is bigger, discard the left half (`left = middle + 1`). If smaller, discard the right half (`right = middle - 1`). Stop when `left > right`.
- **Important code/patterns:** `while (left <= right)`, recalculate `middle` **inside** the loop, compare with `arr[middle]`, return -1 after the loop.

- **Supposed to understand:** Why the array must be sorted, and why the range shrinks every round.
- **Mistakes found:** The algorithm is correct. **Test-data trap:** the array you described (`{10,20,42,10,49,38}`) is not sorted; searching for 38 would wrongly return -1. The array actually saved in the ZIP is sorted and fine.
- **Correct logic:** See `03-DSA/binary-search.md` for the full dry run.
- **Complexity:** Best $O(1)$, Average $O(\log n)$, Worst $O(\log n)$, Space $O(1)$
- **Mastery:** ● (core algorithm built; **dry run not completed**; not mastery-verified)
- **Revision questions:** 1. Why must the array be sorted? 2. When do we move **left**? 3. When do we move **right**? 4. Why `left <= right`? 5. What does -1 mean here?
- **Reconstruction challenge:** Blank-page Binary Search, then dry run for target 25 in `{10,20,30,40,50,60,70}`.
- **Patterns added:** Binary Search

Day 29 (found in ZIP, beyond your stated position)

File: `day29.java` (saved 30 Sep, 7:09 p.m.)

- **Topics learned:** **linear** (find all indices, count $\rightarrow$ allocate $\rightarrow$ fill), `findLastIndex`, **bubble sort**
- **Concepts:** Bubble sort compares neighbours and swaps; after each pass the largest remaining value sits at the end, so the inner loop shrinks (`arr.length - 1 - i`).
- **Important code/patterns:** `if (arr[j] > arr[j+1]) swap using temp`
- **Supposed to understand:** Why the inner loop upper bound shrinks and why `j + 1` stays inside the array.
- **Mistakes found:** None. (Outer loop runs one extra, harmless pass.)
- **Correct logic:** `{39,10,42,19,53}` $\rightarrow$ 10 19 39 42 53.
- **Complexity:** Best / Avg / Worst $O(n^2)$ as written (no early exit), space $O(1)$
- **Mastery:** ○ bubble sort; ● find all / last index (repeated from Day 27)
- **Revision questions:** 1. What does one pass guarantee? 2. Why `arr.length - 1 - i`? 3. Why must the swap use `temp`?
- **Reconstruction challenge:** Sort `{5,1,4}` by hand with pass-by-pass states.
- **Patterns added:** Bubble sort (swap neighbours)

Core Java 1 – Foundations and Conditions

What is it, and why do we need it?

Every program you wrote so far follows one shape: **take input -> decide -> repeat -> print**. Foundations are the pieces of that shape.

```
public class Name {
    public static void main(String args[]) {
        Scanner sc = new Scanner(System.in); // input device
        int a = sc.nextInt(); // read a number
        System.out.println("Sum: " + (a + 5)); // output
    }
}
```

Data types you actually used

Type	Holds	Example from your files
int	whole number	int num = sc.nextInt();
long	big whole number	long fact = 1; (factorial)
double	decimal	double percentage, double average
float	smaller decimal	float avg (Day 8)
boolean	true / false	boolean found = false;
char	one character, single quotes	char ch = str.charAt(i);
String	text, double quotes	String name = sc.nextLine();

Operators

Operator	Meaning	Remember
+ - * /	arithmetic	/ on two ints drops the decimal: 7/2 = 3
%	remainder	n % 2 == 0 means even, n % 10 is the last digit
== != < > <= >=	compare	= assigns, == compares
&& !	and, or, not	i == 1 j == 1 for the hollow rectangle border
++ -- +=	change by 1 / by amount	count++, sum += arr[i]

Mistake box: Integer division in a percentage

Wrong idea: `double p = (m1 + m2 + m3) * 100 / 300;` (all ints, so decimals are lost before saving into double).

Why: Java finishes the int maths first, then converts.

Correct: `double p = (m1 + m2 + m3) * 100.0 / 300;` or `(double) total / 3.`

Rule: **Make one side a double before the division.**

Scanner rules

Need	Code
int	sc.nextInt()
decimal	sc.nextDouble()
one word	sc.next()
a whole line	sc.nextLine()

Need	Code
one character	<code>sc.next().charAt(0)</code>

Mistake box: The vanishing line after `nextInt()`

Wrong idea: `int choice = sc.nextInt(); String s = sc.nextLine();` expects the sentence.

Why: `nextInt()` leaves the Enter key in the buffer, so `nextLine()` eats the empty rest of the line.

Correct: call `sc.nextLine();` once before the real `nextLine()` (you did this on Day 21).

Rule: **After a number, flush the Enter before a line.** Day 22 `addStudent` did not.

Conditions

```
if (cond) { ... }
else if (cond2) { ... }    // first TRUE branch only
else { ... }
```

Question	Code
even or odd	<code>if (n % 2 == 0)</code>
positive, negative, zero	<code>if (n > 0) ... else if (n < 0) ... else</code>
larger of two (equal case)	<code>if (a == b) ... else if (a > b)</code>
in a range / grade band	check from the highest band down (<code>>= 85</code> , then <code>>= 75</code> , ...)

switch

```
switch (x) {
    case 1: ...; break;    // break stops the fall-through
    case 2: ...; break;
    default: ...;         // anything else
}
```

Mistake box: Missing break

Wrong idea: `case '*': System.out.println(a*b);` (no break).

Why: execution **falls into** the next case, so "Invalid operation" also prints (your `switch1.java`).

Correct: `break;` after every case body.

Rule: **A case without break is a bug until proven otherwise.**

Recognise the problem

If the question says...	Use
"is it even / odd / positive"	<code>if, %</code>
"which menu choice"	<code>switch</code>
"between two values / grade"	<code>else if ladder</code>

Complexity

All of these run in constant time: $O(1)$ time, $O(1)$ space.

Core Java 2 – Loops, Patterns and Digit Programs

Loop forms

Loop	Shape	Use when
for	for (int i = 1; i <= n; i++)	you know how many times
while	while (num > 0)	you stop on a condition (digits)
do-while	do { ... } while (choice != 3);	run at least once (menus)

```
for (init ; condition ; update) { body }
```

```
init -> [condition?] --no--> exit
      | yes
      body -> update -> back to condition
```

- **break** leaves the loop. **continue** skips to the next round.
- **i += 2** walks even or odd numbers without an if.

Accumulator and counter

Job	Start	Update
sum	int sum = 0;	sum = sum + i;
factorial / product	int fact = 1;	fact = fact * i;
counter	int count = 0;	count++;

Pattern box: Accumulator

Purpose: collect one answer while walking. **Mental model:** a bucket you keep adding to.
The start value must be neutral: 0 for adding, 1 for multiplying.

Mistake box: Testing the wrong variable

Wrong idea (pluseven.java): for (i...) { if (num % 2 == 0) sum += i; }

Why: num never changes inside the loop, so the test is the same every round.

Correct: if (i % 2 == 0).

Rule: ****Inside a loop, test the loop variable (or arr[i]), not the input.****

Digit programs (your strongest topic)

```
num = 4821
%10 -> 1   (last digit)   num/10 -> 482
%10 -> 2           num/10 -> 48
%10 -> 8           num/10 -> 4
%10 -> 4           num/10 -> 0   stop (while num > 0)
```

Job	Core lines
count digits	count++; num = num / 10;
sum of digits	sum = sum + digit;
reverse	reverse = reverse * 10 + digit;
palindrome	orig == reverse (so keep a copy orig)
largest digit	start 0, if (digit > largest)
smallest digit	start 9 (a digit is never above 9)

Job	Core lines
even digits count	if (digit % 2 == 0) count++

Dry run: reverse of 123.

step	digit = num%10	reverse*10 + digit	num/10
1	3	$0*10+3 = 3$	12
2	2	$3*10+2 = 32$	1
3	1	$32*10+1 = 321$	0

Mistake box: Zero, negatives and factorial overflow

Wrong idea: digit loops with `while (num > 0)` assume a positive number; `int` factorial assumes small input.

Why: input 0 never enters the loop (`count = 0`); `13!` overflows `int`, `21!` overflows `long`. In the Day 16 analyzer the factorial of the entered number itself (e.g. 12345) is attempted.

Correct: special-case 0 when needed (your `count.java` did); factorial only for small `n`.

Rule: Ask "what if the input is 0?" before saving the file.

Note: Why temp copies exist in main

A digit loop destroys `num`. In `main` you must copy first (`temp`, `temp2`, `temp3` on Day 16). A method receives a copy automatically, so Day 18's methods need none.

Nested loops and star patterns

outer `i` = row inner `j` = column in the row

* row `i` has `i` stars: `j <= i`
 ** inverted: row `i` has `n-i+1`: `j <= n-i+1` (or `i` from `n` down to 1)
 *** right-aligned: `n-i` spaces, then `i` stars
 pyramid: `n-i` spaces, then `2*i-1` stars

Pattern	Spaces on row <code>i</code>	Symbols on row <code>i</code>
right triangle	0	<code>i</code>
inverted triangle	0	<code>n - i + 1</code>
right-aligned triangle	<code>n - i</code>	<code>i</code>
pyramid (your <code>adpattern</code>)	<code>n - i</code>	<code>2*i - 1</code>
hollow rectangle	-	* if <code>i==1 j==1 i==m j==n</code> else space
Floyd triangle	0	running <code>num++</code> shared across rows
checkerboard	0	print 1 if <code>(i+j)</code> is even, else 0

Pattern box: Nested-loop pattern

Mental model: outer loop = rows. For each row decide how many spaces and how many symbols, print them, then `println()`.

Complexity: $O(n^2)$ time, $O(1)$ space.

Complexity

Program	Time	Space
sum 1..n, factorial, table	$O(n)$	$O(1)$
digit programs	$O(d)$, d = digits = about $\log_{10}(n)$	$O(1)$
patterns	$O(n^2)$	$O(1)$
GCD by trying every <code>i</code> up to <code>min(a,b)</code>	$O(\min(a,b))$	$O(1)$

Program	Time	Space
Fibonacci n terms	$O(n)$	$O(1)$

Core Java 3 – Methods

What is it, and why do we need it?

A method is a **named job**. Day 16's analyzer put everything in **main** and needed six temp copies. Day 18 moved each job into a method and the code became smaller and testable.

INPUT -> PARAMETERS -> PROCESSING -> RETURN -> CALLER
(main) (copies) (the method) (answer) (uses it)

```
main:   int result = add(15, 25);
        |      |
method: static int add(int a, int b) { return a + b; }
```

Part	Example
return type	int, boolean, double, String, int[], or void (returns nothing)
name	isEven, findIndex (verb or question)
parameters	(int arr[], int target)
return	sends the answer back and ends the method

void vs return

	void	returns a value
Does	prints / changes something	calculates an answer
Caller	greet("Shubham");	int r = square(7);
Reuse	hard	easy (other methods can use the answer)

Mistake box: Calling a method and ignoring the result

Wrong idea (Day 26): `int re = frequency(str, ch);` and never using `re`.

Why: the work is done and thrown away (and the variable is useless).

Correct: use the returned value (`System.out.println(ch + " > " + count);`) or do not call it.

Rule: Every call to a returning method should end up in a variable or an expression.

Pass-by-value: what a method can change

You pass	Method gets	Can it change the caller's data?
int, double, char, boolean	a copy	No
array (int arr[])	the same array	Yes (reverse, moveZero, bubble)

This is why `reverse(num)` as a digit method needs no temp copy but `reverse(int arr[])` really reverses the array in main.

Note: Day 18 example

`isPalindrome(int num)` calls `reverse(num)`. The extra `int original = num;` is unnecessary because `reverse` only worked on its own copy.

Method calling another method

```
isPalindrome(num) -> reverse(num) -> returns rev
charFrequency(s)  -> alreadySeen(s, i) and countChar(s, ch)
mostFrequent(arr) -> frequency(arr, arr[i])
findAllIndex(arr) -> countOccurence(arr, target)
```

Pattern box: Method calling another method**Purpose:** keep every method small. **Mental model:** a manager that delegates.

Build and test the helper first (frequency, countChar, alreadySeen), then the big method.

Menu architecture (Days 16, 18, 21)

```

main
+-- do { show menu; read choice; switch } while (choice != exit)
+-- case 1 -> arrayAnalyzer(sc, arr)    // its own do-while menu
+-- case 2 -> digitAnalyzer(sc)
+-- case 3 -> exit message

```

- Pass **Scanner** `sc` into sub-menus: one scanner only.
- Validate with `if (choice >= 1 && choice <= 8)` before asking for a number.
- Unfinished options are worse than none (Day 22 printed "Search" and "Statistics" with no code).

Recognise the problem

Signal	Use
"do the same job for many inputs"	method with parameters
"yes/no answer"	boolean method
"several answers"	return an array, or print inside a void method
"the answer is needed by another method"	return it (do not print it)

Complexity

A method call itself costs $O(1)$; the cost is the work inside it.

Core Java 4 – Arrays

What is it, and why do we need it?

An array stores many values of one type in numbered boxes (indexes start at 0). You need it whenever the question says "given a list of numbers...".

```
index:  0    1    2    3    4
value: [ 10 | 20 | 30 | 40 | 50 ]    arr.length = 5, last index = arr.length - 1 = 4
```

Need	Code
ready values	<code>int arr[] = {10, 20, 30};</code>
empty, size n	<code>int arr[] = new int[n];</code> (all zeros)
size	<code>arr.length</code> (no brackets!)
read / write	<code>arr[i], arr[i] = 7;</code>
array to a method	<code>static int sum(int arr[]) then sum(numbers)</code>

Note: Array vs logical size

Physical size = `arr.length` (boxes that exist). Logical size = how many boxes you really filled. Day 22's student list had `new String[5]` but only `count` students, so the loops use `i < count`.

1. Traversal

- **PROBLEM:** visit every element (print them).
- **PATTERN:** Traverse.
- **VARIABLES:** `i` (index).
- **PSEUDOCODE:** for each index from 0 to length-1, use `arr[i]`.
- **JAVA:**

```
for (int i = 0; i < arr.length; i++) {
    System.out.println(arr[i]);
}
```
- **DRY RUN:** `{10,20,30}` prints 10, 20, 30.
- **COMPLEXITY:** $O(n)$ time, $O(1)$ space.
- **COMMON MISTAKES:** `i <= arr.length` goes out of bounds (valid indexes stop at `length - 1`).

2. Sum and average

- **PROBLEM:** total and average of elements.
- **PATTERN:** Accumulator.
- **VARIABLES:** `sum = 0`.
- **PSEUDOCODE:** `sum = 0`; for each element `sum += element`; `average = sum / count`.
- **JAVA:**

```
int sum = 0;
for (int i = 0; i < arr.length; i++) {
    sum = sum + arr[i];
}
double avg = (double) sum / arr.length;
```
- **DRY RUN:** `{10,20,37,40,50}`: sum 157, average 31.4.

- **COMPLEXITY:** $O(n)$ time, $O(1)$ space.
- **COMMON MISTAKES:** `sum / arr.length` with ints drops decimals. Day 8 named the total `avg`, which hid the real meaning.

3. Largest and smallest

- **PROBLEM:** biggest (or smallest) value.
- **PATTERN:** Assume $\rightarrow$ Compare $\rightarrow$ Update.
- **VARIABLES:** `largest = arr[0]`.
- **PSEUDOCODE:** assume the first is largest; for each other element, if it is bigger, replace.
- **JAVA:**

```
int largest = arr[0];
for (int i = 1; i < arr.length; i++) {
    if (arr[i] > largest) {
        largest = arr[i];
    }
}
```

- **DRY RUN:** `{-10, -111, -23}`: start `-10`; `-111` no; `-23` no; answer `-10`.
- **COMPLEXITY:** Best / Avg / Worst $O(n)$ (must look at all), space $O(1)$.
- **COMMON MISTAKES:** `largest = 0` (Days 7 to 9) fails for all-negative data; `smallest = 1000` fails for bigger data. Fixed on Day 10.

4. Contains (yes/no)

- **PROBLEM:** is `target` in the array?
- **PATTERN:** Found $\rightarrow$ Return (early return).
- **VARIABLES:** none besides `i`.
- **PSEUDOCODE:** for each element, if equal to target return true; after the loop return false.
- **JAVA:**

```
for (int i = 0; i < arr.length; i++) {
    if (arr[i] == target) {
        return true;
    }
}
return false;
```

- **DRY RUN:** `{10, 32, 53}` target 32 returns on `i = 1` (no need to check 53).
- **COMPLEXITY:** Best $O(1)$, Average $O(n)$, Worst $O(n)$, space $O(1)$.
- **COMMON MISTAKES:** Days 11 and 12 used a `boolean ok` flag and scanned everything (correct, slower). `return false` inside the loop would stop after the first element.

5. Count occurrence and even/odd counts

- **PROBLEM:** how many times does `target` appear? how many evens?
- **PATTERN:** Traverse $\rightarrow$ Condition $\rightarrow$ Counter++.
- **VARIABLES:** `count = 0`.
- **JAVA:**

```
int count = 0;
for (int i = 0; i < arr.length; i++) {
    if (arr[i] == target) { // or arr[i] % 2 == 0
        count++;
    }
}
```

```
}  
return count;
```

- **DRY RUN:** {10,20,20,20,109,10} target 20 -> 3.
- **COMPLEXITY:** $O(n)$ time, $O(1)$ space.
- **COMMON MISTAKES:** `count++` placed outside the `if` counts everything.

6. Array in a method (return vs print)

- **PROBLEM:** reuse a calculation.
- **PATTERN:** Method -> parameters -> processing -> return -> caller.

```
static int findSum(int arr[]) { ... return sum; }  
int result = findSum(numbers);
```

- **COMMON MISTAKES:** printing inside the method when another method needs the number.

See also: reverse and in-place changes in [03-DSA/two-pointers.md](#); search family in [03-DSA/searching.md](#).

Core Java 5 - Strings (a known retention weakness: revise often)

What is it, and why do we need it?

A **String** is text. Treat it like an array of characters: it has a length and each position has a **char**. Almost every string problem is an **array pattern** wearing a different hat.

```
String s = "banana";
index:   0   1   2   3   4   5
char:   [ b | a | n | a | n | a ]    s.length() = 6, last index = s.length() - 1
```

The tools

Need	Code	Note
declare	<code>String str = "shubham";</code>	double quotes
single character	<code>char ch = 'a';</code>	single quotes
length	<code>str.length()</code>	with brackets (arrays: <code>.length</code> without)
char at position	<code>str.charAt(i)</code>	index starts at 0
compare text	<code>str.equals(other)</code>	not <code>==</code>
lowercase	<code>str.toLowerCase()</code>	for ignore-case checks
position of a char	<code>"aeiou".indexOf(ch)</code>	returns -1 when absent

Mistake box: String vs char

Wrong idea: `if (str.charAt(i) == "a")` or `String c = 'a';`

Why: "a" is a String (text), 'a' is a char (one letter). Different types do not compare.

Correct: `str.charAt(i) == 'a'`.

Rule: `charAt` gives a char, so compare with single quotes.

Traversal (the base loop of everything below)

```
for (int i = 0; i < str.length(); i++) {
    char ch = str.charAt(i);
    // do something with ch
}
```

Character checks

Check	Code	Why it works
vowel (lowercase)	<code>"aeiou".indexOf(ch) != -1</code>	-1 means "not found in the vowel list"
digit	<code>ch >= '0' && ch <= '9'</code>	characters are ordered like numbers
space	<code>ch == ' '</code>	a space is a char

Mistake box: Case blindness

Wrong idea: vowel check on "Apple" finds only e.

Why: 'A' is not in "aeiou".

Correct: `str = str.toLowerCase();` first.

Rule: Normalise case before comparing letters.

countChar (how many times is one char in a string)

```
static int countChar(String str, char target) {
    int count = 0;
    for (int i = 0; i < str.length(); i++) {
        if (str.charAt(i) == target) {
            count++;
        }
    }
    return count;
}
```

This is **exactly** the array "count occurrence" pattern. `frequency(arr, target)` (Day 14) is the same code for `int` arrays.

alreadySeen and charFrequency - explained slowly

Goal: print each different character *once*, with how many times it appears.

The trick: the first time a character shows up, nothing before it matches it. If something before it matches, we have already handled this character earlier, so skip it.

```
static boolean alreadySeen(String str, int index) {
    for (int j = 0; j < index; j++) { // ONLY earlier positions
        if (str.charAt(j) == str.charAt(index)) {
            return true;
        }
    }
    return false;
}

static void charFrequency(String str) {
    for (int i = 0; i < str.length(); i++) {
        if (alreadySeen(str, i)) {
            continue; // skip repeats
        }
        int frequency = countChar(str, str.charAt(i));
        System.out.println(str.charAt(i) + " > " + frequency);
    }
}
```

Dry run: "banana"

i	current char	previous chars (j < i)	alreadySeen?	processed?	output
0	b	(none)	false	yes	b > 1
1	a	b	false	yes	a > 3
2	n	b a	false	yes	n > 2
3	a	b a n	true (matches j = 1)	skip	-
4	n	b a n a	true (matches j = 2)	skip	-
5	a	b a n a n	true (matches j = 1)	skip	-

Result: b > 1, a > 3, n > 2. Each letter appears once, with its **total** count (countChar scans the whole string, not only the part before i).

Why for (int j = 0; j < i; j++) checks only previous characters

```
position: 0 1 2 3 4 5
char:     b a n a n a
```

```

      ^
i = 3 -> look at j = 0,1,2 only (b, a, n)
      'a' found at j = 1 -> already seen
```

If the loop were...	What goes wrong
j <= i	charAt(i) always equals itself, so everything looks "seen" and nothing prints

If the loop were...	What goes wrong
<code>j < length</code>	later copies or itself match, so even the first <code>a</code> would be skipped
<code>j < i</code>	exactly the part before the current position, so only repeats are skipped

Pattern box: alreadySeen

Purpose: process each distinct value once. **Mental model:** a guest list checked at the door: "have I let this person in earlier?"

Works for arrays too (Day 14): same loop with `num[j] == num[i]`.

Complexity: $O(n^2)$ time (for each i , a scan of up to i , plus a full count), $O(1)$ space.

Mistake box: Day 26 version: three small inefficiencies

Wrong idea: compute `count = frequency(...)` **before** the seen-check; keep an unused `int re = frequency(...)`; no **break** in the inner loop.

Why: wasted work on skipped characters; the unused variable calls a method and throws the answer away.

Correct: seen-check first, count after, **return true** (or **break**) as soon as a match is found.

Rule: **Skip first, work later.**

Whole-string jobs (all use the same loop)

Job	Core test	Mastery
count vowels	<code>"aeiou".indexOf(ch) != -1</code>	●
count digits	<code>ch >= '0' && ch <= '9'</code>	●
count spaces	<code>ch == ' '</code>	●
reverse a string	loop from <code>length-1</code> down to <code>0</code> , <code>reverse = reverse + ch</code>	●
palindrome (two pointers)	<code>i = 0, j = length-1; mismatch -> false; i++, j--</code>	●
ignore-case palindrome	<code>str = str.toLowerCase(); first</code>	●

Note: Reverse by concatenation

`reverse = reverse + ch` builds a new `String` every round. Fine for small inputs; the cost grows like $O(n^2)$. (A faster tool is a later topic - not in your ZIP yet.)

String Analyzer architecture (Day 21)

```
main()
+-- do { 1.String Analyzer  2.Exit } while (choice != 2)
+-- stringAnalyzer(sc)
    sc.nextLine();           // flush the Enter after nextInt
    String str2 = sc.nextLine();
    do { menu: 1 countChar  2 vowels  3 digits  4 spaces  5 frequency  6 exit }
        +-- countChar(str2, target)  <- char from sc.next().charAt(0)
        +-- countVowels(str2)
        +-- countDigits(str2)
        +-- countSpaces(str2)
        +-- charFrequency(str2) -> alreadySeen(...) + countChar(...)
```

Complexity

Algorithm	Time	Space
traversal, countChar, vowels, digits, spaces	$O(n)$	$O(1)$
palindrome (two pointers)	$O(n)$ ($n/2$ comparisons)	$O(1)$
reverse string by +	$O(n^2)$ copying	$O(n)$
charFrequency with alreadySeen	$O(n^2)$	$O(1)$

DSA 1 – Searching and Scanning Arrays

Note: Same skeleton every time

Almost every algorithm here is: **walk the array once, keep a tiny bit of memory, decide what to do on each element.**
The skill is choosing the memory (a counter, a flag, a best value, an index).

Which one do I need? (decision table)

Question	Pattern	Return
"is it there?"	Found -> Return	boolean
"where is the first one?"	First occurrence	index or -1
"where is the last one?"	Last occurrence	index or -1
"where are all of them?"	Count -> Allocate -> Fill	int[]
"first and last together"	First + Last in one pass	two numbers
"biggest / second biggest"	Assume -> Compare -> Update	value
"most common"	Frequency per element	value

1. First index (stop at the first match)

- **PROBLEM:** index of the first **target**, or -1.
- **PATTERN:** Found -> Return; -1 = Not Found.
- **VARIABLES:** **i**.
- **PSEUDOCODE:** for each index: if equal, return index. After the loop, return -1.
- **JAVA:**

```
static int firstIndex(int arr[], int target) {
    for (int i = 0; i < arr.length; i++) {
        if (arr[i] == target) {
            return i;          // first match is enough
        }
    }
    return -1;
}
```

- **DRY RUN:** {10,20,10,40,10}, target 10: i = 0 matches -> returns 0 (never sees the other 10s).
- **COMPLEXITY:** Best $O(1)$, Avg $O(n)$, Worst $O(n)$, space $O(1)$.
- **COMMON MISTAKES:** returning 0 for not found (0 is a real index); **Array.java** printing "not found" on every non-match.

2. Last index (keep updating)

- **PROBLEM:** index of the last **target**, or -1.
- **PATTERN:** Last occurrence (overwrite, never return early).
- **VARIABLES:** **lastIndex = -1**.
- **JAVA:**

```
static int lastIndex(int arr[], int target) {
    int lastIndex = -1;
    for (int i = 0; i < arr.length; i++) {
        if (arr[i] == target) {
            lastIndex = i;      // keeps changing
        }
    }
    return lastIndex;
}
```

```

    }
  }
  return lastIndex;
}

```

- **DRY RUN:** {10,20,10,40,10}, target 10: lastIndex becomes 0, then 2, then 4 -> returns 4.
- **COMPLEXITY:** Best = Avg = Worst $O(n)$ (must see the end); space $O(1)$.
- **COMMON MISTAKES:** adding `return` inside the `if` turns it into "first".

Tip: Why first stops and last keeps changing

`firstIndex` returns at the first hit, so nothing else can change it. `lastIndex` is only a note on paper: every new hit overwrites the note, and the last overwrite wins.

3. All indices: Count -> Allocate -> Fill

- **PROBLEM:** return every index where `target` appears.
- **PATTERN:** Count -> Allocate -> Fill (scanner `i`, writer `pos`).
- **VARIABLES:** `count`, `ar[]` (size `count`), `pos`.
- **PSEUDOCODE:** count matches; make an array of that size; scan again, write each index at `pos`, `pos++`.
- **JAVA:**

```

static int[] findAllIndex(int arr[], int target) {
    int count = countOccurrence(arr, target); // pass 1 (helper)
    int ar[] = new int[count];                // exact size
    int pos = 0;
    for (int i = 0; i < arr.length; i++) {    // loop the ORIGINAL array
        if (arr[i] == target) {
            ar[pos] = i;
            pos++;
        }
    }
    return ar;
}

```

- **DRY RUN:** {10,20,10,20,10,20,39,43} target 20: count = 3, ar = [_,_,_]; matches at `i = 1, 3, 5` -> [1,3,5].
- **COMPLEXITY:** Time $2n = O(n)$; extra space $O(k)$ where k = matches (that is the output itself).
- **COMMON MISTAKES:** hard-coding the result size; looping over `ar` instead of `arr`; forgetting `pos++`; no match -> returns an empty array (length 0), which is correct.

4. First + Last in one pass

- **PROBLEM:** first and last index of `target` with one scan.
- **PATTERN:** First + Last in one pass.
- **VARIABLES:** `firstIndex = -1`, `lastIndex = -1`.
- **JAVA (recommended version):**

```

static int[] firstLast(int arr[], int target) {
    int firstIndex = -1;
    int lastIndex = -1;
    for (int i = 0; i < arr.length; i++) {
        if (arr[i] == target) {
            if (firstIndex == -1) {
                firstIndex = i; // set once
            }
            lastIndex = i; // overwrite every time
        }
    }
    return new int[]{firstIndex, lastIndex};
}

```

- **DRY RUN:** {10,20,30,49,30,30,20,30} target 30 -> [2, 7]. Target 99 -> [-1, -1].
- **COMPLEXITY:** $O(n)$ time, $O(1)$ space (result has 2 slots).
- **COMMON MISTAKES:** Your Day 27 version creates new `int[2]` (zeros) and fills it only inside the `if`, so a missing target returns {0,0}. Earlier attempt vs corrected: fill after the loop (as above).

5. Second largest

- **PROBLEM:** second biggest value (duplicates may count as separate values).
- **PATTERN:** Assume -> Compare -> Update with two variables.
- **VARIABLES:** largest, second.
- **PSEUDOCODE:** order the first two; for the rest: bigger than largest -> shift down; else bigger than second -> replace second.
- **JAVA:**

```
int largest, second;
if (arr[0] > arr[1]) { largest = arr[0]; second = arr[1]; }
else { largest = arr[1]; second = arr[0]; }
for (int i = 2; i < arr.length; i++) { // start at 2!
    if (arr[i] > largest) {
        second = largest;
        largest = arr[i];
    } else if (arr[i] > second) {
        second = arr[i];
    }
}
```

- **DRY RUN:** {23,53,14,65,-13,43}: start largest 53, second 23; 14 -> no; 65 -> second 53, largest 65; -13 no; 43 -> not above 53, no change. Answer: second = 53.
- **COMPLEXITY:** $O(n)$ time, $O(1)$ space.
- **COMMON MISTAKES:** loop from `i = 0` (Day 13; wrong when `arr[0] > arr[1]`); `second = arr[0]` start (Day 23; wrong when the first element is the max).

6. Second distinct largest

- **PROBLEM:** second biggest *different* value ({10,120,10,10} -> 10; {5,5,5} -> none).
- **PATTERN:** Second distinct largest = sentinel + flag + strict range `second < x < largest`.
- **VARIABLES:** largest = `arr[0]`, second = `Integer.MIN_VALUE`, `secondFound = false`.
- **JAVA (your Day 25 version):**

```
static int secondDistinct(int arr[]) {
    int largest = arr[0];
    int second = Integer.MIN_VALUE;
    boolean secondFound = false;
    for (int i = 0; i < arr.length; i++) {
        if (arr[i] > largest) {
            second = largest;
            largest = arr[i];
            secondFound = true;
        } else if (arr[i] > second && arr[i] < largest) {
            second = arr[i];
            secondFound = true;
        }
    }
    if (!secondFound) {
        return -1;
    }
    return second;
}
```

- **DRY RUN:** {10,120,10,10,10}: $i = 1$: $120 > 10 \rightarrow$ second 10, largest 120, found; later 10s: not > 120 , and $10 > \text{second}(10)$ false $\rightarrow$ unchanged. Return 10.
- **COMPLEXITY:** $O(n)$ time, $O(1)$ space.
- **COMMON MISTAKES:** the Day 23 attempt (smallest-based, dead branch). Full repair lab: 08-Repair/SECOND_DISTINCT_LARGEST.md.

7. Most frequent element

- **PROBLEM:** the value that appears most.
- **PATTERN:** Frequency per element (uses `frequency()`), Assume $\rightarrow$ Compare $\rightarrow$ Update.
- **VARIABLES:** `mostFrequent`, `highestFrequency`.
- **JAVA:**

```
int mostFrequent = arr[0];
int highestFrequency = frequency(arr, arr[0]);
for (int i = 1; i < arr.length; i++) {
    int count = frequency(arr, arr[i]);
    if (count > highestFrequency) {           // strict: ties keep the earlier value
        mostFrequent = arr[i];
        highestFrequency = count;
    }
}
```

- **DRY RUN:** {10,10,20,13,2,3,23,2,3,3}: 10 has 2; 2 has 2 (not $>$); 3 has 3 $\rightarrow$ best is 3.
- **COMPLEXITY:** $O(n^2)$: n calls to an $O(n)$ helper; space $O(1)$.
- **COMMON MISTAKES:** returning the count instead of the value; using $\geq$ (changes tie behaviour).

8. Duplicate (first repeated value)

- **PROBLEM:** return the first value that appears again.
- **PATTERN:** nested pair comparison, $j = i + 1$.
- **JAVA:**

```
for (int i = 0; i < arr.length; i++) {
    for (int j = i + 1; j < arr.length; j++) {
        if (arr[i] == arr[j]) {
            return arr[i];
        }
    }
}
return -1;
```

- **COMPLEXITY:** Best $O(1)$, Worst $O(n^2)$, space $O(1)$.

9. Sorted check

- **PROBLEM:** is the array in non-decreasing order?
- **PATTERN:** Sorted-array reasoning (adjacent compare, early false).
- **JAVA:**

```
for (int i = 1; i < arr.length; i++) {
    if (arr[i - 1] > arr[i]) {
        return false;           // one violation is enough
    }
}
return true;
```

- **DRY RUN:** {1,2,3,4,4,5} $\rightarrow$ no neighbour is out of order $\rightarrow$ true (equal neighbours are allowed).
- **COMPLEXITY:** Best $O(1)$, Worst $O(n)$, space $O(1)$.

- **COMMON MISTAKES:** `return true` inside the loop; starting at `i = 0` (reads `arr[-1]`).

10. Ascending / descending / equal / neither

- **PATTERN:** two flags, both start `true`; one counter-example knocks one down.

```
boolean ascending = true, descending = true;
for (int i = 1; i < arr.length; i++) {
    if (arr[i] > arr[i - 1]) descending = false;
    else if (arr[i - 1] > arr[i]) ascending = false;
}
if (ascending && !descending) return "ascending";
else if (descending && !ascending) return "descending";
else if (ascending && descending) return "equal";
return "neither";
```

Array	ascending	descending	Answer
{1,2,3}	true	false	ascending
{9,7,5}	false	true	descending
{50,50,50}	true	true	equal
{1,3,2}	false	false	neither

- **COMPLEXITY:** $O(n)$ time, $O(1)$ space.
- **COMMON MISTAKES:** mixing up `equal` (both true) and `neither` (both false).

DSA 2 – Two Pointers, Scanner/Writer and a Sorting Preview

What is it, and why do we need it?

Two indexes walk the same array. There are **two flavours** in your code:

Flavour	Pointers	Used in
Facing each other	i from the start, j from the end, both move inward	reverse array, palindrome
Scanner and Writer	i reads every element, pos marks where the next good value goes	move zeros, remove duplicates, find-all-index fill

FACING EACH OTHER

```
[10 20 30 40 50]
  i      j
  i -->      <-- j
```

SCANNER / WRITER

```
[0 10 0 32 5]
  i=reads pos=writes
  pos never passes i (pos <= i)
```

Pattern box: Scanner vs Writer

i = scanner: looks at every element. **pos** = writer: where the next kept value will be stored.

pos moves only when something is kept. Because **pos <= i**, writing never destroys unread data.

1. Reverse an array (in place)

- **PROBLEM:** reverse {10,20,30,40,50} without a new array.
- **PATTERN:** Two pointers facing each other + swap with **temp**.
- **VARIABLES:** **i** = 0, **j** = **arr.length - 1**, **temp**.
- **PSEUDOCODE:** while **i < j**: swap **arr[i]** and **arr[j]**, move **i** up and **j** down.
- **JAVA:**

```
static void reverse(int arr[]) {
    int temp;
    int j = arr.length - 1;
    for (int i = 0; i < j; i++) {
        temp = arr[i];
        arr[i] = arr[j];
        arr[j] = temp;
        j--;
    }
}
```

- **DRY RUN:**

step	i	j	array after swap
start	0	4	10 20 30 40 50
1	0	4	50 20 30 40 10
2	1	3	50 40 30 20 10
stop	2	2	i < j false, middle stays

- **COMPLEXITY:** $n/2$ swaps = $O(n)$ time, $O(1)$ space.
- **COMMON MISTAKES:** swapping without **temp** (**arr[i] = arr[j]; arr[j] = arr[i];** loses a value); looping to **arr.length** swaps everything back to the original.

2. Palindrome (string)

- **PROBLEM:** does the word read the same both ways?
- **PATTERN:** Two pointers facing each other.
- **JAVA (Day 20 corrected):**

```
static boolean isPalindrome(String str) {
    int i = 0;
    int j = str.length() - 1;
    while (i < j) {
        if (str.charAt(i) != str.charAt(j)) {
            return false; // one mismatch is enough
        }
        i++;
        j--;
    }
    return true; // no mismatch found
}
```

- **DRY RUN:** "naman": (n,n) ok -> (a,a) ok -> i = j = 2 stop -> true. "nama": (n,a) differ -> false.
- **COMPLEXITY:** $O(n)$ time, $O(1)$ space.
- **Earlier attempt -> Problem -> Corrected:** the first version returned **true** when characters **differed**, ended with **false**, and only moved **i** (so **j** stayed at the end). The final version returns **false** on mismatch, **true** at the end, and moves both pointers.
- **COMMON MISTAKES:** inverted booleans; forgetting **j--**; **i <= j** (harmless but pointless).

3. Move zeros to the end (Scanner / Writer)

- **PROBLEM:** {0,10,32,0,21} -> {10,32,21,0,0}, keep the order of non-zeros.
- **PATTERN:** Move zeros (copy non-zeros forward, then fill zeros).
- **VARIABLES:** **i** (scanner), **pos** (writer).
- **JAVA:**

```
static void moveZero(int arr[]) {
    int pos = 0;
    for (int i = 0; i < arr.length; i++) {
        if (arr[i] != 0) {
            arr[pos] = arr[i]; // write kept value
            pos++;
        }
    }
    for (int i = pos; i < arr.length; i++) {
        arr[i] = 0; // zeros fill the tail
    }
}
```

- **DRY RUN:** {0, 10, 0, 32}

i	arr[i]	action	pos after	array
0	0	skip	0	0 10 0 32
1	10	arr[0]=10	1	10 10 0 32
2	0	skip	1	10 10 0 32
3	32	arr[1]=32	2	10 32 0 32
fill	-	positions 2..3 = 0	-	10 32 0 0

- **COMPLEXITY:** about $2n$ steps = $O(n)$ time, $O(1)$ extra space.
- **COMMON MISTAKES:** starting the second loop at 0 instead of **pos**; moving **pos** on every element; confusing **i** and **pos**.

4. Remove duplicates from a sorted array

- **PROBLEM:** {1,1,2,2,3,3,4,4,5,5} -> first 5 slots 1 2 3 4 5, return the new length 5.
- **PATTERN:** Scanner / Writer + sorted-array reasoning (equals are neighbours).
- **VARIABLES:** pos = 1 (the first element is always kept), i from 1.
- **JAVA:**

```
static int removeDuplicates(int arr[]) {
    int pos = 1;
    for (int i = 1; i < arr.length; i++) {
        if (arr[i] != arr[i - 1]) { // a new value starts here
            arr[pos] = arr[i];
            pos++;
        }
    }
    return pos; // number of unique values
}
```

- **DRY RUN:** {1,1,2,2,3}

i	arr[i] vs arr[i-1]	different?	write	pos
1	1 vs 1	no	-	1
2	2 vs 1	yes	arr[1]=2	2
3	2 vs 2	no	-	2
4	3 vs 2	yes	arr[2]=3	3

Unique prefix: 1 2 3, return 3.

- **COMPLEXITY:** $O(n)$ time, $O(1)$ space.
- **COMMON MISTAKES:** works **only on sorted data**; pos = 1 is wrong for an empty array; printing the whole array instead of the first pos values.

5. Bubble sort preview (Day 29, first contact)

- **PROBLEM:** sort {39,10,42,19,53} ascending.
- **PATTERN:** Swap neighbours; the biggest value "bubbles" to the end each pass.
- **JAVA:**

```
for (int i = 0; i < arr.length; i++) {
    for (int j = 0; j < arr.length - 1 - i; j++) {
        if (arr[j] > arr[j + 1]) {
            int temp = arr[j + 1];
            arr[j + 1] = arr[j];
            arr[j] = temp;
        }
    }
}
```

- **DRY RUN (pass 1):** 39 10 42 19 53 -> 10 39 42 19 53 -> 10 39 42 19 53 -> 10 39 19 42 53 -> 10 39 19 42 53. 53 is already in place; after pass 1 the last slot is final.
- **COMPLEXITY:** Best / Avg / Worst $O(n^2)$ as written (about $n(n-1)/2$ comparisons), space $O(1)$. (Adding a "no swap, stop" flag would give a best case of $O(n)$.)
- **COMMON MISTAKES:** inner bound arr.length - 1 without - i (extra work); inner bound arr.length (reads arr[j+1] out of range).
- **Status:** ○ introduced.

Tip: Bubble sort and Binary Search

Binary Search needs sorted data. Sorting first (bubble sort works) and then searching is a valid way to fix an unsorted array - but sorting costs more than one linear search, so it only pays off if you search many times.

DSA 3 – Binary Search (Day 28) – special handling

Status: ● understood. The core algorithm was built; the **dry run** was not completed and mastery is not verified.

What is it, and why do we need it?

Linear search checks one element at a time (up to n checks). If the array is **sorted**, you can look at the **middle** and throw away half of the array every round.

Guess a number from 1 to 100: 50? too high -> 25? too low -> 37? ...
You never check one by one. Each guess removes half the remaining numbers.

Your Day 28 code (the algorithm is correct)

```
static int binarySearch(int arr[], int target) {
    int left = 0;
    int right = arr.length - 1;

    while (left <= right) {
        int middle = (left + right) / 2;

        if (target == arr[middle]) {
            return middle;
        } else if (target > arr[middle]) {
            left = middle + 1;
        } else {
            right = middle - 1;
        }
    }

    return -1;
}
```

Note: Verdict

The algorithm is **correct for a sorted ascending array**. Nothing in the method needs fixing. The only problem is the input in some test runs (see the trap below).

Line-by-line meaning

Line	Meaning
<code>left = 0</code>	first index still possible
<code>right = arr.length - 1</code>	last index still possible
<code>while (left <= right)</code>	the search window still has at least one element
<code>middle = (left + right) / 2</code>	the centre of the window; recalculated every round, inside the loop
<code>target == arr[middle]</code>	compare with the value at middle , not with middle itself
<code>target > arr[middle]</code>	target is in the right half -> <code>left = middle + 1</code>
<code>target < arr[middle]</code> (else)	target is in the left half -> <code>right = middle - 1</code>
<code>return -1</code>	the window became empty (<code>left > right</code>): not found

	left		middle		right		
index:	0	1	2	3	4	5	6
value:	10	20	30	40	50	60	70

target > arr[middle] -> left = middle + 1 (throw away the left half AND the middle)
target < arr[middle] -> right = middle - 1 (throw away the right half AND the middle)

Tip: Why +1 and -1?

`arr[middle]` was already checked and is not the answer, so it must leave the window. Without `+1` / `-1` the window may never shrink and the loop could run forever.

Why the array must be sorted

Each decision says: "the target is bigger than the middle, so everything on the left is smaller, so it cannot be there." That sentence is only true when the array is ordered. In an unsorted array the left half can still contain the target, but you threw it away.

Dry run 1: found. {10,20,30,40,50,60,70}, target 60

round	left	right	middle	arr[middle]	compare	action
1	0	6	3	40	$60 > 40$	<code>left = 4</code>
2	4	6	5	60	$60 == 60$	<code>return 5</code>

Dry run 2: not found. Same array, target 25

round	left	right	middle	arr[middle]	compare	action
1	0	6	3	40	$25 < 40$	<code>right = 2</code>
2	0	2	1	20	$25 > 20$	<code>left = 2</code>
3	2	2	2	30	$25 < 30$	<code>right = 1</code>
end	2	1	-	-	$left > right$	<code>return -1</code>

Your ZIP's own run (Day 28 file)

{10,20,30,40,50,60}, target 50: round 1 middle = 2 (30), $50 > 30 \rightarrow left = 3$; round 2 middle = 4 (50) $\rightarrow$ returns 4 (prints Found At : 4).

The unsorted-array trap (flagged)**Mistake box: Testing Binary Search on an unsorted array**

What you described for Day 28: `int num[] = {10, 20, 42, 10, 49, 38};` - this is **not sorted**.

What the ZIP contains: `day28.java` and `day28.class` both use {10, 20, 30, 40, 50, 60} (sorted), so that exact unsorted array is not in the archive. The warning below is still important.

Why it is wrong: the algorithm's logic assumes order. The algorithm is not broken - the test input is invalid.

Correct idea: sort first, or use linear search. Test with {10, 20, 30, 40, 50, 60, 70}.

Mental rule: Before Binary Search, ask: "Is it sorted?"

Proof with {10, 20, 42, 10, 49, 38}:

target	what happens	result	truth
38	middle 2 (42): $38 < 42 \rightarrow right = 1$; middle 0 (10): $38 > 10 \rightarrow left = 1$; middle 1 (20): $38 > 20 \rightarrow left = 2$ > right	-1	38 is at index 5
49	middle 2 (42): $49 > 42 \rightarrow left = 3$; middle 4 (49) $\rightarrow$ found	4	correct, but only by luck

A "correct" answer on an unsorted array is luck, not logic.

Complexity

	Value	Why
Best	$O(1)$	the target is the first middle
Average	$O(\log n)$	each round halves the window
Worst	$O(\log n)$	the window shrinks to 0 or 1 element
Space	$O(1)$	only left, right, middle

How many rounds? Halving n until 1 takes about $\log_2(n)$ rounds: 8 elements $\rightarrow$ 3 rounds; 1,000 $\rightarrow$ about 10; 1,000,000 $\rightarrow$ about 20. Linear search could need 1,000,000.

Common mistakes (checklist)

Mistake	Fix
comparing <code>target</code> with <code>middle</code> instead of <code>arr[middle]</code>	compare with <code>arr[middle]</code>
computing <code>middle</code> once, outside the loop	recompute inside the loop every round
<code>left = middle / right = middle</code> (no <code>+</code> / <code>-</code>)	<code>left = middle + 1</code> , <code>right = middle - 1</code>
<code>while (left < right)</code>	use <code>left <= right</code> or you miss the last element
<code>return target</code> or <code>return 0</code> when absent	<code>return -1</code>
unsorted input	sort first or use linear search

Note: Advanced (not a mistake in your code)

$(\text{left} + \text{right}) / 2$ can overflow for arrays with more than a billion elements. A safer form is `left + (right - left) / 2`. Mention only; your array sizes are tiny.

When to use it

- The data is **sorted** and you search many times.
- Answers like "find the position of x ", "does x exist" in sorted data.
- Not for: unsorted data, searching just once in a short array, or "find all indices" without extra work.

Recall prompts (do these before Day 29)

1. Binary Search needs a _____ array.
2. When do we move `left`? (`target < arr[middle]`)
3. When do we move `right`?
4. Why `left <= right`?
5. What does `-1` mean?
6. Dry run target 25 in {10,20,30,40,50,60,70} on paper. Then compare with the table above.

PATTERN BANK (30 patterns you can rebuild)

Note: How to use

Do not memorise code. Read the **Mental model** and **Pseudocode**, close the book, and write the Java yourself. Check with the **Dry run**. Status badges come from **MASTER_CHECKLIST.md**.

#	Pattern	Status	#	Pattern	Status
1	Assume -> Compare -> Update	●	16	alreadySeen	●
2	Traverse -> Condition -> Counter++	●	17	Frequency counting	●
3	Found -> Return	●	18	Sorted-array reasoning	●
4	-1 = Not Found	●	19	Move zeros	●
5	Accumulator	●	20	Remove duplicates (sorted)	●
6	Scanner vs Writer (i / pos)	●	21	Largest / second largest	●
7	Two pointers (left / right)	●	22	Second distinct largest	●
8	First occurrence	●	23	Asc / desc / equal / neither	●
9	Last occurrence	●	24	Binary Search	●
10	First + Last in one pass	●	25	Digit peel (%10, /10)	●
11	Count -> Allocate -> Fill	●	26	Flag boolean	●
12	Logical vs physical size	○	27	Menu loop (do-while + switch)	●
13	Early return	●	28	Nested-loop patterns	●
14	Method -> params -> processing -> return -> caller	●	29	Adjacent comparison	●
15	Method calling another method	●	30	Swap with temp / bubble	● / ○

1. Assume -> Compare -> Update

- **Purpose:** find the best element (largest, smallest, most frequent).
- **Mental model:** a champion who defends the title against each new challenger.
- **Pseudocode:** best = first; for each other x: if x beats best, best = x.
- **Java:** `int best = arr[0]; for (int i = 1; i < arr.length; i++) if (arr[i] > best) best = arr[i];`
- **Example:** largest of {22,39,12,53} is 53.
- **Dry run:** best 22 -> 39 beats -> 12 no -> 53 beats -> 53.
- **Common mistake:** best = 0 or 1000 as the assumption.
- **Complexity:** $O(n)$ time, $O(1)$ space.
- **When to use:** "largest / smallest / best / most".

2. Traverse -> Condition -> Counter++

- **Purpose:** count how many elements satisfy a rule.
- **Mental model:** a tally sheet; one stroke per match.
- **Pseudocode:** count = 0; for each x: if rule(x), count++.
- **Java:** `int count = 0; for (...) { if (arr[i] % 2 == 0) { count++; } }`
- **Example:** evens in {10,23,5,4,12,54} = 4.
- **Dry run:** 10 yes (1), 23 no, 5 no, 4 yes (2), 12 yes (3), 54 yes (4).
- **Common mistake:** count++ outside the if.

- **Complexity:** $O(n)$, $O(1)$.
- **When to use:** "how many..."

3. Found -> Return

- **Purpose:** stop as soon as the answer is known.
- **Mental model:** you found your keys, so you stop searching the house.
- **Pseudocode:** for each x : if x is the target, return now; after the loop, return the "not found" value.
- **Java:** `if (arr[i] == target) { return i; }`
- **Example:** first 43 in `{10,43,32}` returns 1.
- **Dry run:** $i = 0$ no; $i = 1$ yes -> return 1 ($i = 2$ never runs).
- **Common mistake:** `return false` / `return -1` inside the loop (it ends after the first element).
- **Complexity:** Best $O(1)$, worst $O(n)$.
- **When to use:** "does it exist / where is the first".

4. -1 = Not Found

- **Purpose:** report "no such index" safely.
- **Mental model:** indexes are 0, 1, 2... so a negative number can never be a real answer.
- **Pseudocode:** search; if the loop ends, return -1.
- **Java:** `return -1;` (after the loop)
- **Example:** index of 99 in `{10,32,53}` is -1.
- **Dry run:** all three compared, no match, loop ends, -1.
- **Common mistake:** returning 0 (looks like "found at the first slot").
- **Complexity:** same as the search.
- **When to use:** any method returning an index. Caution: do not use -1 as "no value" if -1 can be real data (Day 25 `secondDistinct`).

5. Accumulator

- **Purpose:** build up one result across the loop (sum, product, reverse).
- **Mental model:** a running total on a calculator.
- **Pseudocode:** total = neutral start; for each x : total = combine(total, x).
- **Java:** `int sum = 0; sum = sum + arr[i]; / int fact = 1; fact = fact * i;`
- **Example:** factorial of 5 = 120.
- **Dry run:** 1, 2, 6, 24, 120.
- **Common mistake:** wrong start (0 for a product).
- **Complexity:** $O(n)$, $O(1)$.
- **When to use:** sum, average numerator, factorial, power, digit reverse.

6. Scanner vs Writer (i = scanner, pos = writer)

- **Purpose:** rearrange an array in place.
- **Mental model:** i reads every card; pos is the slot where the next good card goes.
- **Pseudocode:** $pos = 0$; for each i : if keep($arr[i]$) { $arr[pos] = arr[i]$; $pos++$ }.
- **Java:** `if (arr[i] != 0) { arr[pos] = arr[i]; pos++; }`

- **Example:** move zeros; find-all-index fill.
- **Dry run:** see 03-DSA/two-pointers.md (move zeros).
- **Common mistake:** moving `pos` on every element; mixing up the two names.
- **Complexity:** $O(n)$, $O(1)$.
- **When to use:** remove / compact / filter in place.

7. Two pointers (left / right)

- **Purpose:** compare or swap pairs from both ends.
- **Mental model:** two people walking toward each other.
- **Pseudocode:** $i = 0, j = \text{last}$; while $i < j$: use the pair; $i++, j--$.
- **Java:** `int i = 0, j = s.length() - 1; while (i < j) { ...; i++; j--; }`
- **Example:** palindrome "naman"; reverse array.
- **Dry run:** (n, n) , (a, a) , meet at the centre.
- **Common mistake:** moving only one pointer; stopping with $i \leq j$ when a swap is involved.
- **Complexity:** $O(n)$, $O(1)$.
- **When to use:** reverse, palindrome, pair checks on sorted data.

8. First occurrence

- **Purpose:** position of the first match.
- **Mental model:** a bookmark that you place once and never move.
- **Pseudocode:** loop; on the first match return i .
- **Java:** see pattern 3.
- **Example:** first 10 in $\{10, 20, 10\} = 0$.
- **Dry run:** matches immediately at $i = 0$.
- **Common mistake:** not returning (it then behaves like "last").
- **Complexity:** best $O(1)$, worst $O(n)$.
- **When to use:** "first index".

9. Last occurrence

- **Purpose:** position of the last match.
- **Mental model:** a sticky note you overwrite on every match.
- **Pseudocode:** $\text{last} = -1$; every match: $\text{last} = i$; return last after the loop.
- **Java:** `int lastIndex = -1; if (arr[i] == target) { lastIndex = i; }`
- **Example:** last 10 in $\{10, 20, 10, 40, 10\} = 4$.
- **Dry run:** $\text{last} = 0$, then 2, then 4.
- **Common mistake:** returning inside the loop.
- **Complexity:** $O(n)$ always.
- **When to use:** "last index".

10. First + Last in one pass

- **Purpose:** both ends of the matches with one scan.

- **Mental model:** a first bookmark (set once) and a sticky note (overwritten).
- **Pseudocode:** first = last = -1; on a match: if first is -1, first = i; last = i.
- **Java:** see 03-DSA/searching.md #4.
- **Example:** 30 in {10,20,30,49,30,30,20,30} -> 2 and 7.
- **Dry run:** first set at i = 2; last = 2, 4, 5, 7.
- **Common mistake:** result array `new int[2]` filled only on a match, which returns {0,0} when absent.
- **Complexity:** $O(n)$, $O(1)$.
- **When to use:** range of an element in an array.

11. Count -> Allocate -> Fill

- **Purpose:** return an array whose size is not known in advance.
- **Mental model:** count the guests, order exactly that many chairs, seat them.
- **Pseudocode:** k = count(matches); result = new int[k]; pos = 0; scan again, result[pos++] = i.
- **Java:** `int ar[] = new int[count];` then `ar[pos] = i; pos++;`
- **Example:** indices of 20 in {10,20,10,20} -> [1,3].
- **Dry run:** count 2; ar size 2; fill at i = 1 and i = 3.
- **Common mistake:** hard-coding the size; looping over the result array instead of the original.
- **Complexity:** $O(n)$ time, $O(k)$ result.
- **When to use:** "return all positions / all matches".

12. Logical size vs physical array

- **Purpose:** use only the filled part of a fixed-size array.
- **Mental model:** a 5-seat bus with 3 passengers: the bus size is 5, the passenger count is 3.
- **Pseudocode:** keep count; loop i < count; block adding at `count == arr.length`.
- **Java:** `if (count < 5) { name[count] = n; count++; } for (i < count)`
- **Example:** Day 22 student list.
- **Dry run:** add 2 students: count = 2; view loops i = 0..1.
- **Common mistake:** looping to `arr.length` (shows empty slots).
- **Complexity:** $O(1)$ add, $O(\text{count})$ view.
- **When to use:** fixed arrays used as growing lists.

13. Early return

- **Purpose:** leave the method the moment the result is decided.
- **Mental model:** a bouncer who sends you home on the first rule broken.
- **Pseudocode:** if (bad) return false; ... return true.
- **Java:** `if (arr[i - 1] > arr[i]) return false; ... return true;`
- **Example:** sorted check; palindrome.
- **Dry run:** {1,3,2} fails at i = 2 -> false.
- **Common mistake:** returning the "good" answer inside the loop.
- **Complexity:** best $O(1)$, worst $O(n)$.
- **When to use:** "all must satisfy" or "any exists".

14. Method -> parameters -> processing -> return -> caller

- **Purpose:** package a job.
- **Mental model:** a vending machine: put in coins (parameters), machine works, snack comes out (return), you eat it (caller).
- **Pseudocode:** define method; call it; store the result.
- **Java:** `static int add(int a, int b) { return a + b; } int r = add(15, 25);`
- **Example:** `findSum(numbers)`.
- **Dry run:** `a = 15, b = 25 -> 40 -> r = 40`.
- **Common mistake:** forgetting `return`; printing instead of returning.
- **Complexity:** depends on the body.
- **When to use:** always.

15. Method calling another method

- **Purpose:** split a big job into small testable jobs.
- **Mental model:** a manager delegating to specialists.
- **Pseudocode:** `helper()` does one thing; `bigger()` calls `helper()`.
- **Java:** `static boolean isPalindrome(int n) { return n == reverse(n); }`
- **Example:** `mostFrequent` calls `frequency`; `charFrequency` calls `alreadySeen` and `countChar`.
- **Dry run:** `isPalindrome(121)`; `reverse(121) = 121 -> true`.
- **Common mistake:** ignoring the returned value (Day 26 re).
- **Complexity:** cost of helper times number of calls.
- **When to use:** whenever a loop body is a job by itself.

16. alreadySeen

- **Purpose:** handle each distinct value once.
- **Mental model:** a guest list checked at the door ("has this name come in before?").
- **Pseudocode:** for `i`: for `j` in `0..i-1`: if same as `i`, skip `i`.
- **Java:** `for (int j = 0; j < i; j++) { if (a[j] == a[i]) { seen = true; break; } }`
- **Example:** "banana" -> b, a, n.
- **Dry run:** full table in 02-Core-Java/strings.md.
- **Common mistake:** `j <= i` or `j < length` (skips the first copy as well).
- **Complexity:** $O(n^2)$, $O(1)$.
- **When to use:** "each distinct element once", frequency of each.

17. Frequency counting

- **Purpose:** count how often a value occurs.
- **Mental model:** a roll call: "how many times was this name called?"
- **Pseudocode:** `frequency(x)`: `count = 0`; for each `y`: if `y == x`, `count++`.
- **Java:** `static int frequency(int arr[], int target)` (Day 14).
- **Example:** 10 in {10,32,10,42,10,20,20} = 3.
- **Dry run:** three matches.

- **Common mistake:** counting before the `alreadySeen` check (wasted work).
- **Complexity:** $O(n)$ per call.
- **When to use:** "how many times", most frequent, distinct counts.

18. Sorted-array reasoning

- **Purpose:** use order to simplify a problem.
- **Mental model:** a queue sorted by height: equal people stand side by side.
- **Pseudocode:** compare each element with the one before it.
- **Java:** `if (arr[i] != arr[i - 1])`
- **Example:** remove duplicates only works because equals are neighbours; Binary Search only works because the left is smaller.
- **Dry run:** `{1,1,2}`: $i = 1$ equal (skip); $i = 2$ different (keep).
- **Common mistake:** applying the idea to unsorted data.
- **Complexity:** $O(n)$ (or $O(\log n)$ for search).
- **When to use:** the problem says sorted, or you may sort first.

19. Move zeros

- **Purpose:** push zeros to the end, keep the order of the others.
- **Mental model:** shuffle all real cards to the front, then fill the rest with blanks.
- **Pseudocode:** `pos = 0`; copy non-zeros to `pos`; fill from `pos` to end with 0.
- **Java:** see `03-DSA/two-pointers.md #3`.
- **Example:** `{10,0,40,0,10}` -> `{10,40,10,0,0}`.
- **Dry run:** table in two-pointers.
- **Common mistake:** restarting the fill loop at 0.
- **Complexity:** $O(n)$, $O(1)$.
- **When to use:** compact / partition in place.

20. Remove duplicates from a sorted array

- **Purpose:** keep one copy of each value; return the new length.
- **Mental model:** Scanner/Writer where a value is kept if it differs from its left neighbour.
- **Pseudocode:** `pos = 1`; for i from 1: if `arr[i] != arr[i-1]`, `arr[pos++] = arr[i]`.
- **Java:** see two-pointers #4.
- **Example:** `{1,1,2,2,3}` -> first 3 = `1 2 3`.
- **Dry run:** table in two-pointers.
- **Common mistake:** unsorted input; empty array.
- **Complexity:** $O(n)$, $O(1)$.
- **When to use:** "unique values of a sorted array".

21. Largest / second largest

- **Purpose:** top one or top two.
- **Mental model:** a podium with gold and silver; a newcomer may push gold to silver.
- **Pseudocode:** order the first two, then for $i \geq 2$ shift or replace.

- **Java:** see searching #5.
- **Example:** {23,53,14,65,-13,43} second = 53.
- **Dry run:** gold 53, silver 23; 65 pushes 53 to silver.
- **Common mistake:** loop starting at 0; second = arr[0] start.
- **Complexity:** $O(n)$, $O(1)$.
- **When to use:** "second best / top 2".

22. Second distinct largest

- **Purpose:** the second biggest value that is different from the biggest.
- **Mental model:** a podium where silver must be strictly below gold; a repeat of gold does not get silver.
- **Pseudocode:** largest = first; second = -infinity; found = false; update gold or silver (only if second < x < largest).
- **Java:** see searching #6 and 08-Repair.
- **Example:** {10,120,10,10,10} -> 10.
- **Dry run:** 120 pushes 10 to silver; remaining 10s change nothing.
- **Common mistake:** letting a duplicate of the maximum become the second; forgetting the "no second" case.
- **Complexity:** $O(n)$, $O(1)$.
- **When to use:** "second distinct", "second highest salary" style questions.

23. Ascending / descending / equal / neither

- **Purpose:** classify the order of an array.
- **Mental model:** two switches; any rise turns off "descending", any drop turns off "ascending".
- **Pseudocode:** both flags true; adjacent compare; read the final flag combination.
- **Java:** see searching #10.
- **Example:** {50,50,50} equal; {1,3,2} neither.
- **Dry run:** table in searching #10.
- **Common mistake:** mixing up equal (both true) and neither (both false).
- **Complexity:** $O(n)$, $O(1)$.
- **When to use:** order checks, sorted checks.

24. Binary Search

- **Purpose:** find a value in a sorted array fast.
- **Mental model:** open a dictionary in the middle and throw away half each time.
- **Pseudocode:** left, right; while window not empty: look at middle; found / go right / go left; else -1.
- **Java:** see 03-DSA/binary-search.md.
- **Example:** 60 in {10..70} -> 5.
- **Dry run:** two rounds in the file.
- **Common mistake:** unsorted data; middle not recalculated; comparing with middle not arr[middle].
- **Complexity:** best $O(1)$, avg and worst $O(\log n)$, space $O(1)$.
- **When to use:** sorted + lookup.

25. Digit peel (%10 and /10)

- **Purpose:** take a number apart digit by digit.
- **Mental model:** peeling an onion from the right.
- **Pseudocode:** while num > 0: digit = num % 10; use digit; num = num / 10.
- **Java:** `int digit = num % 10; num = num / 10;`
- **Example:** sum of digits of 1234 = 10.
- **Dry run:** 4, 3, 2, 1.
- **Common mistake:** input 0 (loop never runs).
- **Complexity:** $O(d)$ where d is the digit count.
- **When to use:** digit sum, reverse, palindrome number, count of evens.

26. Flag boolean

- **Purpose:** remember "did it happen at all?"
- **Mental model:** a light switch, off at the start, turned on by evidence.
- **Pseudocode:** found = false; if match, found = true; use after the loop.
- **Java:** `boolean found = false; ... found = true;`
- **Example:** `findAllIndex` prints "Element not found" only if the flag stayed false.
- **Dry run:** no match -> flag false -> message.
- **Common mistake:** printing "not found" inside the loop (`Array.java`).
- **Complexity:** $O(n)$, $O(1)$.
- **When to use:** result is "yes/no" after scanning everything.

27. Menu loop (do-while + switch)

- **Purpose:** an interactive program with options.
- **Mental model:** a restaurant menu that repeats until you say exit.
- **Pseudocode:** do { show; read; switch } while (choice != exit).
- **Java:** `do { ... choice = sc.nextInt(); switch (choice) {...} } while (choice != 3);`
- **Example:** Number Analyzer, String Analyzer.
- **Dry run:** choice 1 -> action; choice 3 -> loop ends.
- **Common mistake:** missing `break`; unfinished options; leftover Enter after `nextInt`.
- **Complexity:** depends on the actions.
- **When to use:** console menus.

28. Nested-loop patterns

- **Purpose:** print shapes.
- **Mental model:** outer = rows, inner = columns; decide spaces and symbols per row.
- **Pseudocode:** for row: spaces; symbols; newline.
- **Java:** `for (i..) { for (j..) { print } println(); }`
- **Example:** pyramid n - i spaces + $2*i - 1$ stars.
- **Dry run:** $n = 3$: row 1: 2 spaces, 1 star.
- **Common mistake:** forgetting `println()` after the inner loop.

- **Complexity:** $O(n^2)$.
- **When to use:** shapes, tables, pair generation.

29. Adjacent comparison

- **Purpose:** compare each element with the one next to it.
- **Mental model:** checking a line of people one pair at a time.
- **Pseudocode:** for i from 1: compare $arr[i-1]$ and $arr[i]$.
- **Java:** `arr[i - 1] > arr[i]`
- **Example:** sorted check, classification, remove duplicates, bubble sort.
- **Dry run:** `{1,3,2}`: (1,3) fine, (3,2) violation.
- **Common mistake:** starting at 0 and reading `arr[-1]`.
- **Complexity:** $O(n)$.
- **When to use:** order or change detection.

30. Swap with temp (and bubble sort)

- **Purpose:** exchange two values safely.
- **Mental model:** two glasses of juice need a third empty glass.
- **Pseudocode:** $temp = a; a = b; b = temp$.
- **Java:** `int temp = arr[j + 1]; arr[j + 1] = arr[j]; arr[j] = temp;`
- **Example:** reverse; bubble sort (Day 29, ○).
- **Dry run:** swap 39 and 10 $\rightarrow$ 10, 39.
- **Common mistake:** `a = b; b = a;` (both become `b`).
- **Complexity:** swap $O(1)$; bubble sort $O(n^2)$.
- **When to use:** reorder in place.

ACTIVE RECALL SYSTEM

Tip: How to use

Cover the answer. Say or write it. Check. Wrong answers go on a "redo tomorrow" list. Do not reread the book every day. Tired? Do the 5-minute level only.

Level 1 - 5-minute recall (answers at the end of the section)

1. What does `% 10` give?
2. Why do we use `/ 10`?
3. What does `-1` represent in `findIndex`?
4. What is `i` in `move zeros`?
5. What is `pos`?
6. Why `j < i` in `alreadySeen`?
7. Why must Binary Search use a sorted array?
8. When do we move `left`?
9. When do we move `right`?
10. Why does `firstIndex` stop changing?
11. Why does `lastIndex` keep changing?
12. Why start `largest` with `arr[0]`?
13. `.length` or `.length()` for a String?
14. `==` or `equals` for text?
15. What does `return` do?

Answers: 1 last digit. 2 removes the last digit. 3 not found (never a real index). 4 scanner (reads every element). 5 writer (next slot for a kept value). 6 only earlier characters count as "already seen". 7 the discard-half decision assumes order. 8 `target > arr[middle]` (`left = middle + 1`). 9 `target < arr[middle]` (`right = middle - 1`). 10 it returns at the first match. 11 each match overwrites it. 12 any real element is a safe start (works for negatives). 13 `length()` for String, `length` for arrays. 14 `equals`. 15 sends a value back and ends the method.

Level 2 - 15-minute recall (moderate reconstruction)

#	Task (blank page)	Check against
1	Write <code>sumDigits(int num)</code>	<code>loops.md</code>
2	Write <code>reverse-number</code> and <code>palindrome-number</code>	<code>loops.md</code>
3	Write <code>findIndex</code> returning <code>-1</code>	<code>searching #1</code>
4	Write <code>lastIndex</code>	<code>searching #2</code>
5	Write <code>countChar(String, char)</code>	<code>strings.md</code>
6	Write <code>alreadySeen(String, int)</code>	<code>strings.md</code>
7	Dry run "banana" through <code>charFrequency</code> (table)	<code>strings.md</code>
8	Write <code>moveZero</code> and dry run <code>{0,5,0,7}</code>	<code>two-pointers #3</code>
9	Write <code>sorted(int[])</code>	<code>searching #9</code>
10	Write Binary Search skeleton (variables + 3 branches)	<code>binary-search.md</code>

Mini-questions:

- Why does the reverse-array loop stop at `i < j`?
- What do you return from `removeDuplicates` and what does it mean?
- Why is `middle` inside the loop?
- What does `new int[count]` rely on? (a first pass that counts)
- Why do digit methods not need `temp` copies?
- Which parameter types can a method change in the caller? (arrays, not primitives)

Level 3 - 30-minute revision (mixed problems)

Pick any 4: write from blank, dry run, state complexity.

Problem	Pattern to name first
Second distinct largest of <code>{30,10,20}</code>	podium: <code>second < x < largest</code>

Problem	Pattern to name first
All indices of 20 in {10,20,10,20,30}	count -> allocate -> fill
First and last of 10 in {10,20,10,40,10}	first + last in one pass
Remove duplicates in {1,1,2,3,3}	scanner / writer + neighbours
Palindrome with toLowerCase ("MAdam")	two pointers
Classify {9,7,7,3}	two flags
Binary Search: 25 in {10,...,70}	window halving
Most frequent in {5,5,6,6,6,7}	assume -> compare -> update + frequency

Level 4 - Full revision checkpoint (once a week)

A. Java core: data types, %, integer division, switch/break, loops, digit peel, methods (return vs void), pass-by-value, array vs String length. **B. Arrays:** traversal, sum, average, largest, smallest, second largest, second distinct, contains, count, first, last, all, first+last, reverse, move zeros, remove duplicates, sorted, classify, most frequent, bubble sort preview. **C. Strings:** charAt, vowel/digit/space, reverse, palindrome, countChar, alreadySeen, charFrequency. **D. Binary Search:** write, dry run found / not found, explain sorted rule, complexity.

Score yourself: ● wrote it from blank and dry-ran it, ● wrote it with one hint, ● needed the book.

Question bank by topic

Digits: What does `num % 10` return for 4821? (1) What is `num / 10` for 4821? (482) Why `while (num > 0)`? What does `reverse = reverse * 10 + digit` do for 123? Why keep `orig` for a palindrome? Why does `smallest-digit` start at 9?

Methods: Where does a return value go? Which is better for reuse: print or return? Why does `reverse(int arr[])` change `main`'s array? Can `isPalindrome(int num)` change `num` in `main`?

Arrays: Last valid index? Why is `largest = 0` risky? What does `second = largest` do? Why does `second largest` loop from 2? Why `j = i + 1` in duplicates?

Search: Why return -1? Difference between `contains` and `index`? Why no early return in last index? Why count before `new int[count]`? What does `firstLast` return when the target is absent in your Day 27 code? ({0,0}, which is a bug.)

Two pointers: When do reverse's pointers stop? What does `pos` mean in `remove duplicates`? Why `pos = 1`? Why does it need sorted data?

Strings: "a" vs 'a'? What does `indexOf` return when absent? Why `equals`? Walk "banana" through `alreadySeen`. What if the loop were `j <= i`?

Binary Search: Condition to continue the loop? Why +1 and -1? What does `left > right` mean? Complexity best/avg/worst? How many rounds for 1000 elements (about 10)?

Complexity: Why is `mostFrequent` $O(n^2)$? Why is `move zeros` $O(1)$ space? Why is `Binary Search` $O(\log n)$?

Spaced-recall calendar (suggestion)

Day after learning	Do
+1	2-minute blank-page recall
+3	write it again, no peeking
+7	mixed problem (Level 3)
+21	checkpoint (Level 4)

Note: Current queue: Binary Search, second distinct largest, alreadySeen/charFrequency, firstLast not-found case.

BLANK PAGE RECONSTRUCTION

Tip: The rule

For each item: (1) read **ONLY** the problem, (2) name the pattern, (3) list variables, (4) write pseudocode, (5) only then open the reveal. Do **not** scroll to the answer first. Reveals are in the "Reveal" lines (one hint each) - the full code lives in the linked chapters.

The 5-step method (use it on every problem)

1. **PROBLEM** say it in one sentence, with an example input and output
2. **PATTERN** which pattern from the Pattern Bank does this look like?
3. **VARIABLES** what must I remember while walking? (count / best / flag / pos / left, right)
4. **PSEUDOCODE** 4 to 6 plain-English lines
5. **CODE** translate line by line, then DRY RUN

Level 1 - Easy

#	Problem	Pattern?	Variables?	Reveal (hint only)
1	Sum of digits of 4821	digit peel + accumulator	sum, digit	%10, /10, sum += digit
2	Largest in {-5, -2, -9}	assume -> compare -> update	largest	start with arr[0]
3	Is 7 in {3, 7, 9}?	found -> return	none	return true inside, false after
4	Index of 9 or -1	found -> return, -1	i	return i, then return -1
5	Count vowels in "education"	traverse -> condition -> count	count	"aeiou".indexOf(ch) != -1
6	Reverse {1, 2, 3, 4} in place	two pointers + temp	i, j, temp	while i < j swap
7	Reverse a number 123 -> 321	digit peel	reverse	reverse*10 + digit

Level 2 - Familiar

#	Problem	Pattern?	Variables?	Reveal (hint only)
8	Last index of 10 in {10, 20, 10}	last occurrence	lastIndex = -1	overwrite on every match
9	Move zeros: {0, 3, 0, 5}	scanner / writer	i, pos	copy non-zeros, fill zeros from pos
10	Palindrome word	two pointers	i, j	mismatch -> false
11	Sorted check {1, 2, 2, 5}	adjacent comparison	none	compare arr[i-1] > arr[i]
12	Count of each distinct char in "banana"	alreadySeen + countChar	alreadySeen	inner loop j < i
13	Second largest of {23, 53, 14, 65}	podium (two variables)	largest, second	loop from i = 2
14	Most frequent in {4, 4, 7, 7, 7}	frequency + best	best, highest	frequency() helper

Level 3 - Combination

#	Problem	Combine	Reveal (hint only)
15	All indices of the maximum value	largest + count/allocate/fill	find max first, then count -> fill
16	First and last index of the most frequent element	most frequent + first/last	two passes, reuse methods
17	Sort with bubble then Binary Search for a target	bubble sort + Binary Search	sort first, then search; mention cost
18	Remove duplicates, then print the unique prefix only	remove duplicates + traversal	print i < returnedLength

#	Problem	Combine	Reveal (hint only)
19	Is the string a palindrome ignoring spaces and case?	two pointers + skipping	skip spaces on each side
20	Count distinct values in an unsorted array	alreadySeen	count where !alreadySeen
21	Palindrome number using your own <code>reverse(int)</code>	method calling method	<code>return n == reverse(n)</code>

Level 4 - Unseen variation (you have not written these yet)

#	Problem	Think about
22	Third distinct largest	one more variable; podium with bronze
23	Second distinct smallest	mirror the podium
24	Rotate an array by 1 (last element goes first)	scanner/writer or temp, which direction?
25	Count how many numbers in a sorted array are less than a target	Binary Search variation: what do you return when not found?
26	First index where the sorted array value is $\geq$ target	Binary Search that keeps a candidate
27	Merge two sorted arrays	two scanners, one writer
28	Check if an array is a palindrome	two pointers on an array

Note: Levels 3 and 4 are practice targets, not facts from your ZIP. Treat them as the next steps. They are not marked as learned anywhere in this book.

Worked example (to copy the way of thinking)

Problem: Return how many times each number appears, but print each once. {10,32,10,42}.

1. **PROBLEM:** 10 > 2, 32 > 1, 42 > 1.
2. **PATTERN:** alreadySeen + frequency.
3. **VARIABLES:** alreadySeen (flag), count.
4. **PSEUDOCODE:**

```
for each i:
    seen = false
    for j from 0 to i-1: if num[j] == num[i], seen = true
    if not seen: count = frequency(num, num[i]); print
```
5. **CODE:** Day 14 `day14.java` (check after your own attempt).

Self-grading sheet

Date	Problem #	Pattern named?	Variables right?	Code from blank?	Dry run?	Grade

MASTER CHECKLIST (status from Day 1 to Day 28)

Legend: [x] done with evidence in the ZIP, [~] introduced / partly, [] future (not in the ZIP). Badges: ● ● ●
○.

CORE JAVA

Done	Topic	Badge	Evidence
[x]	Program skeleton, main , output	●	hello.java
[x]	Scanner input (int, double, String, char)	●	many files
[x]	Data types (int, long, double, boolean, char, String)	●	used; float/casting repeated
[x]	Operators, %, integer division	●	cast bug in project1
[x]	Conditions (if / else-if)	●	conditional.java, Day 9
[x]	switch	●	switch1 break bug, later menus fine
[x]	for / while / do-while	●	loop.java, Day 16
[x]	break / continue	●	loop1.java once
[x]	Nested loops and patterns	●	pattern.java, adpattern.java
[x]	Digit programs	●	Days 5, 6, 7, 9, 16, 18
[x]	Methods (void / return / params)	●	Days 6 to 29
[x]	Method calling method	●	Days 14 to 21
[x]	Arrays (declare, traverse, pass to method)	●	Days 8 to 29
[x]	Strings (length, charAt, equals, toLowerCase, indexOf)	●	Days 19 to 21, 26
[x]	Menu programs (do-while + switch)	●	Days 16, 18, 21; Day 22 unfinished
[~]	Parallel arrays + logical size	○	Day 22 once
[~]	Pass-by-value / reference behaviour	●	used in reverse/moveZero, never explained by you
[~]	static fields	○	static int count Day 22 only
[]	OOP (classes, objects, inheritance, polymorphism)	FUTURE	not in ZIP
[]	Constructors, this , access modifiers	FUTURE	not in ZIP
[]	Exceptions	FUTURE	not in ZIP
[]	Collections (List, Map, Set)	FUTURE	not in ZIP
[]	Generics	FUTURE	not in ZIP
[]	File handling	FUTURE	not in ZIP
[]	Multithreading	FUTURE	not in ZIP
[]	StringBuilder, wrapper classes, 2D arrays	FUTURE	not in ZIP
[]	Recursion	FUTURE	not in ZIP

DSA

Done	Topic	Badge	Notes
[x]	Array traversal	●	
[x]	Sum / average	●	
[x]	Largest / smallest	●	fixed Day 10
[x]	Second largest	●	latent bugs Day 13, 23
[x]	Second distinct largest	●	buggy Day 23, correct Day 25, forgotten

Done	Topic	Badge	Notes
[x]	Contains / linear search	●	
[x]	Count occurrence	●	
[x]	First index	●	
[x]	Last index	●	Days 27, 29
[x]	All indices (count -> allocate -> fill)	●	Days 22, 27, 29
[x]	First + last in one pass	●	not-found bug remains
[x]	Reverse array (two pointers)	●	Days 11 to 16
[x]	Move zeros	●	Days 23, 24, 25
[x]	Remove duplicates (sorted)	●	Days 24, 25
[x]	Sorted check	●	Day 24 once
[x]	Ascending / descending / equal / neither	●	Day 24 once
[x]	Frequency / alreadySeen	●	spaced recall
[x]	Most frequent	●	Days 15, 25
[x]	Duplicate (nested pairs)	○	Day 13 once
[x]	String palindrome (two pointers)	●	bug fixed Day 20
[x]	Binary Search	●	Day 28, dry run pending
[~]	Bubble sort	○	Day 29 only
[]	Selection / insertion / merge / quick sort	FUTURE	not in ZIP
[]	Recursion, hashing, stacks, queues, linked lists, trees	FUTURE	not in ZIP

FULL STACK (all future)

Topic	Status
SQL, JDBC	FUTURE
HTML / CSS / JavaScript / React	FUTURE
Spring, Spring Boot, REST	FUTURE
JPA / Hibernate	FUTURE

Note: These full-stack topics are on your plan, but nothing in JAVA.zip covers them, so they stay unchecked. (01.py is a Python turtle drawing and is not part of this list.)

Skills checklist (not topics)

Skill	Status
Write a method from a blank page	● (unverified)
Dry run on paper	● (Day 28 dry run not done)
Name the pattern before coding	●
Test edge cases (empty, absent, all equal, negatives)	●
Delete unused variables	●

COMMON MISTAKES / BUG BOOK (from your real code)

Each entry: ✗ wrong idea, why it is wrong, ✓ correct idea, mental rule. File names show where it happened.

#	Bug	Where	Fixed later?
1	Wrong start value (0, 1000)	Day 7, 8, 9	Yes, Day 10
2	Comparing with the wrong variable	pluseven.java	Yes, Day 9
3	Missing break in switch	switch1.java	Later menus fine
4	Integer division	project1.java	Casts used from Day 11
5	"Not found" printed in the loop	Array.java	Day 11 uses return -1
6	Second largest loop starts at 0	Day 13	Yes, Day 14
7	second = arr[0] start	Day 23	Day 25
8	Dead else if	Day 23 secondDistinct	Day 25
9	Inverted palindrome booleans	Day 20 attempt 1	Same day
10	Result array filled only on match	Day 27 firstLast	Not yet
11	Calling a method, ignoring the result	Day 26	Not yet
12	nextInt() then nextLine()	Day 22	Day 21 had the fix
13	Unfinished menu options	Day 22	Not yet
14	Unsorted array for Binary Search	Day 28 test data	Use sorted data
15	Equal vs neither	Day 24	Handled
16	Overflow in factorial	Day 16, 18	Not yet
17	Unnecessary variables	Day 7, 18, 26	Partly
18	alreadySeen bounds	Day 14, 20, 26	Correct everywhere, keep revising

1. Wrong start value

- ✗ `int largest = 0; or int smallest = 1000;`
- Why: with `{-5, -2, -9}` nothing beats 0, so the answer is 0 (not in the array). 1000 fails for bigger values.
- ✓ `int largest = arr[0]; and loop from i = 1.`
- Rule: Start from a real element of the data.

2. Testing the input instead of the loop variable

- ✗ `for (int i = 0; i <= num; i++) { if (num % 2 == 0) sum += i; }`
- Why: num never changes inside the loop.
- ✓ `if (i % 2 == 0)`
- Rule: Inside the loop, use i or arr[i].

3. Missing break

- ✗ `case '*': System.out.println(a*b); with no break.`
- Why: falls through to default, prints "Invalid operation" too.
- ✓ `break;` after every case.

- **Rule:** Every case ends with `break` (or a deliberate fall-through comment).

4. Integer division

- **X** `double p = (m1 + m2 + m3) * 100 / 300;`
- **Why:** `int` arithmetic finishes first (85.67 becomes 85).
- **✓** `* 100.0 / 300` or `(double) total / n`
- **Rule:** Cast **before** dividing. (You did this correctly in Days 11, 12 and 23.)

5. "Not found" inside the loop

- **X** `Array.java: else { System.out.println("Element not found"); } inside the for.`
- **Why:** prints for every non-matching element, even when the target exists later.
- **✓** `return -1;` after the loop (or a `found` flag checked after the loop).
- **Rule:** You only know it is missing after checking **all** elements.

6. Second largest loop from 0

- **X** `for (int i = 0; ...) after initialising with arr[0], arr[1].`
- **Why:** {50, 10, 30}: largest 50, second 10; at `i = 0`, `50 > second` is true so `second = 50` (equal to largest, wrong).
- **✓** `for (int i = 2; ...).`
- **Rule:** Start after the elements you already used.

7. `second = arr[0]` start

- **X** `int target = arr[0]; int second = arr[0];`
- **Why:** {30, 10, 20} returns 30: `second` starts equal to the largest (30), and 10 and 20 are never greater than 30, so `second` never changes.
- **✓** Order the first two, or use `MIN_VALUE` plus a flag (Day 25).
- **Rule:** Initialise `second` to something that can never be a real answer.

8. Dead `else if`

- **X** `if (x < s) {...} else if (x < s && x != s) {...}`
- **Why:** the second test needs `x < s`, which the `if` already handled.
- **✓** Describe a different case in the `else if` (`x > second && x < largest`).
- **Rule:** An `else if` must be something the `if` did not catch.

9. Inverted palindrome booleans

- **X** `mismatch -> return true; end -> return false; only i++.`
- **Why:** reversed meaning, and `j` never moves.
- **✓** `mismatch -> false; end -> true; i++ and j--.`
- **Rule:** One failure means false; zero failures means true.

10. Result array filled only on a match

- **✗** `int ar[] = new int[2];` then `ar[0] = firstIndex;` inside the `if`.
- **Why:** if the target is absent the array stays `{0,0}`, which looks like "found at index 0".
- **✓** `return new int[]{firstIndex, lastIndex};` after the loop (gives `{-1,-1}`).
- **Rule:** Never let a default value look like a real answer.

11. Calling a method but not using its return value

- **✗** `int re = frequency(str, ch);` (unused), result computed twice.
- **✓** Call once, use the value.
- **Rule:** Every returning call feeds something.

12. Hard-coding the result size / looping the wrong array

- **✗** `int ar[] = new int[5];` or `for (i < ar.length) if (ar[i] == target)`
- **Why:** the number of matches is unknown; you must scan the original.
- **✓** Count first (`new int[count]`), loop over `arr`, write with `pos`.
- **Rule:** Count -> Allocate -> Fill.

13. Confusing `i` and `pos`

- **✗** `arr[i] = arr[pos]; i++;` or `pos++` on every element.
- **✓** `arr[pos] = arr[i]; pos++;` only for kept values.
- **Rule:** `i` scans, `pos` writes.

14. Forgetting to recalculate `middle` / comparing with the wrong thing

- **✗** `middle` computed before the loop; `if (target > middle).`
- **✓** Inside `while`: `middle = (left+right)/2;` compare with `arr[middle]`.
- **Rule:** The window changes, so the middle changes.

15. Using `arr[0]` when an index is required

- **✗** `return arr[0];` as "the position".
- **✓** `return i;` (a position) vs `return arr[i];` (a value).
- **Rule:** Ask "do they want the index or the value?"

16. Returning `target` instead of `-1`

- **✗** `return target;` when not found.
- **Why:** looks like a real answer.
- **✓** `return -1;`

17. Forgetting `return`

- **✗** a method with `int` return type that has no `return` on some path (compile error).
- **✓** every path ends in `return`.

18. Wrong boolean logic: equal vs neither

- **✗** treating "not ascending and not descending" as equal.
- **Why:** {50,50,50} is **both** ascending and descending (flags both true) -> equal. {1,3,2} has both flags false -> neither.
- **✓** ascending && descending -> equal; !ascending && !descending -> neither.

19. Duplicate handling

- **✗** second largest that counts duplicates ({35, 35, 10} -> 35).
- **✓** distinct version: second < x < largest.
- **✗** remove duplicates on an unsorted array.
- **✓** sort first, or use a different method.

20. Overflow

- **✗** int fact for 13! or more; Day 16 tried the factorial of the entered **number itself** (for 12345, impossible).
- **✓** long up to 20!, and only small n.

21. nextInt() then nextLine()

- **✗** student name reads as empty.
- **✓** extra `sc.nextLine();` after `nextInt()`.

22. Unnecessary variables

- **✗** orig, count, int re, String str1 named without meaning.
- **✓** every variable has a job; remove the rest.

23. Unfinished features

- **✗** menu shows "Search" and "Statistics" with no code.
- **✓** remove them or implement before saving.

24. Testing Binary Search on an unsorted array

- **✗** {10, 20, 42, 10, 49, 38} (unsorted), searching 38 gives -1 although it exists.
- **✓** sorted data, e.g. {10,20,30,40,50,60,70}.
- **Rule:** Sorted first, or Binary Search lies.

25. alreadySeen bounds

- **✗** `j <= i` (everything seen) or `j < length` (first copy skipped too).
- **✓** `j < i`.
- **Rule:** Only the past counts as "already".

Your bug habits (pattern across 29 days)

Habit	Count	Prevention
Wrong initial value	5	Ask "what if all are negative / all equal / absent?"

Habit	Count	Prevention
Default value looks like an answer	3	Use -1 and a flag
Loop bounds off by one	3	Write the first and last iteration by hand
Unused or confusing variables	4	Delete after finishing

Tip: Before you save any file

1. Smallest input (empty / one element / all equal)? 2. Negatives? 3. Target absent? 4. Did I use every returned value?

COMPLEXITY CHEAT SHEET (Big-O with the "why")

Beginner rules (read once)

Rule	Meaning
Count loops over the input	one loop over n items = $O(n)$
Loop inside a loop	$O(n \times n) = O(n^2)$
Halving each step	$O(\log n)$
Constant work	$O(1)$
Ignore small stuff	$2n$ and $n+5$ are both just $O(n)$
Space	extra memory you create (not the input)

$n = 1,000$ items: $O(1)=1$ $O(\log n)=10$ $O(n)=1,000$ $O(n^2)=1,000,000$

Best / Average / Worst: best = luckiest input, worst = unluckiest, average = typical.

Master table

Algorithm	Best	Average	Worst	Space	Why
Sum / average / count / traversal	$O(n)$	$O(n)$	$O(n)$	$O(1)$	one pass, must see all
Largest / smallest	$O(n)$	$O(n)$	$O(n)$	$O(1)$	every element may be the max
Second largest	$O(n)$	$O(n)$	$O(n)$	$O(1)$	one pass, two variables
Second distinct largest	$O(n)$	$O(n)$	$O(n)$	$O(1)$	one pass, three variables
Linear search (contains / first index)	$O(1)$	$O(n)$	$O(n)$	$O(1)$	best: first element; worst: last or absent; average: about $n/2$
Last index	$O(n)$	$O(n)$	$O(n)$	$O(1)$	cannot stop early
Find all indices (count + fill)	$O(n)$	$O(n)$	$O(n)$	$O(k)$	2 passes = $2n$, result array has k items
First + last in one pass	$O(n)$	$O(n)$	$O(n)$	$O(1)$	single pass
Reverse array	$O(n)$	$O(n)$	$O(n)$	$O(1)$	$n/2$ swaps
Move zeros	$O(n)$	$O(n)$	$O(n)$	$O(1)$	scan once + fill the tail
Remove duplicates (sorted)	$O(n)$	$O(n)$	$O(n)$	$O(1)$	one scan comparing neighbours
Sorted check	$O(1)$	$O(n)$	$O(n)$	$O(1)$	best: first pair already out of order
Asc / desc / equal / neither	$O(n)$	$O(n)$	$O(n)$	$O(1)$	one pass, two flags
Duplicate (nested, first repeat)	$O(1)$	$O(n^2)$	$O(n^2)$	$O(1)$	best: first two equal; worst: compare all pairs
frequency() of one value	$O(n)$	$O(n)$	$O(n)$	$O(1)$	one scan
Frequency of each element (alreadySeen)	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	for each i : scan up to i , then count over n
Most frequent using frequency()	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	n calls $\times O(n)$
String traversal / vowels / digits / spaces	$O(n)$	$O(n)$	$O(n)$	$O(1)$	one pass
String palindrome (two pointers)	$O(1)$	$O(n)$	$O(n)$	$O(1)$	best: first and last differ
String reverse with +	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(n)$	each + copies the string
charFrequency	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	same shape as array frequency
Binary Search	$O(1)$	$O(\log n)$	$O(\log n)$	$O(1)$	halves the window every round
Bubble sort (as written)	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	about $n(n-1)/2$ comparisons, no early stop
Digit programs	$O(d)$	$O(d)$	$O(d)$	$O(1)$	d = number of digits (about $\log_{10} n$)
Factorial / power / table	$O(n)$	$O(n)$	$O(n)$	$O(1)$	n multiplications
GCD (try every i)	$O(\min(a,b))$	$O(\min(a,b))$	$O(\min(a,b))$	$O(1)$	tries each i up to the smaller number

Algorithm	Best	Average	Worst	Space	Why
Fibonacci n terms	$O(n)$	$O(n)$	$O(n)$	$O(1)$	n additions, two variables
Star / pyramid patterns	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	rows x columns

The derivations you must be able to say

Linear search

Best $O(1)$: target is arr[0] -> 1 comparison
 Worst $O(n)$: target is last or absent -> n comparisons
 Average $O(n)$: about $n/2$ comparisons, and constants are ignored

Binary Search

$n=64 \rightarrow 32 \rightarrow 16 \rightarrow 8 \rightarrow 4 \rightarrow 2 \rightarrow 1$: 6 halvings = $\log_2(64)$
 Each round does constant work (one compare, two assignments).
 Total = (number of rounds) x (constant) = $O(\log n)$.
 Best $O(1)$: the very first middle is the target.

Move zeros: $O(n)$ time, $O(1)$ space

Pass 1: i runs n times (n steps)
 Pass 2: fill from pos to end (at most n steps)
 Total $\leq 2n \rightarrow O(n)$. Only pos, i, temp -> $O(1)$ space.

Remove duplicates from sorted array: $O(n)$, $O(1)$ - one loop, one compare per element, writes in place.

FindAllIndex using count + fill

Pass 1 (count): n Pass 2 (fill): n -> $2n = O(n)$ time
 New array has k slots (k = matches) -> $O(k)$ space (the output itself).

****Most frequent with nested frequency() -> $O(n^2)$ ****

mostFrequent loops $i = 1..n-1$ -> n iterations
 each iteration calls frequency(), which loops n -> n work
 $n \times n = n^2$

alreadySeen + frequency -> $O(n^2)$

for each i (n times):
 alreadySeen scans $j < i$ -> up to n
 countChar scans the string -> n
 total about $n \times 2n = O(n^2)$

Where could we do better? (preview only - not in your ZIP yet)

Slow now	Idea	Status
frequency of each element $O(n^2)$	store counts in a lookup structure	FUTURE (Collections)
bubble sort $O(n^2)$	faster sorting algorithms	FUTURE
String + in loops	builder class	FUTURE

Note: Do not learn these yet. They stay FUTURE until you reach them.

REPAIR LAB - Second distinct largest

"Bro I forgot second distinct largest."

Good. We repair **only this pattern** using the same book structure. You do not restart Java.

Status: ● (buggy attempt on Day 23, correct version on Day 25, and you told me you forgot it).

1. What the three versions mean

Name	Question	{10, 32, 10, 32, 35, 35, 10}	{5, 5, 5}
Largest	biggest value	35	5
Second largest (Day 10 to 15)	value that finishes 2nd when duplicates count separately	35 (the second 35 counts as 2nd)	5
Second distinct largest	biggest value strictly below the largest	32	none (-1)

2. Mental model (one picture)

A podium: GOLD = the largest value
SILVER = the biggest value that is NOT equal to gold

New number x arrives:

x > gold -> old gold drops to silver, x becomes gold
silver < x < gold -> x becomes silver
otherwise -> ignore it (x == gold, or x <= silver)

The magic phrase: **silver < x < gold**. The "strictly below gold" half is what makes it distinct.

3. Build it from a blank page (4 steps)

Step	Question to ask yourself	Answer
1 PROBLEM	What do I return, and when is there no answer?	the value, or -1 if all values are equal
2 PATTERN	Which known pattern?	Assume -> Compare -> Update with two variables (+ sentinel + flag)
3 VARIABLES	What must I remember?	largest, second, secondFound
4 PSEUDOCODE	What happens per element?	the three-way rule above

```
largest = arr[0]
second = smallest possible int
secondFound = false
for each x in the array:
    if x > largest:    second = largest; largest = x; secondFound = true
    else if x > second and x < largest:    second = x; secondFound = true
if secondFound is false: return -1
return second
```

4. Java (your Day 25 version = the recommended version)

```
static int secondDistinct(int arr[]) {
    int largest = arr[0];
    int second = Integer.MIN_VALUE;
    boolean secondFound = false;
    for (int i = 0; i < arr.length; i++) {
        if (arr[i] > largest) {
            second = largest;
            largest = arr[i];
            secondFound = true;
        } else if (arr[i] > second && arr[i] < largest) {
```

```

        second = arr[i];
        secondFound = true;
    }
}
if (!secondFound) {
    return -1;
}
return second;
}

```

5. Dry runs

****{10, 120, 10, 10, 10}****

i	x	largest	second	found	what happened
start	-	10	MIN	false	assume first is gold
0	10	10	MIN	false	not > largest; 10 < 10 false -> ignore
1	120	120	10	true	new gold, old gold drops to silver
2	10	120	10	true	10 > 10 false -> ignore
3, 4	10	120	10	true	ignore

Answer: **10**.

****{30, 10, 20}**** (first element is the largest - the case that broke Day 23)

i	x	largest	second	found
start	-	30	MIN	false
0	30	30	MIN	false
1	10	30	10	true (10 is between MIN and 30)
2	20	30	20	true (20 > 10 and 20 < 30)

Answer: **20**.

****{5, 5, 5}****: no element is above 5, and no element is strictly below 5, so **secondFound** stays false -> **-1**.

6. Earlier attempt -> Problem -> Corrected

	Day 23 secondLargest	Day 23 secondDistinct	Day 25 secondDistinct
Start	second = arr[0]	smallest = arr[0], second = MAX	second = MIN_VALUE, flag
Problem	wrong when arr[0] is the max ({30,10,20} -> 30)	smallest-based; else if is dead code ; {3,5,8} -> -1	none for normal data
Verdict	bug	bug	recommended

Mistake box: Why Day 23's second branch never runs

```
if (arr[i] < smallest) { ... } else if (arr[i] < smallest && arr[i] != smallest) { ... }
```

If **arr[i] < smallest** were true, the first if already handled it. In the else the condition **arr[i] < smallest** is false by definition, so the second test can never be true.

Rule: an **else if** must describe a case the **if** did **not** already catch.

7. Edge cases to remember

Case	Result
all equal	-1 (no second distinct)
one element	-1
negative numbers	works (uses MIN_VALUE , not 0)

Case	Result
-1 is real data	the "no answer" value is ambiguous - a later topic (exceptions / objects) fixes this

8. Complexity

Best = Average = Worst = $O(n)$ (must see every element), space $O(1)$ (three variables).

9. Your 5-minute repair drill (do it once, then stop)

1. Close the book. Write the podium rule in your own words.
2. Write the pseudocode (5 lines).
3. Write the Java from the pseudocode.
4. Dry run {30, 10, 20} and {5, 5, 5}.
5. Compare with section 4. Fix, don't restart.

Tip: Spaced schedule (suggestion)

Today -> tomorrow (2 minutes, blank page) -> in 3 days -> in a week. Two minutes each. If you are tired, skip the day - do not drill.

10. Reconstruction ladder

Level	Problem
1 easy	second largest value of {4, 9, 7}
2 familiar	second distinct largest of {10, 120, 10, 10}
3 combination	return the index of the second distinct largest (use first-index after finding the value)
4 unseen	third distinct largest (what new variable do you need?)

Appendix A – Your code archive (as saved)

Every **.java** file from your ZIP, in the order you learned: foundation files by save date, then **day6** to **day29**. Blank lines are removed to save space and repeated **// // //** comment markers are collapsed to one **//**. Lines starting with **//** are your **commented-out older attempts** (history). Nothing else is changed. Left out: **.class** files, **01.py** (Python turtle drawing) and the empty files **for.java**, **26.java**, **Text File.txt**.

hello.java – saved 14 Jun 2026

```
import java.util.*;
public class hello
{
    public static void main (String args[])
    {
        Scanner sc = new Scanner(System.in);
        int a = sc.nextInt();
        int b = sc.nextInt();
        int sum = a + b;
        System.out.println("The sum is : " + sum);
    }
}
```

switch1.java – saved 14 Jun 2026

```
// import java.util.*;
// public class switch1{
//     public static void main (String args[]) {
//         Scanner sc = new Scanner(System.in);
//         int button = sc.nextInt();
//         switch(button) {
//             case 1 : System.out.println("Hello");
//             break;
//             case 2 : System.out.println("Namaste");
//             break;
//             case 3 : System.out.println("Bonjour");
//             break;
//             default : System.out.println("Invalid button");
//         }
//     }
// }
import java.util.*;
public class switch1{
    public static void main(String args[])
    {
        Scanner sc = new Scanner(System.in);
        System.out.println("ENTER first number :");
        int a = sc.nextInt();
        System.out.println("Enter second Number :");
        int b = sc.nextInt();
        System.out.println("ENTER operators (+, -, *, /)");
        char res = sc.next().charAt(0);
        switch(res){
            case '+' : System.out.println("SUM : " + (a+b));
            break;
            case '-' : System.out.println("SUB : "+ (a-b));
            break;
            case '/' : System.out.println("DIV : " + (a/b));
            break;
            case '*' : System.out.println("MUL : " + (a*b));
            default : System.out.println("Invalid operation");
        }
    }
}
```

pattern.java – saved 17 Jun 2026 – ENTIRE FILE IS COMMENTED OUT (kept as archive)

```
// public class pattern{
//     public static void main (String args[])
//     {
//         int m = 4;
//         int n = 6;
//         for(int i = 1; i <= m; i++){
//             for (int j = 1; j<=n; j++){
```

```

//          System.out.print("**");
//      }
//      System.out.println();
//  }
// }
// op-----> *****
//          *****
//          *****
//          *****
// public class pattern{
//     public static void main (String args[])
//     {
//         int m = 4;
//         int n = 5;
//         for(int i = 1; i <=m; i++){
//             for(int j = 1; j<=n; j++){
//                 if(i ==1 || j ==1 || i==m || j == n){
//                     System.out.print("");
//                 }else{
//                     System.out.print(" ");
//                 }
//             }
//             System.out.println();
//         }
//     }
// }
// op ----> *****
//          *  *
//          *  *
//          *****
// public class pattern {
//     public static void main(String args[]){
//         int m = 4;
//         for(int i = 1; i<=m; i++){
//             for(int j = 1; j<=i; j++){
//                 System.out.print("");
//             }
//             System.out.println();
//         }
//     }
// }
// op---> *
//          **
//          ***
//          ****
// public class pattern{
//     public static void main(String args[]){
//         int n = 4;
//         for (int i = n; i>=1; i--){
//             for (int j = 1; j<=i; j++){
//                 System.out.print("");
//             }
//             System.out.println();
//         }
//     }
// }
// op ----> ****
//          ***
//          **
//          *
// public class pattern{
//     public static void main(String args[]){
//         int n=4;
//         for (int i=1; i <=n; i++){
//             for (int j = 1; j<= n-i; j++){
//                 System.out.print(" ");
//             }
//             for (int j = 1; j<=i; j++){
//                 System.out.print("");
//             }
//             System.out.println();
//         }
//     }
// }
// op -----> *
//          **
//          ***
//          ****
// public class pattern {
//     public static void main(String args[]){
//         int n = 5;

```

```
//      for (int i = 1; i <= n; i++){
//          for (int j = 1; j<=i; j++){
//              System.out.print(j+" ");
//          }
//          System.out.println();
//      }
//  }
// }
// op -----> 1
//          1 2
//          1 2 3
//          1 2 3 4
//          1 2 3 4 5
// public class pattern{
//     public static void main(String aargs[]){
//         int n = 5;
//         for (int i = 1; i <=n; i++){
//             for(int j =1; j<=n-i+1; j++){
//                 System.out.print(j+" ");
//             }
//             System.out.println();
//         }
//     }
// }
// op -----> 1 2 3 4 5
// 1 2 3 4
// 1 2 3
// 1 2
// 1
// -----Floyd's Triangle below-----
// public class pattern{
//     public static void main(String args[]){
//         int n = 5;
//         int num = 1;
//         for (int i = 1; i<=n; i++ ){
//             for( int j = 1; j<=i; j++){
//                 System.out.print(num+" ");
//                 num++;
//             }
//             System.out.println();
//         }
//     }
// }
// op-----> 1
//          2 3
//          4 5 6
//          7 8 9 10
//          11 12 13 14 15
// public class pattern{
//     public static void main(String args[]){
//         int n = 5;
//         for(int i = 1; i<=n; i++){
//             for (int j = 1; j<=i; j++){
//                 int sum = i+j;
//                 if( sum % 2 == 0 ){
//                     System.out.print("1 ");
//                 }else{
//                     System.out.print("0 ");
//                 }
//             }
//             System.out.println();
//         }
//     }
// }
// op-----> 1
//          0 1
//          1 0 1
//          0 1 0 1
//          1 0 1 0 1
```

exjava - saved 20 Jun 2026

```
import java.util.Scanner;
public class ex {
    public static int greater(int a, int b) {
        if (a > b) {
            return a;
        } else {
            return b;
        }
    }
}
```

```

    public static void main(String args[]) {
        Scanner sc = new Scanner(System.in);
        System.out.println("Enter first number :");
        int a = sc.nextInt();
        System.out.println("Enter your second number :");
        int b = sc.nextInt();
        if (a == b) {
            System.out.println(" both are equal ");
        } else {
            int ans = greater(a, b);
            System.out.println("GREATER NUMBER IS : " + ans);
        }
    }
}

```

funmeth.java - saved 20 Jun 2026

```

// -----function multiplication-----
// import java.util.*;
// public class funmeth {
//     public static int cal(int a, int b) {
//         int mul = a * b;
//         return mul;
//     }
//     public static void main(String args[]) {
//         Scanner sc = new Scanner(System.in);
//         int a = sc.nextInt();
//         int b = sc.nextInt();
//         int mul = cal(a, b);
//         System.out.println("MUL IS : " + mul);
//     }
// }
// -----FACTORIAL OF NUMBER -----
import java.util.*;
public class funmeth {
    public static void pFact(int n) {
        if (n < 0) {
            System.out.println("Invalid input");
            return;
        }
        int fact = 1;
        for (int i = n; i >= 1; i--) {
            fact = fact * i;
        }
        System.out.println(fact);
    }
    public static void main(String args[]) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        pFact(n);
    }
}

```

conditional.java - saved 20 Jun 2026

```

// import java.util.*;
// public class conditional
// {
//     public static void main (String args[])
//     {
//         Scanner sc = new Scanner(System.in);
//         int age = sc.nextInt();
//         if(age > 18){
//             System.out.println("Adult");
//         }else
//             System.out.println("Not Adult");
//     }
// }
// import java.util.*;
// class conditional {
//     public static void main (String args[])
//     {
//         Scanner sc = new Scanner(System.in);
//         int number = sc.nextInt();
//         if(number % 2 == 0)
//         {
//             System.out.println("The number is even ");
//         }else
//             System.out.println("The number is odd");
//     }
// }

```

```
// }
// import java.util.Scanner;
// class conditional {
//     public static void main(String args[]) {
//         Scanner sc = new Scanner(System.in);
//         int a = sc.nextInt();
//         int b = sc.nextInt();
//         if (a == b) {
//             System.out.println("both equal");
//         } else {
//             if (a > b) {
//                 System.out.println("a is greter ");
//             } else System.out.println("a is smaller");
//         }
//     }
// }
import java.util.*;
class conditional {
    public static void main(String args[]) {
        Scanner sc = new Scanner(System.in);
        int button = sc.nextInt();
        if (button == 1) {
            System.out.println("Hello");
        } else if (button == 2) {
            System.out.println("Namste");
        } else if (button == 3) {
            System.out.println("Bonjour");
        } else {
            System.out.println("Invalid button");
        }
    }
}
```

exercise.java - saved 21 Jun 2026

```
// import java.util.Scanner;
// public class exercise {
//     public static int avg(int a, int b, int c) {
//         int avg = (a + b + c) / 3;
//         return avg;
//     }
//     public static void main(String args[]) {
//         Scanner sc = new Scanner(System.in);
//         System.out.println("Enetr your 1st number: ");
//         int a = sc.nextInt();
//         System.out.println("Enter 2nd number ");
//         int b = sc.nextInt();
//         System.out.println("Enter 3rd number :");
//         int c = sc.nextInt();
//         int cal = avg(a, b, c);
//         System.out.println("AVERAGE IS : " + cal);
//     }
// }
// -----Q2-----
// import java.util.Scanner;
// public class exercise {
//     public static int odd(int n) {
//         int sum = 0;
//         for (int i = 1; i <= n; i++) {
//             for (int j = i; j <= i; j++) {
//                 System.out.println("NUMBERS : " + j);
//             }
//             if (i % 2 != 0) {
//                 sum = sum + i;
//             }
//         }
//         System.out.println("ODD NUMBER SUM:" + sum);
//         return sum;
//     }
//     public static void main(String args[]) {
//         Scanner sc = new Scanner(System.in);
//         System.out.println("Enetr your number range :");
//         int n = sc.nextInt();
//         odd(n);
//     }
// }
// -----Q3-----
// import java.util.Scanner;
// public class exercise {
//     public static double cir(double r) {
//         double circum = 2 * Math.PI * r;
```

```

//      return circum;
//    }
//    public static void main(String args[]) {
//      Scanner sc = new Scanner(System.in);
//      System.out.println("Enter radius :");
//      double r = sc.nextDouble();
//      double ans = cir(r);
//      System.out.println("Your circumference is :" + ans);
//    }
//  }
// -----Q4-----
// import java.util.Scanner;
// public class exercise {
//   public static void vote(int age) {
//     if (age > 18) {
//       System.out.println("Eligible to vote");
//     } else {
//       System.out.println("You are a kid mother fucker go and sleep ");
//     }
//   }
//   public static void main(String args[]) {
//     Scanner sc = new Scanner(System.in);
//     System.out.println("Enter you age bitch :");
//     int age = sc.nextInt();
//     vote(age);
//   }
// }
// -----Q5----- ALREADY DONE
// -----Q6-----
// public class exercise {
//   public static void main(String args[]) {
//     int num = 1;
//     do {
//       System.out.println(num);
//     } while (true);
//   }
// }
// -----Q7-----
// import java.util.Scanner;
// public class exercise {
//   public static void main(String args[]) {
//     int pos = 0;
//     int neg = 0;
//     int zero = 0;
//     Scanner sc = new Scanner(System.in);
//     System.out.println("How many number will u enter : ");
//     int n = sc.nextInt();
//     for (int i = 0; i < n; i++) {
//       System.out.println("Enter a number :");
//       int num = sc.nextInt();
//       if (num > 0) {
//         pos++;
//       } else if (num < 0) {
//         neg++;
//       } else {
//         zero++;
//       }
//     }
//     System.out.println("POS : " + pos);
//     System.out.println("NEG : " + neg);
//     System.out.println("ZERO : " + zero);
//   }
// }
// -----Q8-----
// import java.util.Scanner;
// public class exercise {
//   public static int power(int x, int n) {
//     int power = 1;
//     for (int i = 1; i <= n; i++) {
//       power = power * x;
//     }
//     return power;
//   }
//   public static void main(String args[]) {
//     Scanner sc = new Scanner(System.in);
//     int x = sc.nextInt();
//     int n = sc.nextInt();
//     int result = power(x, n);
//     System.out.println("power : " + result);
//   }
// }
// ===== Q9 =====

```

```
// import java.util.Scanner;
// public class exercise {
//     public static int gcd(int a, int b) {
//         int gcd = 1;
//         int min;
//         if (a < b) {
//             min = a;
//         } else {
//             min = b;
//         }
//         for (int i = 1; i <= min; i++) {
//             if (a % i == 0 && b % i == 0) {
//                 gcd = i;
//             }
//         }
//         return gcd;
//     }
//     public static void main(String args[]) {
//         Scanner sc = new Scanner(System.in);
//         System.out.println("Enter your first number :");
//         int a = sc.nextInt();
//         System.out.println("Enter your second number :");
//         int b = sc.nextInt();
//         System.out.println(" GCD : " + gcd(a, b));
//     }
// }

// =====Q 10 =====
import java.util.Scanner;
public class exercise {
    public static void fab(int n) {
        int first = 0;
        int second = 1;
        System.out.print(first + " " + second + " ");
        for (int i = 3; i <= n; i++) {
            int next = first + second;
            System.out.print(next + " ");
            first = second;
            second = next;
        }
    }
    public static void main(String args[]) {
        Scanner sc = new Scanner(System.in);
        System.out.println("Enter number of terms : ");
        int n = sc.nextInt();
        fab(n);
    }
}
```

Array.java - saved 22 Jun 2026

```
// public class Array {
//     public static void main(String args[]) {
//         // int marks[] = new int[3]; OR
//         int marks[] = {88, 78, 90};
//         // marks[0] = 90;
//         // marks[1] = 88;
//         // marks[2] = 95;
//         for (int i = 0; i < 3; i++) {
//             System.out.println(marks[i]);
//         }
//     }
// }

// import java.util.Scanner;
// public class Array {
//     public static void main(String args[]) {
//         Scanner sc = new Scanner(System.in);
//         System.out.println("Enter number of array elements :");
//         int size = sc.nextInt();
//         int numbers[] = new int[size];
//         // input ke liye
//         for (int i = 0; i < size; i++) {
//             System.out.println("Enter array element :");
//             numbers[i] = sc.nextInt();
//         }
//         // output ke liye
//         for (int i = 0; i < size; i++) {
//             System.out.println("ELEMENT : " + numbers[i]);
//         }
//     }
// }

import java.util.Scanner;
```

```

public class Array {
    public static void main(String args[]) {
        Scanner sc = new Scanner(System.in);
        System.out.println("Enter number of array elements :");
        int size = sc.nextInt();
        int numbers[] = new int[size];
        for (int i = 0; i < size; i++) {
            System.out.println("Enter array element :");
            numbers[i] = sc.nextInt();
        }
        System.out.println("Enter which element you want to find ");
        int x = sc.nextInt();
        for (int i = 0; i < numbers.length; i++) {
            if (numbers[i] == x) {
                System.out.println("ELEMENT FOUND AT INDEX : " + i);
            } else {
                System.out.println("Element not found ");
            }
        }
    }
}

```

adpattern.java - saved 25 Aug 2026

```

// public class adpattern {
//     public static void main(String args[]) {
//         int n = 5;
//         for (int i = 1; i <= n; i++) {
//             for (int j = 1; j <= i; j++) {
//                 System.out.print("*");
//             }
//             int spaces = 2 * (n - i);
//             for (int j = 1; j <= spaces; j++) {
//                 System.out.print(" ");
//             }
//             for (int j = 1; j <= i; j++) {
//                 System.out.print("*");
//             }
//             System.out.println();
//         }
//         for (int i = n; i >= 1; i--) {
//             for (int j = 1; j <= i; j++) {
//                 System.out.print("*");
//             }
//             int spaces = 2 * (n - i);
//             for (int j = 1; j <= spaces; j++) {
//                 System.out.print(" ");
//             }
//             for (int j = 1; j <= i; j++) {
//                 System.out.print("*");
//             }
//             System.out.println();
//         }
//     }
// }
// -----0/P----->>>
// * *
// ** **
// *** ***
// **** ****
// *****
// *****
// **** ****
// *** ***
// ** **
// * *
// public class adpattern {
//     public static void main(String args[]) {
//         int n = 5;
//         for (int i = 1; i <= n; i++) {
//             int spaces = n - i;
//             for (int j = 1; j <= spaces; j++) {
//                 System.out.print(" ");
//             }
//             for (int j = 1; j <= 5; j++) {
//                 System.out.print("*");
//             }
//             System.out.println();
//         }
//     }
// }

```

```

// O/P----->
//      *****
//      *****
//      *****
//      *****
//      *****
// public class adpattern {
//     public static void main(String args[]) {
//         int n = 5;
//         int m = 5;
//         for (int i = 1; i <= n; i++) {
//             int spaces = n - i;
//             for (int j = 1; j <= spaces; j++) {
//                 System.out.print(" ");
//             }
//             for (int j = 1; j <= 5; j++) {
//                 if (i == 1 || j == 1 || i == m || j == n) {
//                     System.out.print("*");
//                 } else {
//                     System.out.print(" ");
//                 }
//             }
//             System.out.println();
//         }
//     }
// }
// O/P----->
//      *****
//      *   *
//      *   *
//      *   *
//      *****
// public class adpattern {
//     public static void main(String args[]) {
//         int n = 5;
//         for (int i = 1; i <= n; i++) {
//             int spaces = n - i;
//             for (int j = 1; j <= spaces; j++) {
//                 System.out.print(" ");
//             }
//             for (int j = 1; j <= i; j++) {
//                 System.out.print(i + " ");
//             }
//             System.out.println();
//         }
//     }
// }
// O/P ----->
//      1
//      2 2
//      3 3 3
//      4 4 4 4
//      5 5 5 5 5
// -----PALINDROME PATTERN-----
// public class adpattern {
//     public static void main(String args[]) {
//         int n = 5;
//         for (int i = 1; i <= n; i++) {
//             int spaces = n - i;
//             for (int j = 1; j <= spaces; j++) {
//                 System.out.print(" ");
//             }
//             for (int j = i; j >= 1; j--) {
//                 System.out.print(j);
//             }
//             for (int j = 2; j <= i; j++) {
//                 System.out.print(j);
//             }
//             System.out.println();
//         }
//     }
// }
// OP ----->
//      1
//      212
//      32123
//      4321234
//      543212345
// public class adpattern {
//     public static void main(String args[]) {
//         int n = 4;
//         for (int i = 1; i <= n; i++) {

```

```

        int spaces = n - i;
        for (int j = 1; j <= spaces; j++) {
            System.out.print(" ");
        }
        int stars = 2 * i - 1;
        for (int j = 1; j <= stars; j++) {
            System.out.print("*");
        }
        System.out.println();
    }
    for (int i = n; i >= 1; i--) {
        int spaces = n - i;
        for (int j = 1; j <= spaces; j++) {
            System.out.print(" ");
        }
        int stars = 2 * i - 1;
        for (int j = 1; j <= stars; j++) {
            System.out.print("*");
        }
        System.out.println();
    }
}

// -----OP----->>
//      *
//     ***
//    *****
//   *****
//  *****
// *****
// *****
//      *

```

java0.java - saved 28 Aug 2026

```

// public class java0 {
//     public static void main(String args[]) {
//         int a = 10;
//         int b = 20;
//         System.out.println("SUM " + (a + b));
//     }
// }
import java.util.Scanner;
public class java0 {
    public static void main(String args[]) {
        Scanner sc = new Scanner(System.in);
        System.out.println("Enter a number :");
        int num1 = sc.nextInt();
        if (num1 % 2 == 0) {
            System.out.println("Numebr is even");
        } else {
            System.out.println("numer is odd");
        }
        sc.close();
    }
}

```

switch2.java - saved 28 Aug 2026

```

import java.util.Scanner;
public class switch2 {
    public static void main(String args[]) {
        Scanner sc = new Scanner(System.in);
        System.out.println("Enter a number :");
        int num = sc.nextInt();
        switch (num) {
            case 1:
                System.out.println("Monday");
                break;
            case 2:
                System.out.println("Tuesday");
                break;
            case 3:
                System.out.println("Wednesday");
                break;
            case 4:
                System.out.println("Thursday");
                break;
            case 5:
                System.out.println("Friday");

```

```

        break;
    case 6:
        System.out.println("Saturday");
        break;
    case 7:
        System.out.println("Sunday");
        break;
    default:
        System.out.println("Invalid day ");
    }
    sc.close();
}
}

```

fact.java - saved 29 Aug 2026

```

import java.util.Scanner;
public class fact {
    public static void main(String args[]) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter a number : ");
        int num = sc.nextInt();
        int fact = 1;
        for (int i = 1; i <= num; i++) {
            fact = fact * i;
        }
        System.out.println("fact : " + fact);
        sc.close();
    }
}

```

count.java - saved 29 Aug 2026

```

import java.util.Scanner;
public class count {
    public static void main(String args[]) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter numbers : ");
        int num = sc.nextInt();
        int count = 0;
        if (num == 0) {
            count++;
        } else {
            while (num > 0) {
                num = num / 10;
                count++;
            }
        }
        System.out.println("number of digits : " + count);
        sc.close();
    }
}

```

plus.java - saved 30 Aug 2026

```

// import java.util.Scanner;
// public class plus {
//     public static void main(String args[]) {
//         Scanner sc = new Scanner(System.in);
//         System.out.print("Enter a number : ");
//         int num = sc.nextInt();
//         int sum = 0;
//         for (int i = 1; i <= num; i++) {
//             sum = sum + i;
//         }
//         System.out.println("Sum : " + sum);
//     }
// }
import java.util.*;
public class plus {
    public static void main(String args[]) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter a number : ");
        int num = sc.nextInt();
        int fact = 1;
        for (int i = 1; i <= num; i++) {
            fact = fact * i;
        }
        System.out.println("factorial = " + fact);
    }
}

```

```

    }
}

```

loop.java - saved 30 Aug 2026

```

// _____ FFF000RRR LLLL00000PPPP _____
// public class loop {
//     public static void main(String args[]) {
//         for (int i = 0; i <= 100; i++) {
//             System.out.println("HELLO WORLD");
//         }
//     }
// }

// public class loop{
//     public static void main(String args[]){
//         for(int i = 0; i < 21; i++){
//             System.out.print(i+" ");
//         }
//     }
// }

// ?? _____ WWHHIILLEE LLLL00000PPPP _____
// public class loop{
//     public static void main(String args[]){
//         int i = 0;
//         while(i < 21) {
//             System.out.println(i);
//             i++;
//         }
//     }
// }

// _____ DDD0000 WHILE _____
// public class loop{
//     public static void main(String args[]){
//         int i = 0;
//         do{
//             System.out.println(i);
//             i++;
//         }while( i < 21);
//     }
// }

// public class loop{
//     public static void main(String args[]){
//         int i = 12;
//         while(i < 11){
//             System.out.println("HELLO");
//         }
//         do {
//             System.out.println("HELLO SHUBHAM");
//         }while(i < 11);
//     }
// }

// _____ EXAMPLE _____
// import java.util.*;
// public class loop{
//     public static void main(String args[]){
//         Scanner sc = new Scanner(System.in);
//         int n = sc.nextInt();
//         int sum = 0;
//         for (int i = 0; i <= n; i++){
//             sum = sum+i;
//         }
//         System.out.println("SUM :" + sum);
//     }
// }

// _____ TABLE USING LOOP _____
import java.util.*;
public class loop {
    public static void main(String args[]) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        for (int i = 1; i < 11; i++) {
            System.out.println(n * i);
        }
    }
}

```

count1.java - saved 30 Aug 2026

```

import java.util.Scanner;
public class count1 {

```

```

        public static void main(String args[]) {
            Scanner sc = new Scanner(System.in);
            System.out.print("Enter the digis : ");
            int num = sc.nextInt();
            int count = 0;
            while (num > 0) {
                num = num / 10;
                count++;
            }
            System.out.println("number of digits : " + count);
        }
    }
}

```

reverse.java - saved 31 Aug 2026

```

public class reverse {
    public static void main(String args[]) {
        for (int i = 1; i <= 20; i = i + 2) {
            System.out.println(i);
        }
    }
}

```

table.java - saved 31 Aug 2026

```

import java.util.Scanner;
public class table {
    public static void main(String args[]) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter a number : ");
        int num = sc.nextInt();
        for (int i = 1; i <= 10; i++) {
            int mul = num * i;
            System.out.println(num + " x " + i + " = " + mul);
        }
    }
}

```

project1.java - saved 31 Aug 2026

```

import java.util.Scanner;
public class project1 {
    public static void main(String args[]) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter Student name :");
        String name = sc.nextLine();
        System.out.print("Enter java marks :");
        int mark1 = sc.nextInt();
        System.out.print("Enter DBMS marks :");
        int mark2 = sc.nextInt();
        System.out.print("Enter C++ marks :");
        int mark3 = sc.nextInt();
        System.out.println(" ");
        System.out.println("-----RESULT-----");
        System.out.println("Student : " + name);
        int total = mark1 + mark2 + mark3;
        System.out.println("Total : " + total);
        double percentage = (mark1 + mark2 + mark3) * (100) / (300);
        System.out.println("Percentage : " + percentage);
        if (percentage >= 85) {
            System.out.println("Status : pass");
            System.out.println("Grade : A");
            System.out.println(" ");
            System.out.println("Performance : Excellent");
        } else if (percentage >= 75) {
            System.out.println("Status : Pass");
            System.out.println("Grade : B");
            System.out.println(" ");
            System.out.println("Performance : Good");
        } else if (percentage >= 65) {
            System.out.println("Status : pass");
            System.out.println("Grade : C");
            System.out.println(" ");
            System.out.println("Performance : Average");
        } else {
            System.out.println("Fail!");
            System.out.println(" ");
            System.out.println("Performance : Noob Study kar!");
        }
    }
}

```

```

        sc.close();
    }
}

```

loop1.java - saved 02 Sep 2026

```

// public class loop1 {
//     public static void main(String args[]) {
//         for (int i = 0; i <= 10; i++) {
//             System.out.println(i);
//         }
//     }
// }
// using if but try to do it without if -->
// public class loop1 {
//     public static void main(String args[]) {
//         for (int i = 2; i <= 20; i++) {
//             if (i % 2 == 0) {
//                 System.out.println(i);
//             }
//         }
//     }
// }
// ?? below is a program for even and can be odd too without using if else
// public class loop1 {
//     public static void main(String args[]) {
//         for (int i = 2; i <= 20; i += 2) {
//             System.out.println(i);
//         }
//     }
// }
// WHILE LOOP
// public class loop1 {
//     public static void main(String args[]) {
//         int i = 10;
//         while (i >= 1) {
//             System.out.println(i);
//             i--;
//         }
//     }
// }
// DO WHILE LOOP
// public class loop1 {
//     public static void main(String args[]) {
//         int i = 1;
//         do {
//             System.out.println(i);
//             i++;
//         } while (i <= 5);
//     }
// }
// BREAK STATEMENT
// public class loop1 {
//     public static void main(String args[]) {
//         for (int i = 1; i <= 10; i++) {
//             if (i == 7) {
//                 break;
//             }
//             System.out.println(i);
//         }
//     }
// }
// CONTINUE STATEMENT
// public class loop1{
//     public static void main(String args[])
//     {
//         for(int i = 1; i <= 10; i++)
//         {
//             if (i == 5){
//                 continue;
//             }
//             System.out.println(i);
//         }
//     }
// }
// public class loop1 {
//     public static void main(String args[]) {
//         for (int i = 1; i <= 5; i++) {
//             for (int j = 1; j <= i; j++) {
//                 System.out.print("*");
//             }
//         }
//     }
// }

```

```

//      System.out.println();
//    }
//  }
// }
// OP -----> *
//      **
//      ***
//      ****
//      *****
// public class loop1 {
//     public static void main(String args[]) {
//         for (int i = 1; i <= 5; i++) {
//             for (int j = 5; j >= i; j--) {
//                 System.out.print("*");
//             }
//             System.out.println();
//         }
//     }
// }
// OP----->
// *****
// ****
// ***
// **
// *
// public class loop1 {
//     public static void main(String args[]) {
//         int inc = 1;
//         for (int i = 1; i <= 5; i++) {
//             for (int j = 1; j <= i; j++) {
//                 System.out.print(inc++);
//             }
//             System.out.println();
//         }
//     }
// }
// OP ----->
// 1
// 23
// 456
// 78910
// 1112131415
// import java.util.Scanner;
// public class loop1 {
//     public static void main(String args[]) {
//         Scanner sc = new Scanner(System.in);
//         System.out.print("Enter number of stars : ");
//         int num = sc.nextInt();
//         for (int i = 1; i <= num; i++) {
//             for (int j = 1; j <= i; j++) {
//                 System.out.print(" * ");
//             }
//             System.out.println();
//         }
//     }
// }
// *
// * *
// * * *
// * * * *
// * * * * *
// * * * * * *
// import java.util.Scanner;
// public class loop1 {
//     public static void main(String args[]) {
//         Scanner sc = new Scanner(System.in);
//         System.out.print("Enter number of stars : ");
//         int num = sc.nextInt();
//         for (int i = 1; i <= num; i++) {
//             for (int j = num; j >= i; j--) {
//                 System.out.print(" * ");
//             }
//             System.out.println();
//         }
//     }
// }
// * * * * *
// * * * *
// * * *
// * *
// *
// import java.util.Scanner;

```

```

// public class loop1 {
//     public static void main(String args[]) {
//         Scanner sc = new Scanner(System.in);
//         System.out.print("Enter number of stars : ");
//         int num = sc.nextInt();
//         for (int k = 1; k <= num; k++) {
//             for (int i = 1; i <= num - k; i++) {
//                 System.out.print("*");
//             }
//             for (int j = 1; j <= k; j++) {
//                 System.out.print(" ");
//             }
//             System.out.println();
//         }
//     }
// }

// public class loop1 {
//     public static void main(String args[]) {
//         for (int i = 1; i <= 5; i++) {
//             for (int j = 1; j <= i; j++) {
//                 System.out.print(j);
//             }
//             System.out.println();
//         }
//     }
// }

// 1
// 12
// 123
// 1234
// 12345
// public class loop1 {
//     public static void main(String args[]) {
//         int num = 1;
//         for (int i = 1; i <= 4; i++) {
//             for (int j = 1; j <= i; j++) {
//                 System.out.print(num);
//                 num++;
//             }
//             System.out.println();
//         }
//     }
// }

// 1
// 23
// 456
// 78910
// import java.util.Scanner;
// public class loop1 {
//     public static void main(String args[]) {
//         Scanner sc = new Scanner(System.in);
//         System.out.print("Enter digits : ");
//         int num = sc.nextInt();
//         int count = 0;
//         while (num > 0) {
//             num = num / 10;
//             count++;
//         }
//         System.out.println("number of digits : " + count);
//     }
// }

// Enter digits : 11222345
// number of digits : 8
// import java.util.Scanner;
// public class loop1 {
//     public static void main(String args[]) {
//         Scanner sc = new Scanner(System.in);
//         System.out.print("Enter the digits : ");
//         int num = sc.nextInt();
//         int sum = 0;
//         int digit;
//         while (num > 0) {
//             digit = num % 10;
//             num = num / 10;
//             sum = sum + digit;
//         }
//         System.out.println("sum : " + sum);
//     }
// }

// Enter the digits : 1234
// sum : 10
// import java.util.Scanner;

```

```
// public class loop1 {
//     public static void main(String args[]) {
//         Scanner sc = new Scanner(System.in);
//         System.out.print("Enter this digits : ");
//         int num = sc.nextInt();
//         int reverse = 0;
//         int digit = 0;
//         while (num > 0) {
//             digit = num % 10;
//             reverse = reverse * 10 + digit;
//             num = num / 10;
//         }
//         System.out.print(reverse);
//     }
// }
// Enter this digits : 123
// 321
// import java.util.Scanner;
// public class loop1 {
//     public static void main(String args[]) {
//         Scanner sc = new Scanner(System.in);
//         System.out.print("Enter the digits : ");
//         int num = sc.nextInt();
//         int count = 0;
//         int digit = 0;
//         while (num > 0) {
//             digit = num % 10;
//             if (digit % 2 == 0) {
//                 count++;
//             }
//             num = num / 10;
//         }
//         System.out.println("even : " + count);
//     }
// }
// Enter the digits : 123456789
// even : 4
import java.util.Scanner;
public class loop1 {
    public static void main(String args[]) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter the digits : ");
        int num = sc.nextInt();
        int orig = num;
        int reverse = 0;
        int digit = 0;
        while (num > 0) {
            digit = num % 10;
            reverse = reverse * 10 + digit;
            num = num / 10;
        }
        if (orig == reverse) {
            System.out.println("digit is palindrome : " + reverse);
        } else {
            System.out.println("digit is not palindrome : " + reverse);
        }
    }
}
}
```

pluseven.java - saved 05 Sep 2026

```
import java.util.Scanner;
public class pluseven {
    public static void main(String args[]) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enetr a number : ");
        int num = sc.nextInt();
        int sum = 0;
        for (int i = 0; i <= num; i++) {
            if (num % 2 == 0) {
                sum = sum + i;
            }
        }
        System.out.println("sum : " + sum);
        sc.close();
    }
}
```

day6.java - saved 02 Sep 2026

```
// ----->>old concepts
// import java.util.*;
// public class day6 {
//     public static void main(String args[]) {
//         Scanner sc = new Scanner(System.in);
//         System.out.print("Enter digits : ");
//         int num = sc.nextInt();
//         int count = 0;
//         while (num > 0) {
//             num = num / 10;
//             count++;
//         }
//         System.out.println("number of digits : " + count);
//     }
// }
// Enter digits : 124364
// number of digits : 6
// import java.util.Scanner;
// public class day6 {
//     public static void main(String args[]) {
//         Scanner sc = new Scanner(System.in);
//         System.out.print("Enter digits : ");
//         int num = sc.nextInt();
//         int digit = 0;
//         int sum = 0;
//         while (num > 0) {
//             digit = num % 10;
//             sum = sum + digit;
//             num = num / 10;
//         }
//         System.out.println("Sum : " + sum);
//     }
// }
// import java.util.Scanner;
// public class day6 {
//     public static void main(String args[]) {
//         Scanner sc = new Scanner(System.in);
//         System.out.print("Enter digits : ");
//         int num = sc.nextInt();
//         int count = 0;
//         int digit = 0;
//         while (num > 0) {
//             digit = num % 10;
//             num = num / 10;
//             if (digit % 2 == 0) {
//                 count++;
//             }
//         }
//         System.out.println("EVEN : " + count);
//     }
// }
// java day6
// Enter digits : 123456
// EVEN : 3
// import java.util.Scanner;
// public class day6 {
//     public static void main(String args[]) {
//         Scanner sc = new Scanner(System.in);
//         System.out.print("Enter digits : ");
//         int num = sc.nextInt();
//         int largest = 0;
//         int digit = 0;
//         while (num > 0) {
//             digit = num % 10;
//             if (digit > largest) {
//                 largest = digit;
//             }
//             num = num / 10;
//         }
//         System.out.println("largest : " + largest);
//     }
// }
// Enter digits : 54312398
// largest : 9
// import java.util.Scanner;
// public class day6 {
//     public static void main(String args[]) {
//         Scanner sc = new Scanner(System.in);
//         System.out.print("Enter digits : ");
//         int num = sc.nextInt();
```

```
//      int digit = 0;
//      int sum = 0;
//      int largest = 0;
//      int count = 0;
//      int even = 0;
//      while (num > 0) {
//          digit = num % 10;
//          count++;
//          sum = digit + sum;
//          if (digit > largest) {
//              largest = digit;
//          }
//          if (digit % 2 == 0) {
//              even++;
//          }
//          num = num / 10;
//      }
//      System.out.println("Digits : " + count);
//      System.out.println("sum : " + sum);
//      System.out.println("largest : " + largest);
//      System.out.println("even : " + even);
//  }
// }
// Enter digits : 12864
// Digits : 5
// sum : 21
// largest : 8
// even : 4
// -----METHODS----->>>>>>
// public class day6 {
//     static void greet(String name) {
//         System.out.println("HELLO " + name);
//     }
//     public static void main(String args[]) {
//         greet("Shubham");
//     }
// }
// HELLO Shubham
// public class day6 {
//     static void mul(int a, int b) {
//         System.out.println(a * b);
//     }
//     public static void main(String args[]) {
//         mul(7, 6);
//     }
// }
// OP-->42
public class day6 {
    static int add(int a, int b) {
        return a + b;
    }
    public static void main(String args[]) {
        int sum = add(15, 25);
        System.out.println("sum = " + sum);
    }
}
```

day7.java - saved 04 Sep 2026

```
// import java.util.Scanner;
// public class day7 {
//     public static void main(String args[]) {
//         Scanner sc = new Scanner(System.in);
//         System.out.print("Enter digits : ");
//         int num = sc.nextInt();
//         int count = 0;
//         int digit = 0;
//         int sum = 0;
//         int largest = 0;
//         int even = 0;
//         int orig = 0;
//         while (num > 0) {
//             digit = num % 10;
//             count++;
//             num = num / 10;
//             sum = sum + digit;
//             if (digit > largest) {
//                 largest = digit;
//             }
//             if (digit % 2 == 0) {
//                 even++;
//             }
//         }
//     }
// }
```

```

//      }
//    }
//    System.out.println(" Digits : " + count);
//    System.out.println(" Sum : " + sum);
//    System.out.println(" largest : " + largest);
//    System.out.println(" even : " + even);
//  }
// }
// Enter digits : 128765
// Digits : 6
// Sum : 29
// largest : 8
// even : 3
// public class day7 {
//     static int square(int n) {
//         return n * n;
//     }
//     public static void main(String args[]) {
//         int result = square(7);
//         System.out.println("result" + result);
//     }
// }
// result = 49
// public class day7 {
//     static boolean isEven(int n) {
//         return n % 2 == 0;
//     }
//     public static void main(String args[]) {
//         boolean result = isEven(5);
//         System.out.println("even : " + result);
//     }
// }
// even : false
// public class day7 {
//     static int largest(int a, int b, int c) {
//         int largest = 0;
//         if (a > largest) {
//             largest = a;
//         }
//         if (b > largest) {
//             largest = b;
//         }
//         if (c > largest) {
//             largest = c;
//         }
//         return largest;
//     }
//     public static void main(String args[]) {
//         int result = largest(12, 544, 34);
//         System.out.println("Largest : " + result);
//     }
// }
// Largest : 544
// public class day7 {
//     static int sumDigit(int num) {
//         int digit = 0;
//         int sum = 0;
//         while (num > 0) {
//             digit = num % 10;
//             sum = sum + digit;
//             num = num / 10;
//         }
//         return sum;
//     }
//     public static void main(String args[]) {
//         int result = sumDigit(58392);
//         System.out.println("Sum : " + result);
//     }
// }
// Sum : 27
// public class day7 {
//     static int even(int num) {
//         int digit = 0;
//         int count = 0;
//         while (num > 0) {
//             digit = num % 10;
//             if (digit % 2 == 0) {
//                 count++;
//             }
//             num = num / 10;
//         }
//         return count;
//     }

```

```
//      }
//      public static void main(String args[]) {
//          int result = even(58923);
//          System.out.println("Even : " + result);
//      }
//  }
//  Even : 2
//  public class day7 {
//      static int odd(int num) {
//          int count = 0;
//          int digit = 0;
//          while (num > 0) {
//              digit = num % 10;
//              if (digit % 2 == 1) {
//                  count++;
//              }
//              num = num / 10;
//          }
//          return count;
//      }
//      public static void main(String args[]) {
//          int result = odd(23451);
//          System.out.println("Odd : " + result);
//      }
//  }
//  Odd : 3
//  public class day7 {
//      static int evenSum(int num) {
//          int sum = 0;
//          int count = 0;
//          int digit = 0;
//          while (num > 0) {
//              digit = num % 10;
//              if (digit % 2 == 0) {
//                  sum = sum + digit;
//              }
//              num = num / 10;
//          }
//          return sum;
//      }
//      public static void main(String args[]) {
//          int result = evenSum(58392);
//          System.out.println(" Even sum : " + result);
//      }
//  }
```

day8.java - saved 05 Sep 2026

```
// -----RECALL-----
// class day8{
//     public static void main(String args[]){
//         for(int i = 1; i <= 10; i++){
//             System.out.print(i);
//         }
//     }
// }
// import java.util.Scanner;
// public class day8 {
//     public static void main(String args[]) {
//         Scanner sc = new Scanner(System.in);
//         System.out.print("Enter digits : ");
//         int num = sc.nextInt();
//         int count = 0;
//         int digit = 0;
//         while (num > 0) {
//             digit = num % 10;
//             count++;
//             num = num / 10;
//         }
//         System.out.println("Digits : " + count);
//     }
// }
// public class day8 {
//     static int square(int num) {
//         return num * num;
//     }
//     public static void main(String args[]) {
//         int result = square(5);
//         System.out.println("Square : " + result);
//     }
// }
```

```
// -----ARRAYS-----
// public class day8 {
//     public static void main(String args[]) {
//         int[] arr = {10, 20, 30, 40, 50};
//         System.out.println(arr[0]);
//         System.out.println(arr[2]);
//         System.out.println(arr[4]);
//     }
// }
// 10
// 30
// 50
// public class day8 {
//     public static void main(String args[]) {
//         int[] arr = {10, 20, 30, 40, 50};
//         for (int i = 0; i < arr.length; i++) {
//             System.out.println(arr[i]);
//         }
//     }
// }
// 10
// 20
// 30
// 40
// 50
// public class day8 {
//     public static void main(String args[]) {
//         int[] arr = {10, 20, 30, 40, 50};
//         int sum = 0;
//         for (int i = 0; i < arr.length; i++) {
//             sum = sum + arr[i];
//         }
//         System.out.println("Sum : " + sum);
//     }
// }
// Sum : 150
// public class day8 {
//     public static void main(String args[]) {
//         int largest = 0;
//         int arr[] = {22, 39, 12, 53, 24, 14};
//         for (int i = 0; i < arr.length; i++) {
//             if (arr[i] > largest) {
//                 largest = arr[i];
//             }
//         }
//         System.out.println("Largest : " + largest);
//     }
// }
// Largest : 53
// public class day8 {
//     public static void main(String args[]) {
//         int smallest = 1000;
//         int arr[] = {22, 39, 12, 53, 24, 14};
//         for (int i = 0; i < arr.length; i++) {
//             if (arr[i] < smallest) {
//                 smallest = arr[i];
//             }
//         }
//         System.out.println("Smallest : " + smallest);
//     }
// }
// Smallest : 12
// public class day8 {
//     public static void main(String args[]) {
//         int count = 0;
//         int arr[] = {10, 23, 5, 4, 12, 54};
//         for (int i = 0; i < arr.length; i++) {
//             if (arr[i] % 2 == 0) {
//                 count++;
//             }
//         }
//         System.out.println("Even digits : " + count);
//     }
// }
// Even digits : 4
// public class day8 {
//     public static void main(String args[]) {
//         int arr[] = {10, 20, 30, 40, 50};
//         for (int i = 4; i >= 0; i--) {
//             System.out.println(arr[i]);
//         }
//     }
// }
```

```
// }
// 50
// 40
// 30
// 20
// 10
// public class day8 {
//     public static void main(String args[]) {
//         boolean found = false;
//         int arr[] = {10, 34, 29, 48, 23};
//         for (int i = 0; i < arr.length; i++) {
//             if (arr[i] == 29) {
//                 found = true;
//             }
//         }
//         if (found == true) {
//             System.out.println("Found ");
//         } else {
//             System.out.println("Not found ");
//         }
//     }
// }
// Found
public class day8 {
    public static void main(String args[]) {
        int arr[] = {10, 20, 37, 40, 50};
        float avg = 0;
        float total = 0;
        for (int i = 0; i < arr.length; i++) {
            avg = avg + arr[i];
        }
        total = avg / arr.length;
        System.out.println("Average : " + total);
    }
}
// Average : 31.4
```

day9.java - saved 05 Sep 2026

```
// -----Saturday-----REVISION-----
// import java.util.Scanner;
// public class day9 {
//     public static void main(String args[]) {
//         Scanner sc = new Scanner(System.in);
//         System.out.print("Enter Your name : ");
//         String s = sc.nextLine();
//         System.out.print("Enter your age : ");
//         int age = sc.nextInt();
//         System.out.println(" Name : " + s);
//         System.out.println("Age : " + age);
//     }
// }
// import java.util.Scanner;
// class day9 {
//     public static void main(String args[]) {
//         Scanner sc = new Scanner(System.in);
//         System.out.print("Enter the number : ");
//         int num = sc.nextInt();
//         if (num % 2 == 0) {
//             System.out.println("The number " + num + " is even");
//         } else {
//             System.out.println("The number " + num + " is odd ");
//         }
//     }
// }
// Enter the number : 4
// The number 4 is even
// public class day9 {
//     public static void main(String args[]) {
//         for (int i = 1; i <= 10; i++) {
//             System.out.println(i);
//         }
//     }
// }
// import java.util.Scanner;
// public class day9 {
//     public static void main(String args[]) {
//         Scanner sc = new Scanner(System.in);
//         System.out.print("Enter number a : ");
//         int num1 = sc.nextInt();
//         System.out.print("Enter number b : ");
```

```

//      int num2 = sc.nextInt();
//      System.out.println("Sum : " + (num1 + num2));
//      System.out.println("Difference : " + (num1 - num2));
//      System.out.println("Product : " + (num1 * num2));
//      System.out.println("Division : " + (num1 / num2));
//      System.out.println("Remainder : " + (num1 % num2));
//  }
// }
// Enter number a : 20
// Enter number b : 6
// Sum : 26
// Difference : 14
// Product : 120
// Division : 3
// Remainder : 2
// import java.util.Scanner;
// public class day9 {
//     public static void main(String args[]) {
//         Scanner sc = new Scanner(System.in);
//         System.out.print("Enter the number : ");
//         int num = sc.nextInt();
//         if (num > 0) {
//             System.out.println("Number is positive !");
//         } else if (num < 0) {
//             System.out.println("Number is negative !");
//         } else {
//             System.out.println("Number is 0 !");
//         }
//     }
// }
// import java.util.Scanner;
// public class day9 {
//     public static void main(String args[]) {
//         Scanner sc = new Scanner(System.in);
//         System.out.print("Enter first number : ");
//         int num1 = sc.nextInt();
//         System.out.print("Enter second number : ");
//         int num2 = sc.nextInt();
//         int largest = 0;
//         if (num1 > num2) {
//             largest = num1;
//             System.out.println("Largest : " + largest);
//         } else if (num2 > num1) {
//             largest = num2;
//             System.out.println("Largest : " + largest);
//         } else {
//             System.out.println("Both are equal !");
//         }
//     }
// }
// Enter first number : 25
// Enter second number : 18
// Largest : 25
// import java.util.Scanner;
// public class day9 {
//     public static void main(String args[]) {
//         Scanner sc = new Scanner(System.in);
//         System.out.print("Enter the number : ");
//         int num = sc.nextInt();
//         int sum = 0;
//         for (int i = 1; i <= num; i++) {
//             if (i % 2 == 0) {
//                 sum = sum + i;
//             }
//         }
//         System.out.println("Sum of even till " + num + " is : " + sum);
//         sc.close();
//     }
// }
// Enter the number : 10
// Sum of even till 10 is : 30
// import java.util.Scanner;
// public class day9 {
//     public static void main(String args[]) {
//         Scanner sc = new Scanner(System.in);
//         System.out.print("Enter a number :");
//         int num = sc.nextInt();
//         for (int i = num; i >= 1; i--) {
//             System.out.println(i);
//         }
//     }
// }
// }

```

```

// import java.util.Scanner;
// public class day9 {
//     public static void main(String args[]) {
//         Scanner sc = new Scanner(System.in);
//         System.out.print("Enter a number :");
//         int num = sc.nextInt();
//         for (int i = 0; i <= num; i++) {
//             if (i % 2 == 0) {
//                 System.out.println(i);
//             }
//         }
//     }
// }
// Enter a number :10
// 0
// 2
// 4
// 6
// 8
// 10
// import java.util.Scanner;
// public class day9 {
//     public static void main(String args[]) {
//         Scanner sc = new Scanner(System.in);
//         System.out.print("Enter a number : ");
//         int num = sc.nextInt();
//         int mul = 0;
//         for (int i = 1; i <= 10; i++) {
//             mul = num * i;
//             System.out.println(num + " x " + i + " = " + mul);
//         }
//         sc.close();
//     }
// }
// Enter a number : 5
// 5 x 1 = 5
// 5 x 2 = 10
// 5 x 3 = 15
// 5 x 4 = 20
// 5 x 5 = 25
// 5 x 6 = 30
// 5 x 7 = 35
// 5 x 8 = 40
// 5 x 9 = 45
// 5 x 10 = 50
// import java.util.Scanner;
// public class day9 {
//     public static void main(String args[]) {
//         Scanner sc = new Scanner(System.in);
//         System.out.print("Enter the number : ");
//         int num = sc.nextInt();
//         int sum = 0;
//         for (int i = 1; i <= num; i++) {
//             sum = sum + i;
//         }
//         System.out.println("Sum : " + sum);
//         sc.close();
//     }
// }
// Enter the number : 5
// Sum : 15
// import java.util.Scanner;
// public class day9 {
//     public static void main(String args[]) {
//         Scanner sc = new Scanner(System.in);
//         System.out.print("Enter thhe number : ");
//         int num = sc.nextInt();
//         int fact = 1;
//         for (int i = 1; i <= num; i++) {
//             fact = fact * i;
//         }
//         System.out.println("Factorial : " + fact);
//         sc.close();
//     }
// }
// import java.util.Scanner;
// public class day9 {
//     public static void main(String args[]) {
//         Scanner sc = new Scanner(System.in);
//         System.out.print("Enter digits : ");
//         int num = sc.nextInt();

```

```

//      int digit = 0;
//      int reverse = 0;
//      while (num > 0) {
//          digit = num % 10;
//          reverse = reverse * 10 + digit;
//          num = num / 10;
//      }
//      System.out.println("Reverse : " + reverse);
//  }
// }
// Enter digits : 12345
// Reverse : 54321
// import java.util.Scanner;
// public class day9 {
//     public static void main(String args[]) {
//         Scanner sc = new Scanner(System.in);
//         System.out.print("Enter digits : ");
//         int num = sc.nextInt();
//         int sum = 0;
//         int digit = 0;
//         while (num > 0) {
//             digit = num % 10;
//             sum = sum + digit;
//             num = num / 10;
//         }
//         System.out.println("Sum of digits : " + sum);
//     }
// }
// Enter digits : 1234
// Sum of digits : 10
// import java.util.Scanner;
// public class day9 {
//     public static void main(String args[]) {
//         Scanner sc = new Scanner(System.in);
//         System.out.print("Enter digits : ");
//         int num = sc.nextInt();
//         int digit = 0;
//         int even = 0;
//         int odd = 0;
//         while (num > 0) {
//             digit = num % 10;
//             if (digit % 2 == 0) {
//                 even++;
//             }
//             if (digit % 2 == 1) {
//                 odd++;
//             }
//             num = num / 10;
//         }
//         System.out.println("Even : " + even);
//         System.out.println("Odd : " + odd);
//     }
// }
// Enter digits : 78718741
// Even :3
// Odd :5
// public class day9 {
//     static int square(int n) {
//         return n * n;
//     }
//     public static void main(String args[]) {
//         int result = square(5);
//         System.out.println("Square : " + result);
//     }
// }
// Square : 25
// public class day9 {
//     static int add(int a, int b) {
//         return a + b;
//     }
//     public static void main(String args[]) {
//         int result = add(15, 39);
//         System.out.println("Sum : " + result);
//     }
// }
// Sum : 54
// public class day9 {
//     static boolean isEven(int num) {
//         if (num % 2 == 0) {
//             return true;
//         } else {
//             return false;
//         }
//     }

```

```

//      }
//    }
//    public static void main(String args[]) {
//      boolean result = isEven(31);
//      System.out.println("Even : " + result);
//    }
//  }
// Even : false
// class day9 {
//   static int digits(int n) {
//     int sum = 0;
//     int digit = 0;
//     while (n > 0) {
//       digit = n % 10;
//       sum = sum + digit;
//       n = n / 10;
//     }
//     return sum;
//   }
//   public static void main(String args[]) {
//     int result = digits(58392);
//     System.out.println("Sum : " + result);
//   }
// }
// Sum : 27
// public class day9 {
//   public static void main(String args[]) {
//     int arr[] = {10, 20, 30, 40, 50};
//     int sum = 0;
//     for (int i = 0; i < arr.length; i++) {
//       sum = sum + arr[i];
//     }
//     System.out.println("Sum : " + sum);
//   }
// }
// Sum : 150
// public class day9 {
//   static int findSum(int arr[]) {
//     int sum = 0;
//     for (int i = 0; i < arr.length; i++) {
//       sum = sum + arr[i];
//     }
//     return sum;
//   }
//   public static void main(String args[]) {
//     int numbers[] = {10, 20, 30, 40, 50};
//     int result = findSum(numbers);
//     System.out.println("Sum : " + result);
//   }
// }
// Sum : 150
// public class day9 {
//   static int findMax(int arr[]) {
//     int largest = 0;
//     for (int i = 0; i < arr.length; i++) {
//       if (arr[i] > largest) {
//         largest = arr[i];
//       }
//     }
//     return largest;
//   }
//   public static void main(String args[]) {
//     int max[] = {10, 24, 18, 92, 33};
//     int result = findMax(max);
//     System.out.println("Largest : " + result);
//   }
// }

```

day10.java - saved 07 Sep 2026

```

// Methods + array
// public class day10 {
//   static void newArray(int[] arr) {
//     for (int i = 0; i < arr.length; i++) {
//       System.out.println(arr[i]);
//     }
//   }
//   public static void main(String args[]) {
//     int number[] = {10, 20, 30, 40, 50};
//     newArray(number);
//   }
// }

```

```

// }
// 10
// 20
// 30
// 40
// 50
// class day10 {
//     static void newArray(int arr[]) {
//         for (int i = 0; i < arr.length; i++) {
//             if (arr[i] % 2 == 0) {
//                 System.out.println(arr[i]);
//             }
//         }
//     }
//     public static void main(String args[]) {
//         int arr[] = {110, 21, 29, 34, 29, 22};
//         newArray(arr);
//     }
// }
// 110
// 34
// 22
// public class day10 {
//     static int sumArray(int arr[]) {
//         int sum = 0;
//         for (int i = 0; i < arr.length; i++) {
//             sum = sum + arr[i];
//         }
//         return sum;
//     }
//     public static void main(String args[]) {
//         int numbers[] = {10, 20, 30, 40, 50};
//         int result = sumArray(numbers);
//         System.out.println("Sum : " + result);
//     }
// }
// Sum : 150
// public class day10 {
//     static int findMax(int arr[]) {
//         int largest = arr[0];
//         for (int i = 1; i < arr.length; i++) {
//             if (arr[i] > largest) {
//                 largest = arr[i];
//             }
//         }
//         return largest;
//     }
//     public static void main(String args[]) {
//         int numbers[] = {-10, -111, -23, -64, -1334, -54};
//         int result = findMax(numbers);
//         System.out.println("Largest :" + result);
//     }
// }
// Largest : -10
// public class day10 {
//     static int newSmall(int arr[]) {
//         int smallest = arr[0];
//         for (int i = 1; i < arr.length; i++) {
//             if (arr[i] < smallest) {
//                 smallest = arr[i];
//             }
//         }
//         return smallest;
//     }
//     public static void main(String args[]) {
//         int numbers[] = {10, 3, 45, 224, 55};
//         int res = newSmall(numbers);
//         System.out.println("Smallest :" + res);
//     }
// }
// Smallest :3
// public class day10 {
//     static int newArray(int arr[]) {
//         int count = 0;
//         for (int i = 0; i < arr.length; i++) {
//             if (arr[i] % 2 == 0) {
//                 count++;
//             }
//         }
//         return count;
//     }
//     public static void main(String args[]) {

```

```

//      int numbers[] = {10, 12, 5, 64, 23};
//      int res = newArray(numbers);
//      System.out.println("Evens : " + res);
//  }
// }
// Evens : 3
// public class day10 {
//     static int newArray(int arr[]) {
//         int count = 0;
//         for (int i = 0; i < arr.length; i++) {
//             if (arr[i] % 2 != 0) {
//                 count++;
//             }
//         }
//         return count;
//     }
//     public static void main(String args[]) {
//         int numbers[] = {10, 32, 45, 34, 2, 4, 5, 654, 3};
//         int res = newArray(numbers);
//         System.out.println("odd : " + res);
//     }
// }
// odd : 3
// public class day10 {
//     static int evenSum(int arr[]) {
//         int sum = 0;
//         for (int i = 0; i < arr.length; i++) {
//             if (arr[i] % 2 == 0) {
//                 sum = sum + arr[i];
//             }
//         }
//         return sum;
//     }
//     public static void main(String args[]) {
//         int numbers[] = {24, 2, 4, 5, 21, 86, 56};
//         int res = evenSum(numbers);
//         System.out.println("Even sum : " + res);
//     }
// }
// Even sum : 172
// public class day10 {
//     static int secondL(int arr[]) {
//         int largest;
//         int second;
//         if (arr[0] > arr[1]) {
//             largest = arr[0];
//             second = arr[1];
//         } else {
//             largest = arr[1];
//             second = arr[0];
//         }
//         for (int i = 2; i < arr.length; i++) {
//             if (arr[i] > largest) {
//                 second = largest;
//                 largest = arr[i];
//             } else if (arr[i] > second) {
//                 second = arr[i];
//             }
//         }
//         return second;
//     }
//     public static void main(String args[]) {
//         int numbers[] = {23, 53, 14, 65, -13, 43};
//         int res = secondL(numbers);
//         System.out.println("Second largest : " + res);
//     }
// }

```

day11.java - saved 08 Sep 2026

```

// public class day11 {
//     static boolean newArray(int arr[], int target) {
//         boolean ok = false;
//         for (int i = 0; i < arr.length; i++) {
//             if (arr[i] == target) {
//                 ok = true;
//             }
//         }
//         return ok;
//     }
//     public static void main(String args[]) {

```

```

//      int numbers[] = {10, 32, 53, 39, 29};
//      System.out.println(newArray(numbers, 39));
//      System.out.println(newArray(numbers, 20));
//  }
// }
// true
// false
// public class day11 {
//     static int indexS(int arr[], int target) {
//         for (int i = 0; i < arr.length; i++) {
//             if (arr[i] == target) {
//                 return i;
//             }
//         }
//         return -1;
//     }
//     public static void main(String args[]) {
//         int numbers[] = {10, 43, 32, 63, 22};
//         System.out.println(indexS(numbers, 43));
//         System.out.println(indexS(numbers, 22));
//     }
// }
// 1
// 4
// public class day11 {
//     static int newArray(int arr[], int target) {
//         int count = 0;
//         for (int i = 0; i < arr.length; i++) {
//             if (arr[i] == target) {
//                 count++;
//             }
//         }
//         return count;
//     }
//     public static void main(String args[]) {
//         int numbers[] = {10, 20, 20, 20, 109, 10};
//         System.out.println(newArray(numbers, 10));
//         System.out.println(newArray(numbers, 20));
//     }
// }
// 2
// 3
// public class day11 {
//     static void reverse(int arr[]) {
//         int temp;
//         int j = arr.length - 1;
//         for (int i = 0; i < j; i++) {
//             temp = arr[i];
//             arr[i] = arr[j];
//             arr[j] = temp;
//             j--;
//         }
//     }
//     public static void main(String args[]) {
//         int numbers[] = {10, 229, 308, 4012, 503};
//         reverse(numbers);
//         System.out.print("Reverse : ");
//         for (int i = 0; i < numbers.length; i++) {
//             System.out.print(numbers[i] + " ");
//         }
//     }
// }
// Reverse : 503 4012 308 229 10
public class day11 {
    static double averageI(int arr[]) {
        double total = 0.0;
        for (int i = 0; i < arr.length; i++) {
            total = total + arr[i];
        }
        return total / arr.length;
    }
    public static void main(String args[]) {
        int numbers[] = {10, 20, 30, 40, 50};
        double res = averageI(numbers);
        System.out.println("Average : " + res);
    }
}
// Average : 30.0

```

day12.java - saved 09 Sep 2026

```

// class day12 {
//     static int findLargest(int arr[]) {
//         int largest = arr[0];
//         for (int i = 1; i < arr.length; i++) {
//             if (arr[i] > largest) {
//                 largest = arr[i];
//             }
//         }
//         return largest;
//     }
//     public static void main(String args[]) {
//         int num[] = {10, 20, 231, 324, 13, 432, 643, 65, 22};
//         int res = findLargest(num);
//         System.out.println("Largest : " + res);
//     }
// }
// Largest : 643
// public class day12 {
//     static int findSmallest(int arr[]) {
//         int smallest = arr[0];
//         for (int i = 1; i < arr.length; i++) {
//             if (arr[i] < smallest) {
//                 smallest = arr[i];
//             }
//         }
//         return smallest;
//     }
//     public static void main(String args[]) {
//         int num[] = {10, 20, 301, 424, 48, 9};
//         int res = findSmallest(num);
//         System.out.println("Smallest : " + res);
//     }
// }
// Smallest : 9
// public class day12 {
//     static void countEvenOdd(int arr[]) {
//         int even = 0;
//         int odd = 0;
//         for (int i = 0; i < arr.length; i++) {
//             if (arr[i] % 2 == 0) {
//                 even++;
//             } else {
//                 odd++;
//             }
//         }
//         System.out.println("Even: " + even);
//         System.out.println("Odd: " + odd);
//     }
//     public static void main(String args[]) {
//         int[] num = {10, 25, 8, 42, 17, 6, 31};
//         countEvenOdd(num);
//     }
// }
// Even: 4
// Odd: 3
// public class day12 {
//     static boolean searchElement(int arr[], int target) {
//         boolean found = false;
//         for (int i = 0; i < arr.length; i++) {
//             if (arr[i] == target) {
//                 found = true;
//             }
//         }
//         return found;
//     }
//     public static void main(String args[]) {
//         int num[] = {10, 221, 53, 13, 54};
//         System.out.println(searchElement(num, 13));
//         System.out.println(searchElement(num, 99));
//     }
// }
// true
// false
// public class day12 {
//     static int secondL(int arr[]) {
//         int largest;
//         int second;
//         if (arr[0] > arr[1]) {
//             largest = arr[0];
//             second = arr[1];
//         }
//     }
// }

```

```

//      } else {
//          largest = arr[1];
//          second = arr[0];
//      }
//      for (int i = 2; i < arr.length; i++) {
//          if (arr[i] > largest) {
//              second = largest;
//              largest = arr[i];
//          } else if (arr[i] > second) {
//              second = arr[i];
//          }
//      }
//      return second;
//  }
//  public static void main(String args[]) {
//      int number[] = {110, 32, 14, 23, 42, 13};
//      int res = secondL(number);
//      System.out.println("Second largest : " + res);
//  }
// }
// Second largest : 42
// public class day12 {
//     static void reverse(int arr[]) {
//         int temp;
//         int j = arr.length - 1;
//         for (int i = 0; i < j; i++) {
//             temp = arr[i];
//             arr[i] = arr[j];
//             arr[j] = temp;
//             j--;
//         }
//     }
//     public static void main(String args[]) {
//         int numbers[] = {10, 20, 30, 40, 50};
//         reverse(numbers);
//         System.out.print("Reverse : ");
//         for (int i = 0; i < numbers.length; i++) {
//             System.out.print(numbers[i] + " ");
//         }
//     }
// }
// Reverse : 50 40 30 20 10
// public class day12 {
//     static double average(int arr[]) {
//         int total = arr[0];
//         for (int i = 1; i < arr.length; i++) {
//             total = total + arr[i];
//         }
//         return (double) total / arr.length;
//     }
//     public static void main(String args[]) {
//         int num[] = {10, 34, 13, 53, 21};
//         double res = average(num);
//         System.out.println("Average : " + res);
//     }
// }
// Average : 26.2
// public class day12 {
//     static int index(int arr[], int target) {
//         for (int i = 0; i < arr.length; i++) {
//             if (arr[i] == target) {
//                 return i;
//             }
//         }
//         return -1;
//     }
//     public static void main(String args[]) {
//         int num[] = {10, 42, 54, 16, 31};
//         System.out.println("index : " + index(num, 54));
//     }
// }

```

day13.java - saved 10 Sep 2026 - ENTIRE FILE IS COMMENTED OUT (kept as archive)

```

// public class day13 {
//     static int indexFind(int arr[], int target) {
//         for (int i = 0; i < arr.length; i++) {
//             if (arr[i] == target) {
//                 return i;
//             }
//         }
//     }
// }

```

```

//      return -1;
//    }
//    public static void main(String args[]) {
//      int num[] = {10, 32, 53, 54, 24};
//      System.out.println("Index : " + indexFind(num, 53 ));
//      System.out.println("Index : " + indexFind(num, 24));
//      System.out.println("Index : " + indexFind(num, 99));
//    }
//  }
// Index : 2
// Index : 4
// Index : -1
// public class day13 {
//   static int findOccurance(int arr[], int target) {
//     int count = 0;
//     for (int i = 0; i < arr.length; i++) {
//       if (arr[i] == target) {
//         count++;
//       }
//     }
//     return count;
//   }
//   public static void main(String args[]) {
//     int num[] = {10, 10, 32, 13, 42, 10, 2, 2, 12, 664};
//     int res = findOccurance(num, 10);
//     System.out.println("Occurance : " + res);
//   }
// }
// Occurance : 3
// public class day13 {
//   static int sumOfElements(int arr[]) {
//     int sum = arr[0];
//     for (int i = 1; i < arr.length; i++) {
//       sum = sum + arr[i];
//     }
//     return sum;
//   }
//   public static void main(String args[]) {
//     int num[] = {10, 32, 42, 54, 24};
//     int res = sumOfElements(num);
//     System.out.println("Sum : " + res);
//   }
// }
// Sum : 162
// public class day13 {
//   static void reverse(int arr[]) {
//     int temp;
//     int j = arr.length - 1;
//     for (int i = 0; i < j; i++) {
//       temp = arr[i];
//       arr[i] = arr[j];
//       arr[j] = temp;
//       j--;
//     }
//   }
//   public static void main(String args[]) {
//     int num[] = {10, 20, 30, 40, 50, 60};
//     reverse(num);
//     System.out.print("Reverse : ");
//     for (int i = 0; i < num.length; i++) {
//       System.out.print(num[i] + " ");
//     }
//   }
// }
// Reverse : 60 50 40 30 20 10
// public class day13 {
//   static int secondL(int arr[]) {
//     int second;
//     int largest;
//     if (arr[0] > arr[1]) {
//       largest = arr[0];
//       second = arr[1];
//     } else {
//       largest = arr[1];
//       second = arr[0];
//     }
//     for (int i = 0; i < arr.length; i++) {
//       if (arr[i] > largest) {
//         second = largest;
//         largest = arr[i];
//       } else if (arr[i] > second) {
//         second = arr[i];
//       }
//     }
//   }
// }

```

```
//      }
//      }
//      return second;
//  }
//  public static void main(String args[]) {
//      int num[] = {10, 32, 42, 543, 2344, 13, 3222, 5, 432, 3221};
//      int res = secondL(num);
//      System.out.println("Second largest : " + res);
//  }
// }
// Second largest : 3221
// public class day13 {
//     static int duplicate(int arr[]) {
//         for (int i = 0; i < arr.length; i++) {
//             for (int j = i + 1; j < arr.length; j++) {
//                 if (arr[i] == arr[j]) {
//                     return arr[i];
//                 }
//             }
//         }
//         return -1;
//     }
//     public static void main(String args[]) {
//         int num[] = {10, 20, 30, 10, 32, 43};
//         System.out.println("Duplicate: " + duplicate(num));
//     }
// }
// Duplicate: 10
```

day14.java - saved 18 Sep 2026

```
// -----RECALL -----
// public class day14 {
//     static void reverse(int arr[]) {
//         int temp;
//         int j = arr.length - 1;
//         for (int i = 0; i < j; i++) {
//             temp = arr[i];
//             arr[i] = arr[j];
//             arr[j] = temp;
//             j--;
//         }
//     }
//     public static void main(String args[]) {
//         int num[] = {10, 20, 30, 40, 50};
//         reverse(num);
//         System.out.print("Reverse :");
//         for (int i = 0; i < num.length; i++) {
//             System.out.print(num[i] + " ");
//         }
//     }
// }
// Reverse :50 40 30 20 10
// public class day14 {
//     static int secondL(int arr[]) {
//         int largest;
//         int second;
//         if (arr[0] > arr[1]) {
//             largest = arr[0];
//             second = arr[1];
//         } else {
//             largest = arr[1];
//             second = arr[0];
//         }
//         for (int i = 2; i < arr.length; i++) {
//             if (arr[i] > largest) {
//                 second = largest;
//                 largest = arr[i];
//             } else if (arr[i] > second) {
//                 second = arr[i];
//             }
//         }
//         return second;
//     }
//     public static void main(String args[]) {
//         int num[] = {10, 22, 43, 31, 44, 133};
//         int res = secondL(num);
//         System.out.println("Second largest : " + res);
//     }
// }
// Second largest : 44
```

```
// public class day14 {
//     static int occurrence(int arr[], int target) {
//         int count = 0;
//         for (int i = 0; i < arr.length; i++) {
//             if (arr[i] == target) {
//                 count++;
//             }
//         }
//         return count;
//     }
// }
// public static void main(String args[]) {
//     int num[] = {10, 32, 13, 10, 10, 32, 42};
//     System.out.println("occurance : " + occurrence(num, 10));
// }
// }
// occurrence : 3
public class day14 {
    static int frequency(int arr[], int target) {
        int count = 0;
        for (int i = 0; i < arr.length; i++) {
            if (arr[i] == target) {
                count++;
            }
        }
        return count;
    }
    public static void main(String args[]) {
        int num[] = {10, 32, 10, 42, 10, 20, 20};
        for (int i = 0; i < num.length; i++) {
            boolean alreadySeen = false;
            for (int j = 0; j < i; j++) {
                if (num[i] == num[j]) {
                    alreadySeen = true;
                    break;
                }
            }
            if (!alreadySeen) {
                int count = frequency(num, num[i]);
                System.out.println(num[i] + " > " + count);
            }
        }
    }
}
// 10 > 3
// 32 > 1
// 42 > 1
// 20 > 2
```

day15.java - saved 12 Sep 2026

```
// -----RECALL-----
// public class day15 {
//     static void reverse(int arr[]) {
//         int temp;
//         int j = arr.length - 1;
//         for (int i = 0; i < j; i++) {
//             temp = arr[i];
//             arr[i] = arr[j];
//             arr[j] = temp;
//             j--;
//         }
//     }
// }
// public static void main(String args[]) {
//     int num[] = {19, 53, 13, 55, 12, 52};
//     reverse(num);
//     System.out.print("Reverse : ");
//     for (int i = 0; i < num.length; i++) {
//         System.out.print(num[i] + " ");
//     }
// }
// }
// REVERSE DONE
// public class day15 {
//     static int secondL(int arr[]) {
//         int largest;
//         int second;
//         if (arr[0] > arr[1]) {
//             largest = arr[0];
//             second = arr[1];
//         } else {
//             largest = arr[1];
//         }
//     }
// }
```

```

//      second = arr[0];
//    }
//    for (int i = 2; i < arr.length; i++) {
//      if (arr[i] > largest) {
//        second = largest;
//        largest = arr[i];
//      } else if (arr[i] > second) {
//        second = arr[i];
//      }
//    }
//    return second;
//  }
//  public static void main(String args[]) {
//    int num[] = {193, 13, 5443, 1334, 1342, 2432};
//    int res = secondL(num);
//    System.out.println("Second largest : " + res);
//  }
// }
// SECOND LARGEST DONE
// public class day15 {
//   static int frequency(int arr[], int target) {
//     int count = 0;
//     for (int i = 0; i < arr.length; i++) {
//       if (arr[i] == target) {
//         count++;
//       }
//     }
//     return count;
//   }
//   public static void main(String args[]) {
//     int num[] = {10, 10, 10, 43, 392, 43, 13, 533};
//     for (int i = 0; i < num.length; i++) {
//       boolean alreadySeen = false;
//       for (int j = 0; j < i; j++) {
//         if (num[i] == num[j]) {
//           alreadySeen = true;
//           break;
//         }
//       }
//       if (!alreadySeen) {
//         int count = frequency(num, num[i]);
//         System.out.println(num[i] + " > " + count);
//       }
//     }
//   }
// }
// FREQUENCY OF EACH ELEMENT DONE
public class day15 {
  static int frequency(int arr[], int target) {
    int count = 0;
    for (int i = 0; i < arr.length; i++) {
      if (arr[i] == target) {
        count++;
      }
    }
    return count;
  }
  static int mostFrequent(int arr[]) {
    int mostfrequent = arr[0];
    int highestFrequency = frequency(arr, arr[0]);
    for (int i = 1; i < arr.length; i++) {
      int count = frequency(arr, arr[i]);
      if (count > highestFrequency) {
        mostfrequent = arr[i];
        highestFrequency = count;
      }
    }
    return mostfrequent;
  }
  public static void main(String args[]) {
    int num[] = {10, 10, 20, 13, 2, 3, 23, 2, 3, 3,};
    int res = mostFrequent(num);
    System.out.println("Most frequent : " + res);
  }
}
// Most frequent : 3

```

day16.java - saved 13 Sep 2026

```

// SATURDAY REVISION-----?>>
// import java.util.Scanner;

```

```

// public class day16 {
//     public static void main(String args[]) {
//         Scanner sc = new Scanner(System.in);
//         System.out.print("Enter the digits : ");
//         int num = sc.nextInt();
//         int sum = 0;
//         int digit = 0;
//         while (num > 0) {
//             digit = num % 10;
//             sum = sum + digit;
//             num = num / 10;
//         }
//         System.out.println("Sum of digits : " + sum);
//         sc.close();
//     }
// }
// Enter the digits : 122345
// Sum of digits : 17
// import java.util.Scanner;
// public class day16 {
//     public static void main(String args[]) {
//         Scanner sc = new Scanner(System.in);
//         System.out.print("Enter digits : ");
//         int num = sc.nextInt();
//         int reverse = 0;
//         int digit = 0;
//         while (num > 0) {
//             digit = num % 10;
//             reverse = reverse * 10 + digit;
//             num = num / 10;
//         }
//         System.out.println("Reverse : " + reverse);
//     }
// }
// Enter digits : 12345
// Reverse : 54321
// import java.util.Scanner;
// public class day16 {
//     public static void main(String args[]) {
//         Scanner sc = new Scanner(System.in);
//         System.out.print("Enter a number : ");
//         int num = sc.nextInt();
//         int fact = 1;
//         for (int i = 1; i <= num; i++) {
//             fact = fact * i;
//         }
//         System.out.println("Factorial : " + fact);
//     }
// }
// Enter a number : 6
// Factorial : 720
// import java.util.*;
// public class day16 {
//     public static void main(String args[]) {
//         Scanner sc = new Scanner(System.in);
//         System.out.print("Enter the digits : ");
//         int num = sc.nextInt();
//         int orig = num;
//         int reverse = 0;
//         int digit = 0;
//         while (num > 0) {
//             digit = num % 10;
//             num = num / 10;
//             reverse = reverse * 10 + digit;
//         }
//         if (orig == reverse) {
//             System.out.println("The number is Palindrome number ");
//         } else {
//             System.out.println("The number is not Palindrome number ");
//         }
//     }
// }
// Enter the digits : 111
// The number is Palindrome number
// import java.util.*;
// public class day16 {
//     public static void main(String args[]) {
//         Scanner sc = new Scanner(System.in);
//         System.out.print("Enter digits : ");
//         int num = sc.nextInt();
//         int count = 0;
//         int digit = 0;

```

```

//      while (num > 0) {
//          digit = num % 10;
//          count++;
//          num = num / 10;
//      }
//      System.out.println("Digits : " + count);
//  }
// }
// Enter digits : 123456
// Digits : 6
// import java.util.Scanner;
// public class day16 {
//     public static void main(String args[]) {
//         Scanner sc = new Scanner(System.in);
//         System.out.print("Enter the digits : ");
//         int num = sc.nextInt();
//         int digit = 0;
//         int largest = 0;
//         while (num > 0) {
//             digit = num % 10;
//             if (digit > largest) {
//                 largest = digit;
//             }
//             num = num / 10;
//         }
//         System.out.println("Largest : " + largest);
//     }
// }
// Enter the digits : 12322728
// Largest : 8
// import java.util.*;
// public class day16 {
//     public static void main(String args[]) {
//         Scanner sc = new Scanner(System.in);
//         System.out.print("Enter the digits : ");
//         int num = sc.nextInt();
//         int smallest = 9;
//         int digit = 0;
//         while (num > 0) {
//             digit = num % 10;
//             if (digit < smallest) {
//                 smallest = digit;
//             }
//             num = num / 10;
//         }
//         System.out.println("Smallest : " + smallest);
//     }
// }
// Enter the digits : 324234
// Smallest : 2
// import java.util.Scanner;
// public class day16 {
//     public static void main(String args[]) {
//         Scanner sc = new Scanner(System.in);
//         System.out.print("Enter digits : ");
//         int num = sc.nextInt();
//         int digit = 0;
//         int evencounter = 0;
//         int oddcounter = 0;
//         while (num > 0) {
//             digit = num % 10;
//             if (digit % 2 == 0) {
//                 evencounter++;
//             } else {
//                 oddcounter++;
//             }
//             num = num / 10;
//         }
//         System.out.println("Evens : " + evencounter);
//         System.out.println("Odds : " + oddcounter);
//     }
// }
// Enter digits : 362789
// Evens : 3
// Odds : 3
// import java.util.Scanner;
// public class day16 {
//     public static void main(String args[]) {
//         Scanner sc = new Scanner(System.in);
//         System.out.print("Enter digits : ");
//         int num = sc.nextInt();
//         int sum = 0;

```

```

//      int digit = 0;
//      while (num > 0) {
//          digit = num % 10;
//          if (digit % 2 == 0) {
//              sum = sum + digit;
//          }
//          num = num / 10;
//      }
//      System.out.println("Sum of Evens : " + sum);
//  }
// }
// Enter digits : 321432
// Sum of Evens : 8
import java.util.*;
public class day16 {
    public static void main(String args[]) {
        Scanner sc = new Scanner(System.in);
        int choices;
        double average = 0.0;
        do {
            System.out.println();
            System.out.println("=====ANALYZER=====");
            System.out.println("1.Number Analyzer ");
            System.out.println("2.Student marks Analyzer ");
            System.out.println("3.Exit");
            System.out.print("Enter your choice : ");
            choices = sc.nextInt();
            switch (choices) {
                case 1:
                    System.out.println("Number Analyzer Selected ----> ");
                    System.out.println();
                    System.out.print("Enter the number : ");
                    int num1 = sc.nextInt();
                    int digit = 0;
                    int sum = 0;
                    int temp = num1;
                    int reverse = 0;
                    int original = num1;
                    long fact = 1;
                    int largest = 0;
                    int smallest = 9;
                    int temp2 = original;
                    int evencount = 0;
                    int oddcount = 0;
                    int temp3 = original;
                    int evensum = 0;
                    while (num1 > 0) {
                        digit = num1 % 10;
                        sum = sum + digit;
                        num1 = num1 / 10;
                    }
                    while (temp > 0) {
                        digit = temp % 10;
                        reverse = reverse * 10 + digit;
                        temp = temp / 10;
                    }
                    if (original == reverse) {
                        System.out.println("It's a palindrome");
                    } else {
                        System.out.println("Its not palindrome ");
                    }
                    for (int i = 1; i <= original; i++) {
                        fact = fact * i;
                    }
                    while (temp2 > 0) {
                        digit = temp2 % 10;
                        if (digit > largest) {
                            largest = digit;
                        }
                        if (digit < smallest) {
                            smallest = digit;
                        }
                        temp2 = temp2 / 10;
                    }
                    while (temp3 > 0) {
                        digit = temp3 % 10;
                        if (digit % 2 == 0) {
                            evensum = evensum + digit;
                            evencount++;
                        } else {
                            oddcount++;
                        }
                    }

```

```

        temp3 = temp3 / 10;
    }
    System.out.println("The sum of digits : " + sum);
    System.out.println("Reverse : " + reverse);
    System.out.println("Factorial : " + fact);
    System.out.println("Largest : " + largest);
    System.out.println("Smallest : " + smallest);
    System.out.println("Evens : " + evencount);
    System.out.println("Odds : " + oddcount);
    System.out.println("Even sum : " + evensum);
    System.out.println();
    break;
}
case 2:
    System.out.println("Student marks analyzer selected ---> ");
    System.out.println();
    int marks[] = new int[5];
    int total = 0;
    for (int i = 0; i < marks.length; i++) {
        System.out.print("Enter marks : ");
        marks[i] = sc.nextInt();
        total = total + marks[i];
    }
    int largest1 = marks[0];
    int smallest1 = marks[0];
    for (int i = 0; i < marks.length; i++) {
        if (marks[i] > largest1) {
            largest1 = marks[i];
        }
        if (marks[i] < smallest1) {
            smallest1 = marks[i];
        }
    }
    average = (double) total / marks.length;
    int evencount1 = 0;
    for (int i = 0; i < marks.length; i++) {
        if (marks[i] % 2 == 0) {
            evencount1++;
        }
    }
    boolean found = false;
    System.out.print("Enter the target : ");
    int target = sc.nextInt();
    int index = -1;
    for (int i = 0; i < marks.length; i++) {
        if (marks[i] == target) {
            found = true;
            index = i;
            break;
        }
    }
    int count2 = 0;
    System.out.print("Enter target to count : ");
    int target1 = sc.nextInt();
    for (int i = 0; i < marks.length; i++) {
        if (marks[i] == target1) {
            count2++;
        }
    }
    int largest2 = 0;
    int second = 0;
    if (marks[0] > marks[1]) {
        largest2 = marks[0];
        second = marks[1];
    } else {
        largest2 = marks[1];
        second = marks[0];
    }
    for (int i = 2; i < marks.length; i++) {
        if (marks[i] > largest2) {
            second = largest2;
            largest2 = marks[i];
        } else if (marks[i] > second) {
            second = marks[i];
        }
    }
    int j = marks.length - 1;
    int temp5;
    for (int i = 0; i < j; i++) {
        temp5 = marks[i];
        marks[i] = marks[j];
        marks[j] = temp5;
        j--;
    }

```

```

    }
    System.out.print("Reverse :");
    for (int i = 0; i < marks.length; i++) {
        System.out.print(marks[i] + " ");
    }
    System.out.println();
    System.out.println("Total marks : " + total);
    System.out.println("Average marks : " + average);
    System.out.println("Largest : " + largest1);
    System.out.println("Smallest : " + smallest1);
    System.out.println("Even marks : " + evencount1);
    System.out.println("Search marks found : " + found);
    System.out.println("Index : " + index);
    System.out.println("Number of occurrences : " + count2);
    System.out.println("Second largest : " + second);
    System.out.println();
    break;
case 3:
    System.out.println("Exiting... ");
    break;
default:
    System.out.println("Invalid input ");
}
} while (choices != 3);
}
}

```

day17.java - saved 14 Sep 2026

```

import java.util.*;
public class day17 {
    public static void main(String args[]) {
        Scanner sc = new Scanner(System.in);
        int choices;
        double average = 0.0;
        do {
            System.out.println();
            System.out.println("=====ANALYZER=====");
            System.out.println("1.Number Analyzer ");
            System.out.println("2.Student marks Analyzer ");
            System.out.println("3.Exit");
            System.out.print("Enter your choice : ");
            choices = sc.nextInt();
            switch (choices) {
                case 1:
                    System.out.println("Number Analyzer Selected ----> ");
                    System.out.println();
                    System.out.print("Enter the number : ");
                    int num1 = sc.nextInt();
                    int digit = 0;
                    int sum = 0;
                    int temp = num1;
                    int reverse = 0;
                    int original = num1;
                    long fact = 1;
                    int largest = 0;
                    int smallest = 9;
                    int temp2 = original;
                    int evencount = 0;
                    int oddcount = 0;
                    int temp3 = original;
                    int evensum = 0;
                    while (num1 > 0) {
                        digit = num1 % 10;
                        sum = sum + digit;
                        num1 = num1 / 10;
                    }
                    while (temp > 0) {
                        digit = temp % 10;
                        reverse = reverse * 10 + digit;
                        temp = temp / 10;
                    }
                    if (original == reverse) {
                        System.out.println("It's a palindrome");
                    } else {
                        System.out.println("Its not palindrome ");
                    }
                    for (int i = 1; i <= original; i++) {
                        fact = fact * i;
                    }
                    while (temp2 > 0) {

```

```

        digit = temp2 % 10;
        if (digit > largest) {
            largest = digit;
        }
        if (digit < smallest) {
            smallest = digit;
        }
        temp2 = temp2 / 10;
    }
    while (temp3 > 0) {
        digit = temp3 % 10;
        if (digit % 2 == 0) {
            evensum = evensum + digit;
            evencount++;
        } else {
            oddcount++;
        }
        temp3 = temp3 / 10;
    }
    System.out.println("The sum of digits : " + sum);
    System.out.println("Reverse : " + reverse);
    System.out.println("Factorial : " + fact);
    System.out.println("Largest : " + largest);
    System.out.println("Smallest : " + smallest);
    System.out.println("Evens : " + evencount);
    System.out.println("Odds : " + oddcount);
    System.out.println("Even sum : " + evensum);
    System.out.println();
    break;
case 2:
    System.out.println("Student marks analyzer selected ---> ");
    System.out.println();
    int marks[] = new int[5];
    int total = 0;
    for (int i = 0; i < marks.length; i++) {
        System.out.print("Enter marks : ");
        marks[i] = sc.nextInt();
        total = total + marks[i];
    }
    int largest1 = marks[0];
    int smallest1 = marks[0];
    for (int i = 0; i < marks.length; i++) {
        if (marks[i] > largest1) {
            largest1 = marks[i];
        }
        if (marks[i] < smallest1) {
            smallest1 = marks[i];
        }
    }
    average = (double) total / marks.length;
    int evencount1 = 0;
    for (int i = 0; i < marks.length; i++) {
        if (marks[i] % 2 == 0) {
            evencount1++;
        }
    }
    boolean found = false;
    System.out.print("Enter the target : ");
    int target = sc.nextInt();
    int index = -1;
    for (int i = 0; i < marks.length; i++) {
        if (marks[i] == target) {
            found = true;
            index = i;
            break;
        }
    }
    int count2 = 0;
    System.out.print("Enter target to count : ");
    int target1 = sc.nextInt();
    for (int i = 0; i < marks.length; i++) {
        if (marks[i] == target1) {
            count2++;
        }
    }
    int largest2 = 0;
    int second = 0;
    if (marks[0] > marks[1]) {
        largest2 = marks[0];
        second = marks[1];
    } else {
        largest2 = marks[1];

```

```

        second = marks[0];
    }
    for (int i = 2; i < marks.length; i++) {
        if (marks[i] > largest2) {
            second = largest2;
            largest2 = marks[i];
        } else if (marks[i] > second) {
            second = marks[i];
        }
    }
    int j = marks.length - 1;
    int temp5;
    for (int i = 0; i < j; i++) {
        temp5 = marks[i];
        marks[i] = marks[j];
        marks[j] = temp5;
        j--;
    }
    System.out.print("Reverse :");
    for (int i = 0; i < marks.length; i++) {
        System.out.print(marks[i] + " ");
    }
    System.out.println();
    System.out.println("Total marks : " + total);
    System.out.println("Average marks : " + average);
    System.out.println("Largest : " + largest1);
    System.out.println("Smallest : " + smallest1);
    System.out.println("Even marks : " + evencount1);
    System.out.println("Search marks found : " + found);
    System.out.println("Index : " + index);
    System.out.println("Number of occurrences : " + count2);
    System.out.println("Second largest : " + second);
    System.out.println();
    break;
case 3:
    System.out.println("Exiting... ");
    break;
default:
    System.out.println("Invalid input ");
}
} while (choices != 3);
}
}

```

day18.java - saved 21 Sep 2026

```

import java.util.*;
public class day18 {
    // ===== ARRAY METHODS =====
    static boolean contains(int arr[], int target) {
        for (int i = 0; i < arr.length; i++) {
            if (arr[i] == target) {
                return true;
            }
        }
        return false;
    }
    static int countOccurrence(int arr[], int target) {
        int count = 0;
        for (int i = 0; i < arr.length; i++) {
            if (arr[i] == target) {
                count++;
            }
        }
        return count;
    }
    static int index(int arr[], int target) {
        for (int i = 0; i < arr.length; i++) {
            if (arr[i] == target) {
                return i;
            }
        }
        return -1;
    }
    static int sumOfArray(int arr[]) {
        int sum = 0;
        for (int i = 0; i < arr.length; i++) {
            sum = sum + arr[i];
        }
        return sum;
    }
}

```

```
// ===== ARRAY ANALYZER =====
static void arrayAnalyzer(Scanner sc, int arr[]) {
    int choice;
    do {
        System.out.println();
        System.out.println("===== ARRAY ANALYZER =====");
        System.out.println("1. Count Occurrence");
        System.out.println("2. Contains");
        System.out.println("3. Find Index");
        System.out.println("4. Sum Array");
        System.out.println("5. Back to Main Menu");
        System.out.print("Enter your choice: ");
        choice = sc.nextInt();
        System.out.println();
        switch (choice) {
            case 1:
                System.out.print("Enter target: ");
                int target = sc.nextInt();
                int result = countOccurrence(arr, target);
                System.out.println("Occurrence: " + result);
                break;
            case 2:
                System.out.print("Enter target: ");
                int target2 = sc.nextInt();
                boolean result2 = contains(arr, target2);
                System.out.println("Contains: " + result2);
                break;
            case 3:
                System.out.print("Enter target: ");
                int target3 = sc.nextInt();
                int result3 = index(arr, target3);
                System.out.println("Index: " + result3);
                break;
            case 4:
                int result4 = sumOfArray(arr);
                System.out.println("Sum of array: " + result4);
                break;
            case 5:
                System.out.println("Returning to Main Menu...");
                break;
            default:
                System.out.println("Invalid choice!");
        }
    } while (choice != 5);
}

// ===== Digit analyzer =====
static int sumDigits(int num) {
    int sum = 0;
    while (num > 0) {
        int digit = num % 10;
        sum = sum + digit;
        num = num / 10;
    }
    return sum;
}

static int reverse(int num) {
    int rev = 0;
    while (num > 0) {
        int digit = num % 10;
        rev = rev * 10 + digit;
        num = num / 10;
    }
    return rev;
}

static boolean isPalindrome(int num) {
    int original = num;
    int rev = reverse(num);
    return original == rev;
}

static int factorial(int num) {
    int fact = 1;
    for (int i = 1; i <= num; i++) {
        fact = fact * i;
    }
    return fact;
}

static int largestDigit(int num) {
    int largest = 0;
    while (num > 0) {
        int digit = num % 10;
        if (digit > largest) {
            largest = digit;
        }
    }
}
```

```

        }
        num = num / 10;
    }
    return largest;
}
static int smallestDigit(int num) {
    int smallest = 9;
    while (num > 0) {
        int digit = num % 10;
        if (digit < smallest) {
            smallest = digit;
        }
        num = num / 10;
    }
    return smallest;
}
static int countEvenDigits(int num) {
    int count = 0;
    while (num > 0) {
        int digit = num % 10;
        if (digit % 2 == 0) {
            count++;
        }
        num = num / 10;
    }
    return count;
}
static int sumEvenDigits(int num) {
    int sum = 0;
    while (num > 0) {
        int digit = num % 10;
        if (digit % 2 == 0) {
            sum = sum + digit;
        }
        num = num / 10;
    }
    return sum;
}
static void digitAnalyzer(Scanner sc) {
    int choice;
    do {
        System.out.println();
        System.out.println("===== DIGIT ANALYZER =====");
        System.out.println("1. Sum Digits");
        System.out.println("2. Reverse Number");
        System.out.println("3. Palindrome");
        System.out.println("4. Factorial");
        System.out.println("5. Largest Digit");
        System.out.println("6. Smallest Digit");
        System.out.println("7. Count Even Digits");
        System.out.println("8. Sum Even Digits");
        System.out.println("9. Back to Main Menu");
        System.out.print("Enter your choice: ");
        choice = sc.nextInt();
        if (choice >= 1 && choice <= 8) {
            System.out.print("Enter number: ");
            int num = sc.nextInt();
            switch (choice) {
                case 1:
                    System.out.println("Sum: " + sumDigits(num));
                    break;
                case 2:
                    System.out.println("Reverse: " + reverse(num));
                    break;
                case 3:
                    System.out.println("Palindrome: " + isPalindrome(num));
                    break;
                case 4:
                    System.out.println("Factorial: " + factorial(num));
                    break;
                case 5:
                    System.out.println("Largest digit: " + largestDigit(num));
                    break;
                case 6:
                    System.out.println("Smallest digit: " + smallestDigit(num));
                    break;
                case 7:
                    System.out.println("Even digit count: " + countEvenDigits(num));
                    break;
                case 8:
                    System.out.println("Even digit sum: " + sumEvenDigits(num));
                    break;
            }
        }
    } while (choice != 9);
}

```

```

        }
    } else if (choice == 9) {
        System.out.println("Returning to Main Menu...");
    } else {
        System.out.println("Invalid choice!");
    }
} while (choice != 9);
}
// ===== MAIN =====
public static void main(String args[]) {
    int num[] = {
        10, 20, 20, 10, 35,
        20, 43, 64, 23
    };
    Scanner sc = new Scanner(System.in);
    int choice;
    do {
        System.out.println();
        System.out.println("===== JAVA ANALYZER =====");
        System.out.println("1. Array Analyzer");
        System.out.println("2. Digit Analyzer");
        System.out.println("3. Exit");
        System.out.print("Enter your choice: ");
        choice = sc.nextInt();
        System.out.println();
        switch (choice) {
            case 1:
                arrayAnalyzer(sc, num);
                break;
            case 2:
                digitAnalyzer(sc);
                break;
            case 3:
                System.out.println("Exiting Java Analyzer...");
                break;
            default:
                System.out.println("Invalid choice!");
        }
    } while (choice != 3);
    sc.close();
}
}

```

day19.java - saved 18 Sep 2026

```

// public class day19 {
//     public static void main(String args[]) {
//         String str = "shubham";
//         for (int i = 0; i < str.length(); i++) {
//             System.out.println(str.charAt(i));
//         }
//     }
// }
// s
// h
// u
// b
// h
// a
// m
// public class day19 {
//     static int countVowels(String str) {
//         int count = 0;
//         String vowels = "aeiou";
//         for (int i = 0; i < str.length(); i++) {
//             char ch = str.charAt(i);
//             if (vowels.indexOf(ch) != -1) {
//                 count++;
//             }
//         }
//         return count;
//     }
//     public static void main(String args[]) {
//         int res = countVowels("shubham");
//         System.out.println("Vowels : " + res);
//     }
// }
// Vowels : 2
// public class day19 {
//     static int countDigits(String str) {
//         int count = 0;

```

```
//      for (int i = 0; i < str.length(); i++) {
//          char ch = str.charAt(i);
//          if (ch >= '0' && ch <= '9') {
//              count++;
//          }
//      }
//      return count;
//  }
//  public static void main(String args[]) {
//      int res = countDigits("shu3833wdje193nn391");
//      System.out.println("Digits : " + res);
//  }
// }
// Digits : 10
// public class day19 {
//     static int countSpaces(String str) {
//         int count = 0;
//         for (int i = 0; i < str.length(); i++) {
//             char ch = str.charAt(i);
//             if (ch == ' ') {
//                 count++;
//             }
//         }
//         return count;
//     }
//     public static void main(String args[]) {
//         {
//             int res = countSpaces("Hello bro how are you");
//             System.out.println("Spaces : " + res);
//         }
//     }
// }
// Spaces : 4
// public class day19 {
//     static String reverseString(String str) {
//         String reverse = "";
//         for (int i = str.length() - 1; i >= 0; i--) {
//             char ch = str.charAt(i);
//             reverse = reverse + ch;
//         }
//         return reverse;
//     }
//     public static void main(String args[]) {
//         String res = reverseString("shubham24");
//         System.out.println("String reverse : " + res);
//     }
// }
// String reverse : 42mahbuhs
public class day19 {
    static String reverseString(String str) {
        String reverse = "";
        for (int i = str.length() - 1; i >= 0; i--) {
            char ch = str.charAt(i);
            reverse = reverse + ch;
        }
        return reverse;
    }
    static boolean palindromeString(String str) {
        String reverse = reverseString(str);
        return str.equals(reverse);
    }
    public static void main(String args[]) {
        boolean res = palindromeString("madam");
        System.out.println("Palindrome : " + res);
    }
}
// Palindrome : true
```

day20.java - saved 19 Sep 2026

```
// public class day20 {
//     static boolean isPalindrome(String str) {
//         int j = str.length() - 1;
//         for (int i = 0; i < j; i++) {
//             if (str.charAt(i) != str.charAt(j)) {
//                 return true;
//             }
//         }
//         return false;
//     }
//     public static void main(String args[]) {
```

```

//      boolean res = isPalindrome("naman");
//      System.out.println("Palindrome : " + res);
//    }
// }
// public class day20 {
//     static boolean isPalindrome(String str) {
//         int i = 0;
//         int j = str.length() - 1;
//         while (i < j) {
//             if (str.charAt(i) != str.charAt(j)) {
//                 return false;
//             }
//             i++;
//             j--;
//         }
//         return true;
//     }
//     public static void main(String args[]) {
//         boolean res = isPalindrome("naman");
//         System.out.println("Palindrome : " + res);
//     }
// }
// Palindrome : true
// public class day20 {
//     static boolean isPalindromeIgnoreCase(String str) {
//         int i = 0;
//         int j = str.length() - 1;
//         str = str.toLowerCase();
//         while (i < j) {
//             if (str.charAt(i) != str.charAt(j)) {
//                 return false;
//             }
//             i++;
//             j--;
//         }
//         return true;
//     }
//     public static void main(String args[]) {
//         boolean res = isPalindromeIgnoreCase("MAdam");
//         System.out.println("Palindrome : " + res);
//     }
// }
// Palindrome : true
// public class day20 {
//     static int countChar(String str, char target) {
//         int count = 0;
//         for (int i = 0; i < str.length(); i++) {
//             if (str.charAt(i) == target) {
//                 count++;
//             }
//         }
//         return count;
//     }
//     public static void main(String args[]) {
//         int res = countChar("programming", 'g');
//         System.out.println("occured : " + res);
//     }
// }
// occured :2
public class day20 {
    static int countChar(String str, char target) {
        int count = 0;
        for (int i = 0; i < str.length(); i++) {
            if (str.charAt(i) == target) {
                count++;
            }
        }
        return count;
    }
    static void charFrequency(String str) {
        for (int i = 0; i < str.length(); i++) {
            if (alreadySeen(str, i)) {
                continue;
            }
            int frequency = countChar(str, str.charAt(i));
            System.out.println(str.charAt(i) + " > " + frequency);
        }
    }
    static boolean alreadySeen(String str, int index) {
        for (int i = 0; i < index; i++) {
            if (str.charAt(i) == str.charAt(index)) {
                return true;
            }
        }
    }
}

```

```

    }
    return false;
}
public static void main(String args[]) {
    charFrequency("programming");
}
}

```

day21.java - saved 21 Sep 2026

```

import java.util.Scanner;
public class day21 {
    static int countChar(String str, char target) {
        int count = 0;
        for (int i = 0; i < str.length(); i++) {
            if (str.charAt(i) == target) {
                count++;
            }
        }
        return count;
    }
    static int countVowels(String str) {
        int count = 0;
        String str1 = "aeiou";
        for (int i = 0; i < str.length(); i++) {
            char ch = str.charAt(i);
            if (str1.indexOf(ch) != -1) {
                count++;
            }
        }
        return count;
    }
    static int countDigits(String str) {
        int count = 0;
        for (int i = 0; i < str.length(); i++) {
            char ch = str.charAt(i);
            if (ch >= '0' && ch <= '9') {
                count++;
            }
        }
        return count;
    }
    static int countSpaces(String str) {
        int count = 0;
        for (int i = 0; i < str.length(); i++) {
            char ch = str.charAt(i);
            if (ch == ' ') {
                count++;
            }
        }
        return count;
    }
    static void charFrequency(String str) {
        for (int i = 0; i < str.length(); i++) {
            if (alreadySeen(str, i)) {
                continue;
            }
            int frequency = countChar(str, str.charAt(i));
            System.out.println(str.charAt(i) + " > " + frequency);
        }
    }
    static boolean alreadySeen(String str, int index) {
        for (int i = 0; i < index; i++) {
            if (str.charAt(i) == str.charAt(index)) {
                return true;
            }
        }
        return false;
    }
    static void stringAnalyzer(Scanner sc) {
        int choices;
        sc.nextLine();
        System.out.print("Enter a String : ");
        String str2 = sc.nextLine();
        do {
            System.out.println("1. Count Character");
            System.out.println("2. Count Vowels");
            System.out.println("3. Count digits");
            System.out.println("4. Count Spaces");
            System.out.println("5. Count Frequency");
        }
    }
}

```

```

        System.out.println("6. Exit");
        System.out.println();
        System.out.print("Enter your choices : ");
        choices = sc.nextInt();
        if (choices >= 1 && choices <= 5) {
            switch (choices) {
                case 1:
                    System.out.print("Enter target character : ");
                    char target = sc.next().charAt(0);
                    int res = countChar(str2, target);
                    System.out.println("Characters : " + res);
                    System.out.println();
                    break;
                case 2:
                    int res2 = countVowels(str2);
                    System.out.println("Vowels : " + res2);
                    System.out.println();
                    break;
                case 3:
                    int res3 = countDigits(str2);
                    System.out.println("Digits : " + res3);
                    System.out.println();
                    break;
                case 4:
                    int res4 = countSpaces(str2);
                    System.out.println("Spaces : " + res4);
                    System.out.println();
                    break;
                case 5:
                    charFrequency(str2);
                    break;
            }
        } else if (choices == 6) {
            System.out.println("Returning to main menu...");
            System.out.println();
        } else {
            System.out.println("Invalid input");
        }
    } while (choices != 6);
}

public static void main(String args[]) {
    System.out.println("=====String Analyzer=====");
    Scanner sc = new Scanner(System.in);
    int choices1;
    do {
        System.out.println("1. String Analyzer");
        System.out.println("2. Exit");
        System.out.println();
        System.out.print("Enter a choice : ");
        choices1 = sc.nextInt();
        switch (choices1) {
            case 1:
                stringAnalyzer(sc);
                break;
            case 2:
                System.out.println("Exiting...");
                break;
        }
    } while (choices1 != 2);
}

}

// =====String Analyzer=====
// 1. String Analyzer
// 2. Exit
// Enter a choice : 1
// Enter a String : shubham
// 1. Count Character
// 2. Count Vowels
// 3. Count digits
// 4. Count Spaces
// 5. Count Frequency
// 6. Exit
// Enter your choices :
// public class day21 {
//     static void febonacchi(int n) {
//         int first = 0;
//         int second = 1;
//         for (int i = 0; i < n; i++) {
//             System.out.println(first + " ");
//             int next = first + second;
//             first = second;
//             second = next;
//         }
//     }
// }

```

```
//      }
//    }
//    public static void main(String args[]) {
//        int n = 10;
//        febonacchi(n);
//    }
// }
```

day22.java - saved 23 Sep 2026

```
// import java.util.Scanner;
// public class day22 {
//     static int count = 0;
//     static void addStudent(Scanner sc, String name[], int marks[]) {
//         if (count < 5) {
//             System.out.println("Enter student name: ");
//             String name1 = sc.nextLine();
//             System.out.println("Enter student marks: ");
//             int marks1 = sc.nextInt();
//             sc.nextLine();
//             name[count] = name1;
//             marks[count] = marks1;
//             count++;
//         } else {
//             System.out.println("No space remaining");
//         }
//     }
//     static void viewStudents(String name[], int marks[]) {
//         if (count == 0) {
//             System.out.println("No students available");
//         } else {
//             for (int i = 0; i < count; i++) {
//                 System.out.println("Student " + (i + 1) + ": " + name[i] + " - " + marks[i]);
//             }
//         }
//     }
//     public static void main(String args[]) {
//         int choice;
//         Scanner sc = new Scanner(System.in);
//         String name[] = new String[5];
//         int marks[] = new int[5];
//         do {
//             System.out.println("=====Student Analyzer=====");
//             System.out.println();
//             System.out.println("1. Add student");
//             System.out.println("2. View Students");
//             System.out.println("3. Search students");
//             System.out.println("4. Student Statistics");
//             System.out.println("5. Exit");
//             System.out.println();
//             System.out.print("Enter your choice : ");
//             choice = sc.nextInt();
//             switch (choice) {
//                 case 1:
//                     addStudent(sc, name, marks);
//                     break;
//                 case 2:
//                     viewStudents(name, marks);
//                     break;
//             }
//         } while (choice != 5);
//     }
// }
// public class day23 {
//     public static int lowest(int arr[]) {
//         int lowest = arr[0];
//         for (int i = 0; i < arr.length; i++) {
//             if (arr[i] < lowest) {
//                 lowest = arr[i];
//             }
//         }
//         return lowest;
//     }
//     public static void main(String args[]) {
//         int marks[] = {31, 42, 38, 12, 48};
//         int res = lowest(marks);
//         System.out.println("Lowest : " + res);
//     }
// }
// -----DAY continued-----
// public class day22 {
```

```

//      static int largest(int arr[]) {
//          int largest = arr[0];
//          for (int i = 0; i < arr.length; i++) {
//              if (arr[i] > largest) {
//                  largest = arr[i];
//              }
//          }
//          return largest;
//      }
//      public static void main(String args[]) {
//          int num[] = {10, 34, 256, 94, 29, 432, 26};
//          int res = largest(num);
//          System.out.println("Largest : " + res);
//      }
// }
// Largest : 432
// -----IF FIRST OCCURENCE-----
// public class day22 {
//     static int findIndex(int arr[], int target) {
//         for (int i = 0; i < arr.length; i++) {
//             if (arr[i] == target) {
//                 return i;
//             }
//         }
//         return -1;
//     }
//     static void findAllIndex(int arr[], int target) {
//         boolean found = false;
//         for (int i = 0; i < arr.length; i++) {
//             if (arr[i] == target) {
//                 System.out.println(i);
//                 found = true;
//             }
//         }
//         if (found == false) {
//             System.out.println("Element not found");
//         }
//     }
//     public static void main(String args[]) {
//         int num[] = {19, 492, 22, 42, 22, 52, 22, 53};
//         findAllIndex(num, 22);
//         int res = findIndex(num, 22);
//         System.out.println("Index of target : " + res);
//     }
// }
public class day22 {
    static int countOccurence(int arr[], int target) {
        int count = 0;
        for (int i = 0; i < arr.length; i++) {
            if (arr[i] == target) {
                count++;
            }
        }
        return count;
    }
    public static void main(String args[]) {
        int num[] = {10, 32, 19, 31, 39, 31, 31, 92, 31};
        int res = countOccurence(num, 31);
        System.out.println("Occurence : " + res);
    }
}

```

day23.java - saved 26 Sep 2026

```

// public class day23 {
//     static int secondLargest(int arr[]) {
//         int target = arr[0];
//         int second = arr[0];
//         for (int i = 0; i < arr.length; i++) {
//             if (arr[i] > target) {
//                 second = target;
//                 target = arr[i];
//             } else if (arr[i] > second && arr[i] != target) {
//                 second = arr[i];
//             }
//         }
//         return second;
//     }
//     public static void main(String args[]) {
//         int num[] = {10, 32, 10, 32, 35, 35, 10};
//         int res = secondLargest(num);
//     }
// }

```

```

//      System.out.println("Second largest : " + res);
//    }
//  }
// ! Second largest : 32 -----> handles duplicate too
// public class day23 {
//     static int sumOfArray(int arr[]) {
//         int sum = 0;
//         for (int i = 0; i < arr.length; i++) {
//             sum = sum + arr[i];
//         }
//         return sum;
//     }
//     public static void main(String args[]) {
//         int num[] = {10, 20, 30, 40, 50};
//         int res = sumOfArray(num);
//         System.out.println("Sum of Array : " + res);
//     }
// }
// Sum of Array : 150
// public class day23 {
//     static double avgOfArray(int arr[]) {
//         int total = 0;
//         for (int i = 0; i < arr.length; i++) {
//             total += arr[i];
//         }
//         return (double) total / arr.length;
//     }
//     public static void main(String args[]) {
//         int num[] = {10, 20, 30, 40, 50};
//         double res = avgOfArray(num);
//         System.out.println("Average : " + res);
//     }
// }
// Average : 102.6
// public class day23 {
//     static int smallestInArray(int arr[]) {
//         int smallest = arr[0];
//         for (int i = 0; i < arr.length; i++) {
//             if (arr[i] < smallest) {
//                 smallest = arr[i];
//             }
//         }
//         return smallest;
//     }
//     public static void main(String args[]) {
//         int num[] = {10, 31, 3, 10, 3, 10};
//         int res = smallestInArray(num);
//         System.out.println("Smallest : " + res);
//     }
// }
// Smallest : 3
// public class day23 {
//     static int secondDistinct(int arr[]) {
//         int smallest = arr[0];
//         int second = Integer.MAX_VALUE;
//         boolean secondFound = false;
//         for (int i = 0; i < arr.length; i++) {
//             if (arr[i] < smallest) {
//                 second = smallest;
//                 smallest = arr[i];
//                 secondFound = true;
//             } else if (arr[i] < second && arr[i] != smallest) {
//                 second = arr[i];
//                 secondFound = true;
//             }
//         }
//         if (!secondFound) {
//             return -1;
//         }
//         return second;
//     }
//     public static void main(String args[]) {
//         int num[] = {3, 3, 3, 3, 3};
//         int res = secondDistinct(num);
//         System.out.println("Second distinct : " + res);
//     }
// }
// public class day23 {
//     static void moveZero(int arr[]) {
//         int pos = 0;
//         for (int i = 0; i < arr.length; i++) {
//             if (arr[i] != 0) {

```

```
//          arr[pos] = arr[i];
//          pos++;
//      }
//  }
//  for (int i = pos; i < arr.length; i++) {
//      arr[i] = 0;
//  }
//  }
//  public static void main(String args[]) {
//      int num[] = {0, 10, 32, 0, 21, 0, 23, 0, 9};
//      moveZero(num);
//      for (int i = 0; i < num.length; i++) {
//          System.out.print(num[i] + " ");
//      }
//  }
// }
```

day24.java - saved 25 Sep 2026

```
// public class day24 {
//     static void moveZero(int arr[]) {
//         int pos = 0;
//         for (int i = 0; i < arr.length; i++) {
//             if (arr[i] != 0) {
//                 arr[pos] = arr[i];
//                 pos++;
//             }
//         }
//         for (int i = pos; i < arr.length; i++) {
//             arr[i] = 0;
//         }
//     }
//     public static void main(String args[]) {
//         int num[] = {10, 0, 40, 0,
//             10, 4, 53, 0, 0, 3};
//         moveZero(num);
//         for (int i = 0; i < num.length; i++) {
//             System.out.print(num[i] + " ");
//         }
//     }
// }
// 10 40 10 4 53 3 0 0 0 0
// public class day24 {
//     static int duplicate(int arr[]) {
//         int pos = 1;
//         for (int i = 1; i < arr.length; i++) {
//             if (arr[i] != arr[i - 1]) {
//                 arr[pos] = arr[i];
//                 pos++;
//             }
//         }
//         return pos;
//     }
//     public static void main(String args[]) {
//         int num[] = {1, 1, 2, 2, 3, 4, 5, 5, 6, 6};
//         int res = duplicate(num);
//         for (int i = 0; i < res; i++) {
//             System.out.print(num[i] + " ");
//         }
//     }
// }
// public class day24 {
//     static void moveZero(int arr[]) {
//         int pos = 0;
//         for (int i = 0; i < arr.length; i++) {
//             if (arr[i] != 0) {
//                 arr[pos] = arr[i];
//                 pos++;
//             }
//         }
//         for (int i = pos; i < arr.length; i++) {
//             arr[i] = 0;
//         }
//     }
//     public static void main(String args[]) {
//         int num[] = {10, 0, 224, 0, 13, 0, 31, 34, 0};
//         moveZero(num);
//         for (int i = 0; i < num.length; i++) {
//             System.out.print(num[i] + " ");
//         }
//     }
// }
```

```
// }
// public class day24 {
//     static int duplicate(int arr[]) {
//         int pos = 1;
//         for (int i = 1; i < arr.length; i++) {
//             if (arr[i] != arr[i - 1]) {
//                 arr[pos] = arr[i];
//                 pos++;
//             }
//         }
//         return pos;
//     }
//     public static void main(String args[]) {
//         int num[] = {1, 1, 2, 2, 3, 3, 4, 4, 5, 5};
//         int res = duplicate(num);
//         for (int i = 0; i < res; i++) {
//             System.out.print(num[i] + " ");
//         }
//     }
// }
// ! 1 2 3 4 5 ---0P----->
// public class day24 {
//     static boolean sorted(int arr[]) {
//         for (int i = 1; i < arr.length; i++) {
//             if (arr[i - 1] > arr[i]) {
//                 return false;
//             }
//         }
//         return true;
//     }
//     public static void main(String args[]) {
//         int num[] = {1, 2, 3, 4, 4, 5};
//         System.out.println(sorted(num));
//     }
// }
// sorted : true
public class day24 {
    static String neither(int arr[]) {
        boolean ascending = true;
        boolean descending = true;
        for (int i = 1; i < arr.length; i++) {
            if (arr[i] > arr[i - 1]) {
                descending = false;
            } else if (arr[i - 1] > arr[i]) {
                ascending = false;
            }
        }
        if (ascending && !descending) {
            return "ascending";
        } else if (descending && !ascending) {
            return "descending";
        } else if (ascending && descending) {
            return "equal";
        }
        return "neither";
    }
    public static void main(String args[]) {
        int num[] = {50, 50, 50};
        System.out.println(neither(num));
    }
}
```

day25java - saved 28 Sep 2026

```
// public class day25 {
//     static int duplicate(int arr[]) {
//         int pos = 1;
//         for (int i = 1; i < arr.length; i++) {
//             if (arr[i] != arr[i - 1]) {
//                 arr[pos] = arr[i];
//                 pos++;
//             }
//         }
//         return pos;
//     }
//     public static void main(String args[]) {
//         int num[] = {1, 1, 2, 2, 3, 3, 4, 4, 5, 5};
//         int res = duplicate(num);
//         for (int i = 0; i < res; i++) {
//             System.out.print(num[i] + " ");
//         }
//     }
// }
```

```

//      }
// }
// 1 2 3 4 5
// public class day25 {
//     static void moveZero(int arr[]) {
//         int pos = 0;
//         for (int i = 0; i < arr.length; i++) {
//             if (arr[i] != 0) {
//                 arr[pos] = arr[i];
//                 pos++;
//             }
//         }
//         for (int i = pos; i < arr.length; i++) {
//             arr[i] = 0;
//         }
//     }
//     public static void main(String args[]) {
//         int num[] = {10, 2, 0, 28, 0, 3, 0, 19, 0, 27};
//         moveZero(num);
//         for (int i = 0; i < num.length; i++) {
//             System.out.print(num[i] + " ");
//         }
//     }
// }
// public class day25 {
//     static int secondDistinct(int arr[]) {
//         int largest = arr[0];
//         int second = Integer.MIN_VALUE;
//         boolean secondFound = false;
//         for (int i = 0; i < arr.length; i++) {
//             if (arr[i] > largest) {
//                 second = largest;
//                 largest = arr[i];
//                 secondFound = true;
//             } else if (arr[i] > second && arr[i] < largest) {
//                 second = arr[i];
//                 secondFound = true;
//             }
//         }
//         if (!secondFound) {
//             return -1;
//         }
//         return second;
//     }
//     public static void main(String args[]) {
//         int num[] = {10, 120, 10, 10, 10};
//         int res = secondDistinct(num);
//         System.out.println("Second Distinct largest : " + res);
//     }
// }
public class day25 {
    static int frequency(int arr[], int target) {
        int count = 0;
        for (int i = 0; i < arr.length; i++) {
            if (arr[i] == target) {
                count++;
            }
        }
        return count;
    }
    static int mostFrequent(int arr[]) {
        int mostFrequent = arr[0];
        int highestFrequency = frequency(arr, arr[0]);
        for (int i = 0; i < arr.length; i++) {
            int count = frequency(arr, arr[i]);
            if (count > highestFrequency) {
                mostFrequent = arr[i];
                highestFrequency = count;
            }
        }
        return mostFrequent;
    }
    public static void main(String args[]) {
        int num[] = {1, 12, 20, 332, 30, 10, 49, 20, 20};
        int res = mostFrequent(num);
        System.out.println("Most Frequent : " + res);
    }
}

```

day26.java - saved 29 Sep 2026

```

public class day26 {
    static int frequency(String str, char target) {
        int count = 0;
        for (int i = 0; i < str.length(); i++) {
            if (str.charAt(i) == target) {
                count++;
            }
        }
        return count;
    }

    static void charFrequency(String str) {
        for (int i = 0; i < str.length(); i++) {
            char ch = str.charAt(i);
            int count = frequency(str, ch);
            boolean alreadySeen = false;
            for (int j = 0; j < i; j++) {
                if (str.charAt(j) == ch) {
                    alreadySeen = true;
                }
            }
            if (alreadySeen) {
                continue;
            } else {
                int re = frequency(str, ch);
                System.out.println(ch + " > " + count);
            }
        }
    }

    public static void main(String args[]) {
        String str = "banana";
        charFrequency(str);
    }
}

```

day27.java - saved 29 Sep 2026

```

// public class day27 {
//     static void findIndex(int arr[], int target) {
//         for (int i = 0; i < arr.length; i++) {
//             if (arr[i] == target) {
//                 System.out.println(i);
//             }
//         }
//     }
// }

// public static void main(String args[]) {
//     int num[] = {10, 20, 30, 40, 50, 30, 30};
//     findIndex(num, 30);
// }
// }

public class day27 {
    static int countOccurrence(int arr[], int target) {
        int count = 0;
        for (int i = 0; i < arr.length; i++) {
            if (arr[i] == target) {
                count++;
            }
        }
        return count;
    }

    static int[] findAllIndex(int arr[], int target) {
        int pos = 0;
        int count = countOccurrence(arr, target);
        int ar[] = new int[count];
        for (int i = 0; i < arr.length; i++) {
            if (arr[i] == target) {
                ar[pos] = i;
                pos++;
            }
        }
        return ar;
    }

    public static void main(String args[]) {
        int num[] = {10, 20, 10, 20, 10, 20, 39, 43,};
        int res[] = findAllIndex(num, 20);
        for (int i = 0; i < res.length; i++) {
            System.out.println(res[i]);
        }
    }
}

```

```
// public class day27 {
//     static int lastIndex(int arr[], int target) {
//         int lastIndex = -1;
//         for (int i = 0; i < arr.length; i++) {
//             if (arr[i] == target) {
//                 lastIndex = i;
//             }
//         }
//         return lastIndex;
//     }
//     static int firstIndex(int arr[], int target) {
//         for (int i = 0; i < arr.length; i++) {
//             if (arr[i] == target) {
//                 return i;
//             }
//         }
//         return -1;
//     }
// }
// public static void main(String args[]) {
//     int num[] = {10, 20, 10, 40, 10};
//     int res1 = firstIndex(num, 10);
//     System.out.println("First index : " + res1);
//     int res = lastIndex(num, 10);
//     System.out.println("Last index : " + res);
// }
// }
// public class day27 {
//     static int[] firstLast(int arr[], int target) {
//         int firstIndex = -1;
//         int lastIndex = -1;
//         int ar[] = new int[2];
//         for (int i = 0; i < arr.length; i++) {
//             if (arr[i] == target) {
//                 if (firstIndex == -1) {
//                     firstIndex = i;
//                 }
//                 lastIndex = i;
//                 ar[0] = firstIndex;
//                 ar[1] = lastIndex;
//             }
//         }
//         return ar;
//     }
// }
// public static void main(String args[]) {
//     int num[] = {10, 20, 30, 40, 30, 30, 20, 30};
//     int res[] = firstLast(num, 30);
//     for (int i = 0; i < res.length; i++) {
//         System.out.println(res[i]);
//     }
// }
// }
```

day28.java - saved 30 Sep 2026

```
public class day28 {
    static int binarySearch(int arr[], int target) {
        int left = 0;
        int right = arr.length - 1;
        while (left <= right) {
            int middle = (left + right) / 2;
            if (target == arr[middle]) {
                return middle;
            } else if (target > arr[middle]) {
                left = middle + 1;
            } else {
                right = middle - 1;
            }
        }
        return -1;
    }
    public static void main(String args[]) {
        int num[] = {10, 20, 30, 40, 50, 60};
        int res = binarySearch(num, 50);
        System.out.println("Found At : " + res);
    }
}
```

day29.java - saved 30 Sep 2026

```

// public class day29 {
//     static int countOccurence(int arr[], int target) {
//         int count = 0;
//         for (int i = 0; i < arr.length; i++) {
//             if (arr[i] == target) {
//                 count++;
//             }
//         }
//         return count;
//     }
//     static int[] linear(int arr[], int target) {
//         int count = countOccurence(arr, target);
//         int ar[] = new int[count];
//         int pos = 0;
//         for (int i = 0; i < arr.length; i++) {
//             if (arr[i] == target) {
//                 ar[pos] = i;
//                 pos++;
//             }
//         }
//         return ar;
//     }
//     public static void main(String args[]) {
//         int num[] = {10, 20, 10, 39, 49, 20, 10};
//         int res[] = linear(num, 20);
//         for (int i = 0; i < res.length; i++) {
//             System.out.println(res[i]);
//         }
//     }
// }
// public class day29 {
//     static int findLastIndex(int arr[], int target) {
//         int lastIndex = -1;
//         for (int i = 0; i < arr.length; i++) {
//             if (arr[i] == target) {
//                 lastIndex = i;
//             }
//         }
//         return lastIndex;
//     }
//     public static void main(String args[]) {
//         int num[] = {10, 20, 10, 20, 30, 10};
//         int res = findLastIndex(num, 10);
//         System.out.println("Last Index : " + res);
//     }
// }
public class day29 {
    static void bubble(int arr[]) {
        int temp;
        for (int i = 0; i < arr.length; i++) {
            for (int j = 0; j < arr.length - 1 - i; j++) {
                if (arr[j] > arr[j + 1]) {
                    temp = arr[j + 1];
                    arr[j + 1] = arr[j];
                    arr[j] = temp;
                }
            }
        }
    }
    public static void main(String args[]) {
        int num[] = {39, 10, 42, 19, 53};
        bubble(num);
        for (int i = 0; i < num.length; i++) {
            System.out.print(num[i] + " ");
        }
    }
}

```